

**SISTEM PENGELOLAAN DATA PRAKTIKAN DENGAN
ALGORITMA MERGE SORT DAN SEQUENTIAL
SEARCH BERBASIS WEB**

SKRIPSI

Oleh:

**VISTA MELLYNA ATSFI
2209106096**



**FAKULTAS TEKNIK
UNIVERSITAS MULAWARMAN**

**SAMARINDA
2026**

SISTEM PENGELOLAAN DATA PRAKTIKAN DENGAN ALGORITMA MERGE SORT DAN SEQUENTIAL SEARCH BERBASIS WEB

SKRIPSI

Diajukan sebagai salah satu syarat untuk menyelesaikan pendidikan
pada Program Studi Strata 1 Informatika,
Fakultas Teknik, Universitas Mulawarman

Oleh:

**VISTA MELLYNA ATSFI
2209106096**



**FAKULTAS TEKNIK
UNIVERSITAS MULAWARMAN**

**SAMARINDA
2026**

PERNYATAAN KEASLIAN SKRIPSI

Saya menyatakan dengan sesungguhnya bahwa skripsi dengan judul:

SISTEM PENGELOLAAN DATA PRAKTIKAN DENGAN ALGORITMA MERGE SORT DAN SEQUENTIAL SEARCH BERBASIS WEB

yang dibuat sebagai salah satu syarat untuk menyelesaikan pendidikan pada Program Studi S1 Informatika, Fakultas Teknik, Universitas Mulawarman, sejauh yang saya ketahui bukan merupakan tiruan atau duplikasi dari skripsi yang sudah dipublikasikan dan atau pernah dipakai untuk mendapatkan gelar kesarjanaan di lingkungan Universitas Mulawarman maupun di Perguruan Tinggi atau instansi manapun, kecuali bagian yang sumber informasinya dicantumkan sebagaimana mestinya.

Samarinda, Tgl bln thn (akan diisi dengan tanggal sidang)

Materai

Vista Mellyna Atsfi

NIM. 2209106096

SISTEM PENGELOLAAN DATA PRAKTIKAN DENGAN ALGORITMA MERGE SORT DAN SEQUENTIAL SEARCH BERBASIS WEB

Oleh:

Vista Mellyna Atsfi
2209106096

Telah diujikan pada tgl bln thn dan dinyatakan telah
memenuhi syarat

Samarinda, Tanggal Bulan Tahun (akan diisi dengan tanggal sidang)

Disahkan oleh:

Pembimbing I,

Pembimbing II,

Anton Prafanto, S.Kom., M.T.
NIP. 199310222019031016

Ir. Novianti Puspitasari, S.Kom., M.Eng.
NIP. 198811062015042002

Mengetahui,
Dekan Fakultas Teknik
Universitas Mulawarman

Prof. Dr. Ir. H. Tamrin, S.T., M.T., IPU., ASEAN.Eng., APEC Eng.
NIP. 197002272000121001

Skripsi ini penulis persembahkan kepada kedua orangtua, saudara, dosen pembimbing, sahabat, serta semua pihak yang telah memberikan do'a, dukungan, dan semangat selama proses penyusunan skripsi ini hingga berhasil diselesaikan dengan baik.

Vista Mellyna Atsfi
NIM. 2209106096
Program Studi Informatika

Dosen Pembimbing
I. Anton Prafanto, S.Kom., M.T.
II. Ir. Novianti Puspitasari, S.Kom., M.Eng.

SISTEM PENGELOLAAN DATA PRAKTIKAN DENGAN ALGORITMA MERGE SORT DAN SEQUENTIAL SEARCH BERBASIS WEB

ABSTRAK

Pengelolaan data praktikan Fisika Dasar di Fakultas Teknik Universitas Mulawarman sebelumnya menggunakan beberapa file spreadsheet terpisah. Kondisi tersebut membuat proses pencarian, pengurutan, presensi, penilaian, dan rekapitulasi data kurang terpusat serta berisiko menimbulkan duplikasi dan kesalahan pengolahan. Penelitian ini bertujuan membangun sistem pengelolaan data praktikan berbasis web serta menerapkan *Merge Sort* dan *Sequential Search* pada proses pengurutan dan pencarian data. Penelitian menggunakan metode *Waterfall* yang mencakup analisis kebutuhan, perancangan, implementasi, pengujian, dan evaluasi. Sistem dikembangkan menggunakan Laravel dan MySQL. Data penelitian mencakup 538 data praktikan tahun akademik 2025 serta *dataset* sintetis berukuran 100, 500, 1.000, dan 5.000 entri. Pengujian fungsional menggunakan *Black Box Testing*, sedangkan pengujian kinerja mengukur waktu eksekusi, penggunaan memori, jumlah iterasi, dan jumlah perbandingan. Sistem menjalankan 50 pengujian utama dan 10 *warm-up* pada setiap kombinasi metode serta skenario, kemudian membersihkan *outlier* menggunakan *Interquartile Range*. Sistem menggunakan 3.071 data bersih dari 3.200 hasil pengujian utama setelah mengeluarkan 129 *outlier*. Hasil pengujian fungsional menunjukkan bahwa seluruh fitur utama berjalan sesuai rancangan. *Merge Sort* menghasilkan waktu 0,38 – 0,59 ms pada 100 data dan 27,66 – 65,07 ms pada 5.000 data, dengan penggunaan memori meningkat dari 6,63 KB menjadi sekitar 301,87 KB. *Sequential Search* menghasilkan waktu 0,01 – 0,06 ms pada 100 data dan 0,01 – 4,53 ms pada 5.000 data, dengan memori stabil pada 0,37 – 0,50 KB. Kedua algoritma menghasilkan waktu dan penggunaan memori lebih rendah daripada MySQL *ORDER BY* dan *WHERE/LIKE* pada lingkungan pengujian. Penelitian ini menerapkan kedua algoritma secara langsung dalam sistem praktikum dan membandingkan kinerjanya dengan *baseline* MySQL. Sistem yang dihasilkan mendukung pengelolaan data praktikum secara lebih terpusat, terstruktur, dan efisien.

Kata kunci: *Merge Sort, Sequential Search, MySQL, Praktikum, Black Box Testing*

Vista Mellyna Atsfi
NIM. 2209106096
Informatics Study Program

Supervisors
I. Anton Prafanto, S.Kom., M.T.
II. Ir. Novianti Puspitasari, S.Kom., M.Eng.

SISTEM PENGELOLAAN DATA PRAKTIKAN DENGAN ALGORITMA MERGE SORT DAN SEQUENTIAL SEARCH BERBASIS WEB

ABSTRACT

The management of Basic Physics practicum participant data at the Faculty of Engineering, Mulawarman University previously relied on several separate spreadsheet files. This condition made searching, sorting, attendance recording, assessment, and data recapitulation less centralized and increased the risk of duplication and processing errors. This study aims to develop a web-based practicum participant data management system and implement Merge Sort and Sequential Search for data sorting and searching. The Waterfall method was applied through requirements analysis, design, implementation, testing, and evaluation. The system was developed using Laravel and MySQL. The research data consisted of 538 practicum participant records from the 2025 academic year and synthetic datasets containing 100, 500, 1,000, and 5,000 entries. Functional testing used Black Box Testing, while performance testing measured execution time, memory usage, iteration count, and comparison count. The system performed 50 main runs and 10 warm-up runs for each combination of method and scenario, followed by outlier removal using the Interquartile Range. A total of 3,071 clean records from 3,200 main test results were used after removing 129 outliers. Functional testing showed that all main features operated according to the design. Merge Sort required 0.38 – 0.59 ms for 100 records and 27.66 – 65.07 ms for 5,000 records, while memory usage increased from 6.63 KB to approximately 301.87 KB. Sequential Search required 0.01 – 0.06 ms for 100 records and 0.01 – 4.53 ms for 5,000 records, with stable memory usage of 0.37 – 0.50 KB. Both algorithms produced lower execution time and memory usage than MySQL ORDER BY and WHERE/LIKE in the testing environment. This study directly integrates both algorithms into a practicum system and compares their performance with MySQL baselines. The developed system supports more centralized, structured, and efficient practicum data management.

Keywords: Merge Sort, Sequential Search, MySQL, Practicum, Black Box Testing

KATA PENGANTAR

Puji syukur kepada Allah Subhanahu wa Ta'ala, Tuhan Yang Maha Esa sehingga dapat menyelesaikan proposal skripsi dengan judul “**SISTEM PENGELOLAAN DATA PRAKTIKAN DENGAN ALGORITMA MERGE SORT DAN SEQUENTIAL SEARCH BERBASIS WEB**”. Proposal ini disusun sebagai salah satu tahapan dalam menyelesaikan skripsi pada Fakultas Teknik, Universitas Mulawarman.

Oleh karena itu, pada kesempatan ini penulis ingin mengucapkan terima kasih kepada semua pihak yang telah mendukung serta membantu selama proses penyusunan proposal skripsi, kepada:

1. Orangtua tercinta, Ibu Henny Novita dan Ayah Yoga Winarto, serta adik penulis, Vito Zilanayo atas do'a, kasih sayang, serta dukungannya.
2. Bapak Prof. Dr. Ir. H. Tamrin, S.T., M.T., IPU., ASEAN.Eng., APEC Eng., selaku Dekan Fakultas Teknik, Universitas Mulawarman.
3. Bapak Awang Harsa Kridalaksana, S.Kom., M.Kom., selaku Koordinator Program Studi Informatika.
4. Bapak Anton Prafanto, S.Kom., M.T., selaku Pembimbing I yang selalu memberikan arahan dan masukan terhadap penelitian ini.
5. Ibu Ir. Novianti Puspitasari, S.Kom., M.Eng., selaku Pembimbing II yang selalu memberikan arahan dan masukan terhadap penelitian ini.
6. Ibu Prof. Dr. Anindita Septiarini, S.T., M.Cs., selaku Penguji I atas saran dan masukan terhadap penelitian ini.
7. Ibu Andi Tejawati, M.Si., selaku Penguji II atas saran dan masukan terhadap penelitian ini.
8. Segenap Dosen Program Studi Informatika, yang telah memberikan ilmu pengetahuan selama mengikuti perkuliahan.
9. Rekan-rekan seperjuangan yang terus memberikan dukungan semangat demi terselesainya tugas ini.

Penulis menyadari bahwa proposal skripsi ini tidak luput dari berbagai kekurangan. Oleh karena itu, semua kritik dan saran yang bersifat memperbaiki demi kesempurnaan sangat diharapkan.

Samarinda, 2026 (akan diisi dengan tanggal sidang)

Penulis

DAFTAR ISI

	<i>halaman</i>
HALAMAN JUDUL	i
PERNYATAAN KEASLIAN SKRIPSI.....	ii
HALAMAN PENGESAHAN.....	iii
ABSTRAK	v
ABSTRACT	vi
KATA PENGANTAR.....	vii
DAFTAR ISI	viii
DAFTAR TABEL	xi
DAFTAR GAMBAR.....	xii
DAFTAR KODE	xiii
DAFTAR LAMPIRAN.....	xiv
DAFTAR ISTILAH	xvi
DAFTAR SINGKATAN.....	xviii

BAB I PENDAHULUAN..... 1

1.1	Latar Belakang.....	1
1.2	Rumusan Masalah.....	4
1.3	Batasan Masalah	4
1.4	Tujuan Penelitian	4
1.5	Manfaat Penelitian	5
1.6	Kontribusi Penelitian	5

BAB II TINJAUAN PUSTAKA 6

2.1	Penelitian Terkait.....	6
2.2	Teori Pendukung.....	10
2.2.1	Sistem Berbasis Web	10
2.2.2	Pengelolaan Data Praktikum	11
2.2.3	Algoritma <i>Sorting – Merge Sort</i>	11
2.2.4	Analisis Kompleksitas Waktu Algoritma <i>Merge Sort</i>	7
2.2.5	Algoritma <i>Searching – Sequential Search</i>	14
2.2.6	Analisis Kompleksitas Waktu Algoritma <i>Sequential Search</i>	15
2.2.7	Pengujian Kinerja Algoritma.....	16
2.2.8	<i>Database Indexing dan Query Optimization</i>	17
2.2.9	Model <i>Waterfall</i>	18
2.3	Perancangan Sistem	19
2.3.1	<i>Entity Relationship Diagram (ERD)</i>	19
2.3.2	<i>Flowchart</i>	20
2.3.3	<i>Data Flow Diagram (DFD)</i>	20
2.4	Perancangan <i>Website</i>	21

2.4.1	<i>Framework</i> Laravel	21
2.4.2	MySQL <i>Database</i>	23
2.4.3	Pengujian <i>Black Box</i>	23

BAB III METODOLOGI PENELITIAN 24

3.1	Tahapan Pelaksanaan Penelitian	24
3.2	Pengumpulan Data	26
3.3	Perancangan Data	26
3.4	Perancangan Proses	29
3.4.1	<i>Data Flow Diagram</i> (DFD)	30
3.4.2	<i>Flowchart</i> Sistem	34
3.4.3	<i>Flowchart Sequential Search</i>	36
3.4.4	<i>Flowchart Merge Sort</i>	36
3.5	Perancangan Tampilan	38
3.6	Perancangan Pengujian	44
3.6.1	Pengujian Sistem	45
3.6.2	Evaluasi Kinerja Algoritma	51
3.6.3	Teknik Pengukuran	57
3.6.4	Lingkungan Pengujian	58

BAB IV HASIL DAN PEMBAHASAN 59

4.1	Pengolahan Data	59
4.1.1	Data Praktikan pada Sistem	59
4.1.2	Data Sintetis untuk Pengujian Algoritma	60
4.1.3	Penyimpanan <i>Log</i> Hasil Pengujian	61
4.2	Implementasi Proses Algoritma	62
4.2.1	Implementasi <i>Merge Sort</i>	62
4.2.2	Implementasi <i>Sequential Search</i>	66
4.2.3	Implementasi <i>Baseline</i> MySQL <i>ORDER BY</i> dan <i>WHERE/LIKE</i>	68
4.2.4	Otomatisasi Pengujian Menggunakan <i>Artisan Command</i>	69
4.3	Penerapan Tampilan	72
4.3.1	Tampilan Halaman Awal dan Autentikasi	73
4.3.2	Tampilan Halaman Praktikan	76
4.3.3	Tampilan Halaman Asisten Praktikum	78
4.3.4	Tampilan Halaman Koordinator Praktikum	80
4.4	Hasil Pengujian	86
4.4.1	Hasil Pengujian Fungsional Sistem	86
4.4.2	Hasil Pengujian <i>Merge Sort</i>	92
4.4.3	Hasil Pengujian <i>Sequential Search</i>	93
4.4.4	Hasil Pengujian <i>Baseline</i> MySQL	94
4.4.5	Perbandingan Kinerja Algoritma dan MySQL	95
4.4.6	Statistik Deskriptif dan Pembersihan <i>Outlier</i> IQR	96
4.4.7	Visualisasi Hasil Pengujian	97

4.5	Pembahasan	99
4.5.1	Pembahasan Performa <i>Merge Sort</i>	100
4.5.2	Pembahasan Performa <i>Sequential Search</i>	100
4.5.3	Perbandingan Algoritma dengan <i>Baseline</i> MySQL	101
4.5.4	Analisis Waktu Eksekusi, Memori, dan Pembersihan Data	102
4.5.5	Implikasi terhadap Sistem Praktikum Berbasis Web	103

BAB V KESIMPULAN DAN SARAN..... 106

5.1	Kesimpulan.....	106
5.2	Saran	106

DAFTAR PUSTAKA..... 108

LAMPIRAN 113

DAFTAR TABEL

Tabel 2.1 Kompleksitas Waktu Algoritma <i>Merge Sort</i>	7
Tabel 2.2 Kompleksitas Waktu Algoritma <i>Sequential Search</i>	16
Tabel 2.3 Simbol ERD	19
Tabel 2.4 Simbol <i>Flowchart</i>	20
Tabel 3.1 Pengujian Masuk	45
Tabel 3.2 Pengujian Registrasi Praktikan	46
Tabel 3.3 Pengujian Fitur Praktikan	46
Tabel 3.4 Pengujian Fitur Asisten.....	47
Tabel 3.5 Pengujian Koordinator.....	48
Tabel 3.6 Pengujian Navigasi dan Tombol Umum.....	49
Tabel 3.7 Pengujian Algoritma <i>Sequential Search</i>	50
Tabel 3.8 Pengujian Algoritma <i>Merge Sort</i>	51
Tabel 3.9 Parameter Evaluasi Algoritma	52
Tabel 3.10 Skenario Pengujian <i>Merge Sort</i>	53
Tabel 3.11 Skenario Pengujian <i>Sequential Search</i>	54
Tabel 3.12 Prosedur Pengujian Algoritma <i>Merge Sort</i>	55
Tabel 3.13 Prosedur Pengujian Algoritma <i>Sequential Search</i>	56
Tabel 4.1 Prosedur Pengujian Algoritma <i>Sequential Search</i>	60
Tabel 4.2 Prosedur Pengujian Algoritma <i>Sequential Search</i>	61
Tabel 4.4 Contoh Input dan Output <i>Merge Sort</i>	65
Tabel 4.5 Contoh <i>Input</i> dan <i>Output Sequential Search</i>	67
Tabel 4.6 Skenario <i>Baseline MySQL</i>	69
Tabel 4.7 Hasil Eksekusi <i>Artisan Command</i> Pengujian Algoritma.....	72
Tabel 4.8 Pengujian Autentikasi dan Hak Akses.....	87
Tabel 4.9 Pengujian Fitur Berdasarkan Peran Pengguna.....	88
Tabel 4.10 Pengujian Pengelolaan Data Praktikum.....	89
Tabel 4.11 Pengujian Pencarian, Pengurutan, Penyaringan, dan Navigasi	90
Tabel 4.12 Pengujian Fungsional Algoritma	91
Tabel 4.13 Hasil Pengujian <i>Merge Sort</i>	92
Tabel 4.14 Hasil Pengujian <i>Sequential Search</i>	93
Tabel 4.15 Hasil Pengujian MySQL <i>ORDER BY</i>	94
Tabel 4.16 Hasil Pengujian MySQL <i>WHERE/LIKE</i>	94
Tabel 4.17 Perbandingan <i>Merge Sort</i> dan MySQL <i>ORDER BY</i>	95
Tabel 4.18 Perbandingan <i>Sequential Search</i> dan MySQL <i>WHERE/LIKE</i>	96
Tabel 4.19 Hasil Pembersihan <i>Outlier</i> Berdasarkan Ukuran <i>Dataset</i>	97

DAFTAR GAMBAR

	<i>Halaman</i>
Gambar 2.1 Model <i>Waterfall</i>	18
Gambar 3.1 Tahapan Pelaksanaan Penelitian	24
Gambar 3.2 ERD Sistem Praktikum Fisika Dasar	27
Gambar 3.3 DFD Level 0 Sistem Praktikum Fisika Dasar	30
Gambar 3.4 DFD Level 1 Sistem Praktikum Fisika Dasar	31
Gambar 3.5 DFD Level 2 pada Proses 1 Sistem Praktikum Fisika Dasar	32
Gambar 3.6 DFD Level 2 pada Proses 2 Sistem Praktikum Fisika Dasar	33
Gambar 3.7 DFD Level 2 pada Proses 3 Sistem Praktikum Fisika Dasar	34
Gambar 3.8 <i>Flowchart</i> Pendaftaran dan Masuk Sistem Praktikum Fisika Dasar	34
Gambar 3.9 <i>Flowchart</i> Sistem Praktikum Fisika Dasar	35
Gambar 3.10 <i>Flowchart Sequential Search</i>	36
Gambar 3.11 <i>Flowchart Merge Sort</i>	37
Gambar 3.12 <i>Wireframe</i> Halaman Awal Sistem: (a) halaman utama, (b) halaman masuk, dan (c) halaman registrasi praktikan	38
Gambar 3.13 <i>Wireframe</i> Halaman Praktikan: (a) halaman beranda, (b) halaman materi praktikum, dan (c) halaman biodata diri	39
Gambar 3.14 <i>Wireframe</i> Halaman Asisten: (a) halaman beranda, (b) halaman presensi praktikan, (c) halaman materi praktikum, (d) halaman penilaian, dan (e) halaman beri nilai	40
Gambar 3.15 <i>Wireframe</i> Halaman Koordinator: (a) halaman beranda, (b) halaman periode praktikum, (c) halaman materi praktikum, (d) halaman kategori penilaian, dan (e) halaman evaluasi algoritma	42
Gambar 3.16 <i>Wireframe</i> Pengelolaan Data Periode Praktikum: (a) halaman detail periode, (b) halaman asisten, (c) halaman data asisten, (d) halaman data praktikan, (e) halaman kelompok praktikum, dan (f) halaman detail kelompok praktikum	43
Gambar 3.17 <i>Wireframe</i> Pemantauan Hasil Praktikum: (a) halaman penilaian, (b) halaman presensi praktikan, dan (c) halaman nilai akhir praktikan	44
Gambar 4.1 Tampilan Beranda Awal Sistem	73
Gambar 4.2 Halaman Masuk ke Sistem	73
Gambar 4.3 Halaman Pendaftaran Praktikan	74
Gambar 4.4 Halaman Pedoman K3	74
Gambar 4.5 Halaman SOP Praktikum	75
Gambar 4.6 Halaman Atribut Praktikum	75
Gambar 4.7 Halaman Materi Praktikum Praktikan	76
Gambar 4.8 Halaman Materi Praktikum Praktikan	76
Gambar 4.9 Halaman Jadwal Praktikan	77
Gambar 4.10 Halaman Biodata Diri Praktikan	77
Gambar 4.11 Beranda Asisten Praktikum	78
Gambar 4.12 Halaman Presensi Praktikan	78
Gambar 4.13 Halaman Materi Praktikum Asisten	79

Gambar 4.14 Halaman Jadwal Praktikum Asisten	79
Gambar 4.15 Halaman Penilaian Praktikan	80
Gambar 4.16 Halaman Beranda Koordinator Praktikum.....	80
Gambar 4.17 Halaman Periode Praktikum	81
Gambar 4.18 Halaman Detail Periode Praktikum	81
Gambar 4.19 Halaman Materi Praktikum Koordinator	82
Gambar 4.20 Halaman Data Asisten.....	82
Gambar 4.21 Halaman Data Praktikan	83
Gambar 4.22 Halaman Kelompok Praktikum.....	83
Gambar 4.23 Halaman Jadwal Praktikum Koordinator.....	84
Gambar 4.24 Halaman Kategori Penilaian	84
Gambar 4.25 Halaman Presensi Praktikan	85
Gambar 4.26 Halaman Nilai Akhir Praktikan	85
Gambar 4.27 Halaman Evaluasi Algoritma	86
Gambar 4.28 Ringkasan Hasil Evaluasi Algoritma	97
Gambar 4.29 Perbandingan Waktu <i>Sorting</i>	98
Gambar 4.30 Perbandingan Waktu <i>Searching</i>	98
Gambar 4.31 Perbandingan Penggunaan Memori	99

DAFTAR KODE

	Halaman
<i>Source Code 2.1 Alur Merge Sort</i>	12
<i>Source Code 2.2 Alur Proses Merge.....</i>	12
<i>Source Code 2.3 Alur Sequential Search.....</i>	15
<i>Source Code 4.1 Pemanggilan Sequential Search dan Merge Sort</i>	63
<i>Source Code 4.2 Proses Pembagian Data pada Merge Sort</i>	63
<i>Source Code 4.3 Proses Penggabungan Data pada Merge Sort.....</i>	64
<i>Source Code 4.4 Implementasi Sequential Search pada Data Praktikan.....</i>	66
<i>Source Code 4.5 Pemetaan Metode Pengujian</i>	68
<i>Source Code 4.6 Penentuan Waktu Eksekusi Pengujian</i>	68
<i>Source Code 4.7 Perintah Artisan Command Pengujian Algoritma.....</i>	69
<i>Source Code 4.8 Pencatatan Hasil Pengujian dan Status Warm-up</i>	70
<i>Source Code 4.9 Pencatatan Hasil Pengujian Algoritma pada Tabel Log.....</i>	104

DAFTAR LAMPIRAN

	<i>Halaman</i>
Lampiran 1 Repositori Kode Sumber Sistem Praktikum Fisika Dasar.....	113
Lampiran 2 Hasil wawancara dengan Koordinator Praktikum Fisika Dasar.....	113
Lampiran 3 Surat balasan dari Lab. Layanan Dasar FT Unmul mengenai konfirmasi penerimaan survei dan penyerahan data penelitian	120

DAFTAR ISTILAH

<i>Istilah</i>	<i>Arti</i>
<i>Merge Sort</i>	Algoritma pengurutan <i>divide and conquer</i> yang stabil.
<i>Sequential Search</i>	Algoritma pencarian berurutan dari awal hingga akhir data.
<i>Array</i>	Kumpulan data yang disimpan secara berurutan dalam satu struktur.
<i>Subarray</i>	Pembagian data berdasarkan urutan indeks.
$O(n \log n)$	Kompleksitas waktu algoritma <i>Merge Sort</i> .
$O(1)$	Kompleksitas waktu terbaik algoritma <i>Sequential Search</i> .
$O(n)$	Kompleksitas waktu rata-rata dan terburuk algoritma <i>Sequential Search</i> .
n	Jumlah data atau ukuran <i>dataset</i> .
<i>Dataset</i>	Kumpulan data praktikan yang digunakan dalam pengujian.
<i>In-place</i>	Karakteristik algoritma yang bekerja tanpa membutuhkan memori tambahan besar.
<i>Divide and conquer</i>	Strategi algoritma dengan membagi masalah menjadi submasalah yang lebih kecil.
<i>Presensi</i>	Data kehadiran praktikan dalam kegiatan praktikum.
<i>Pretest</i>	Tes awal yang dilakukan sebelum praktikum.
<i>Posttest</i>	Tes akhir yang dilakukan setelah praktikum.
<i>Server lokal</i>	<i>Server</i> yang dijalankan pada komputer sendiri (<i>localhost</i>).
<i>Linear Search</i>	Metode pencarian berurutan dengan kompleksitas $O(n)$.

<i>Bubble Sort</i>	Algoritma pengurutan dengan pertukaran elemen berulang.
<i>Insertion Sort</i>	Algoritma pengurutan dengan menyisipkan elemen ke posisi yang tepat.
<i>Selection Sort</i>	Algoritma pengurutan dengan memilih elemen minimum secara iteratif.
<i>Best case</i>	Kondisi terbaik kinerja algoritma.
<i>Average case</i>	Kondisi rata-rata kinerja algoritma.
<i>Worst case</i>	Kondisi terburuk kinerja algoritma.
<i>Group-By-Aggregate</i>	Operasi pengelompokan dan agregasi data pada basis data.
<i>Sort-based</i>	Metode pemrosesan data berbasis pengurutan.
<i>Hash-based</i>	Metode pemrosesan data berbasis fungsi <i>hash</i> .
<i>Full table scan</i>	Proses pemindaian seluruh tabel basis data.
<i>Query optimization</i>	Teknik optimasi meningkatkan efisiensi eksekusi <i>query</i> .
<i>Database indexing</i>	Teknik pemberian indeks mempercepat pencarian data.
<i>B-tree Index</i>	Struktur indeks berbasis pohon seimbang pada DBMS.
<i>Black Box Testing</i>	Metode pengujian fungsional tanpa melihat kode internal.
<i>Ad-hoc</i>	Pencarian langsung sesuai kebutuhan pengguna saat itu tanpa persiapan data sebelumnya.
<i>Client-server</i>	Arsitektur sistem dengan pemisahan klien dan <i>server</i> .
<i>Real-time</i>	Kemampuan sistem memperbarui data secara langsung.
<i>Cache memori</i>	Penyimpanan sementara data di memori untuk akses cepat.
<i>Trade-off</i>	Kondisi ketika peningkatan suatu aspek menyebabkan pengorbanan pada aspek lain.

<i>Baseline</i>	Acuan pembandingan dalam performa sistem.
<i>Cache locality</i>	Karakteristik akses data yang memanfaatkan kedekatan lokasi data di memori sehingga meningkatkan efisiensi.
<i>Overhead</i>	Beban tambahan dalam proses komputasi.
<i>Low-cardinality</i>	Kondisi ketika suatu atribut atau kolom dalam basis data memiliki jumlah variasi nilai yang relatif sedikit dibandingkan jumlah total data.
<i>Quick Sort</i>	Algoritma pengurutan berbasis <i>divide and conquer</i> .
<i>Binary Search</i>	Algoritma pencarian pada data terurut.
<i>Spreadsheet</i>	Lembar kerja elektronik untuk mengelola data dalam bentuk tabel.
<i>Rendering</i>	Proses menghasilkan tampilan antarmuka pada <i>browser</i> .
<i>Warm-up</i>	Tahap awal sebelum pengukuran kinerja dilakukan.
<i>Measurement</i>	Proses pengukuran kinerja algoritma.
<i>Microbenchmark</i>	Pengujian untuk mengukur kinerja komponen secara spesifik.
<i>Outlier</i>	Data yang menyimpang jauh dari data lainnya.
<i>Standard deviation</i>	Ukuran penyebaran data dari nilai rata-rata.
<i>Rata-rata (mean)</i>	Nilai tengah hasil pengujian.
<i>Median</i>	Nilai tengah setelah data diurutkan.

DAFTAR SINGKATAN

<i>Singkatan</i>	<i>Arti</i>
<i>Unmul</i>	Universitas Mulawarman
<i>FT</i>	Fakultas Teknik
<i>NIM</i>	Nomor Induk Mahasiswa
<i>MySQL</i>	<i>My Structured Query Language</i>
<i>PHP</i>	<i>Hypertext Preprocessor</i>
<i>SQL</i>	<i>Structured Query Language</i>
<i>DFD</i>	<i>Data Flow Diagram</i>
<i>ERD</i>	<i>Entity Relationship Diagram</i>
<i>PK</i>	<i>Primary Key</i>
<i>PDF</i>	<i>Portable Document Format</i>
<i>SDLC</i>	<i>Software Development Life Cycle</i>
<i>UML</i>	<i>Unified Modeling Language</i>
<i>JDBC</i>	<i>Java Database Connectivity</i>
<i>GUI</i>	<i>Graphical User Interface</i>
<i>HTML</i>	<i>HyperText Markup Language</i>
<i>CSS</i>	<i>Cascading Style Sheets</i>

<i>MVC</i>	<i>Model–View–Controller</i>
<i>ORM</i>	<i>Object-Relational Mapping</i>
<i>UCD</i>	<i>Use Case Diagram</i>
<i>B-tree</i>	<i>Balanced Tree</i>
<i>XAMPP</i>	<i>Apache, MySQL/MariaDB, PHP, dan Perl</i>
<i>DBMS</i>	<i>Database Management System</i>
<i>IQR</i>	<i>Interquartile Range</i>

BAB I

PENDAHULUAN

1.1 Latar Belakang

Pelaksanaan praktikum merupakan kegiatan pembelajaran yang dirancang untuk memberikan pengalaman langsung kepada mahasiswa dalam menerapkan konsep teoretis melalui kegiatan eksperimen, observasi, dan analisis di laboratorium. Praktikum tidak hanya berfungsi sebagai sarana penguatan pemahaman akademik, tetapi juga sebagai media untuk melatih keterampilan teknis, ketelitian, serta kemampuan pemecahan masalah. Pelaksanaan praktikum membutuhkan sistem pengelolaan data yang terintegrasi dan terstruktur agar seluruh proses administratif dan akademik dapat berjalan secara efektif.

Pada praktikum Fisika Dasar di Fakultas Teknik Universitas Mulawarman, pengelolaan data menjadi aspek penting karena kegiatan ini melibatkan berbagai program studi, yaitu Teknik Elektro, Teknik Lingkungan, Teknik Industri, Teknik Geologi, Teknik Pertambangan, Teknik Sipil, dan Teknik Kimia. Praktikum Fisika Dasar terdiri atas Fisika Dasar 1 yang diikuti seluruh program studi tersebut serta Fisika Dasar 2 yang pelaksanaannya bergantung pada kebijakan dosen pengampu masing-masing program studi. Dalam operasionalnya, asisten praktikum bertanggung jawab terhadap pencatatan kehadiran, penilaian praktikum, serta penilaian laporan mahasiswa (praktikan) dari berbagai program studi, sehingga ketepatan pengelolaan data menjadi faktor penting dalam mendukung kelancaran administrasi praktikum. Seluruh proses tersebut masih dilakukan menggunakan *spreadsheet*, sehingga pengolahannya sangat bergantung pada ketelitian manual dan rentan kesalahan.

Pengelolaan data praktikan saat ini diawali melalui proses pendaftaran menggunakan *Google Form* yang disebar oleh ketua tingkat masing-masing program studi. Data awal yang dikumpulkan meliputi nama lengkap, NIM, *email*, dan nomor ponsel, kemudian diverifikasi kembali agar sesuai dengan data dosen pengampu sebelum digunakan dalam administrasi praktikum. Data penilaian dan presensi disimpan dalam *Google Sheet* secara terpisah berdasarkan program studi dan periode praktikum, sehingga pengelola harus menelusuri beberapa dokumen ketika membutuhkan informasi tertentu.

Pada satu periode praktikum tahun 2025, jumlah praktikan Fisika Dasar 1 mencapai 382 mahasiswa, sedangkan Fisika Dasar 2 sebanyak 156 mahasiswa. Volume data tersebut berdampak langsung pada meningkatnya beban kerja pengelolaan, terutama dalam proses pengecekan data seperti duplikasi NIM, sinkronisasi antar-*sheet*, serta pencarian manual menggunakan *Ctrl+F*. Meskipun jumlah praktikan tidak selalu meningkat setiap tahun, jumlah praktikan yang berubah-ubah mengharuskan sistem pengelolaan data mampu beradaptasi terhadap skala data praktikan yang bervariasi.

Fragmentasi data praktikan ke dalam berbagai *file* menyebabkan proses pencarian dan pengurutan data menjadi kurang efisien. Asisten sering mengalami kesulitan dalam menelusuri informasi seperti nama, NIM, nilai, maupun status kehadiran karena data tersebar di banyak dokumen dengan format yang tidak seragam. Selain itu, penggunaan *spreadsheet* secara terpisah berpotensi menimbulkan duplikasi data, ketidakkonsistenan informasi, serta kesalahan *input*, sehingga proses rekapitulasi nilai membutuhkan waktu dan ketelitian yang lebih tinggi.

Permasalahan utama yang muncul adalah meningkatnya beban waktu dan tenaga dalam proses pengolahan serta penelusuran data, meskipun *spreadsheet* menyediakan fitur pengurutan dan pencarian data, mekanisme tersebut tidak terintegrasi lintas *file* dan tidak berjalan secara otomatis pada seluruh data praktikan. Asisten harus melakukan penyesuaian dan pemeriksaan pada setiap dokumen secara terpisah sebelum melakukan pencarian atau penyusunan nilai. Kondisi ini menyebabkan proses pengolahan data menjadi lambat dan tidak efisien, terutama ketika jumlah praktikan bertambah pada periode tertentu. Untuk mengatasi permasalahan tersebut, diperlukan sebuah sistem berbasis web yang mampu mengelola data praktikum secara terpusat serta menyediakan mekanisme pengurutan dan pencarian data secara otomatis dan konsisten. Namun, penerapan sistem praktikum berbasis web sering kali belum disertai dengan evaluasi kinerja algoritma pengurutan dan pencarian yang digunakan, khususnya terhadap pengaruh ukuran dataset terhadap waktu eksekusi dan respons sistem. Oleh karena itu, diperlukan kajian lebih lanjut mengenai performa algoritma yang diterapkan dalam pengelolaan data praktikan berskala menengah.

Karakteristik pengelolaan data praktikan yang melibatkan ratusan hingga ribuan entri membutuhkan algoritma pengurutan yang efisien secara komputasi. Algoritma

Merge Sort dipilih karena memiliki kompleksitas rata-rata $O(n \log n)$ yang stabil dan konsisten pada berbagai kondisi data (terurut, acak, atau hampir terurut), serta bersifat *stable sort* karena prosesnya membagi data menjadi bagian-bagian kecil sebelum digabung kembali (Esau Taiwo et al., 2020). Sementara itu, *Sequential Search* dapat digunakan untuk melakukan pencarian data secara sederhana, tidak memerlukan data terurut terlebih dahulu atau kondisi awal tertentu, sehingga sesuai untuk diterapkan pada pencarian dinamis pada data yang baru di-input atau belum memiliki struktur tetap.

Penerapan algoritma *Merge Sort* dan *Sequential Search* memberikan dasar bagi sistem praktikum berbasis web. *Merge Sort* memastikan data terus berada dalam kondisi terurut sehingga memudahkan proses pemeriksaan dan penyusunan administrasi, sementara *Sequential Search* memberikan prosedur pencarian yang langsung, mudah dipahami, dan tanpa memerlukan penyesuaian struktur data yang rumit. Kombinasi ini memungkinkan proses penelusuran data praktikan, penyusunan nilai, serta rekapitulasi administrasi berlangsung lebih cepat dan tidak lagi bergantung pada *spreadsheet* yang rawan kesalahan input maupun ketidakkonsistenan format.

Sistem basis data seperti MySQL menyediakan mekanisme pengurutan dan pencarian data secara bawaan. Penelitian ini menerapkan algoritma *Merge Sort* dan *Sequential Search* pada lapisan aplikasi (*application layer*) untuk memproses data praktikan yang telah dimuat dari basis data. Pendekatan ini memungkinkan sistem mengatur proses pengurutan dan pencarian secara *real-time* pada data yang berada di memori aplikasi tanpa ketergantungan pada eksekusi *query database* secara berulang. Penerapan algoritma pada *application layer* juga memberikan kendali terhadap proses pengukuran waktu eksekusi, penggunaan memori, dan operasi internal algoritma.

Sistem pengelolaan data berbasis web diharapkan dapat memusatkan proses administrasi Praktikum Fisika Dasar yang sebelumnya menggunakan beberapa *file spreadsheet*. *Merge Sort* digunakan untuk mengurutkan data praktikan, sedangkan *Sequential Search* digunakan untuk mencari data berdasarkan NIM atau nama. Sistem juga menggunakan MySQL *ORDER BY* dan *WHERE/LIKE* sebagai *baseline* untuk mengevaluasi kinerja kedua algoritma. Berdasarkan kebutuhan tersebut, penelitian ini disusun dengan judul “**Sistem Pengelolaan Data Praktikan dengan Algoritma *Merge Sort* dan *Sequential Search* Berbasis Web**”.

1.2 Rumusan Masalah

Berdasarkan latar belakang penelitian, rumusan masalah dalam penelitian ini adalah “Bagaimana mekanisme penerapan algoritma *Merge Sort* dan *Sequential Search* pada sistem praktikum berbasis web, serta bagaimana perbandingan kinerjanya dengan mekanisme bawaan basis data MySQL (*ORDER BY* dan *WHERE/LIKE*) ditinjau dari waktu eksekusi, penggunaan memori, dan skalabilitas terhadap variasi jumlah data?”.

1.3 Batasan Masalah

Penelitian ini disusun berdasarkan data-data yang telah diperoleh. Guna menjaga fokus penelitian dari luasnya cakupan pembahasan yang dihadapi, maka ruang lingkup penelitian ini dibatasi pada:

1. Penelitian dilakukan pada sistem pengelolaan data praktikan Praktikum Fisika Dasar Fakultas Teknik Universitas Mulawarman tahun akademik 2025 semester ganjil dan genap.
2. Sistem memiliki tiga hak akses pengguna, yaitu praktikan, asisten, dan koordinator praktikum, dengan menu serta kewenangan yang berbeda.
3. Sistem mengelola data praktikan, asisten, periode praktikum, kelompok, materi, jadwal, presensi, kategori penilaian, dan nilai akhir praktikan.
4. Fitur sistem mencakup pendaftaran dan autentikasi akun, pengelolaan data praktikum, pencatatan presensi melalui *QR Code* atau *input* manual, pengelolaan materi dan jadwal, pengisian nilai, serta rekapitulasi nilai akhir.
5. Sistem menyediakan fitur pencarian, pengurutan, penyaringan data, pembatasan hak akses, serta pengunduhan hasil rekapitulasi untuk mendukung proses administrasi praktikum berbasis web.
6. Penelitian difokuskan pada pengembangan fungsi pengelolaan data praktikan dan pengujian performa algoritma dalam sistem berbasis web.

1.4 Tujuan Penelitian

Tujuan yang ingin dicapai dalam penelitian ini adalah untuk menerapkan algoritma *Merge Sort* dan *Sequential Search* pada sistem praktikum berbasis web, serta menganalisis dan membandingkan kinerjanya dengan mekanisme *ORDER BY* dan

WHERE/LIKE pada MySQL berdasarkan waktu eksekusi, penggunaan memori, dan skalabilitas data.

1.5 Manfaat Penelitian

Penelitian ini diharapkan dapat memberikan manfaat pada berbagai pihak, khususnya:

1. Penulis

Penelitian ini memberikan manfaat berupa peningkatan kemampuan dan wawasan dalam merancang, mengimplementasikan, serta mengevaluasi kinerja algoritma *Merge Sort* dan *Sequential Search* pada sistem berbasis web. Selain itu, penulis memperoleh pemahaman yang lebih mendalam mengenai pengaruh penerapan algoritma terhadap kinerja pengolahan data praktikan.

2. Mahasiswa

Penelitian ini dapat menjadi sumber pembelajaran mengenai penerapan algoritma pengurutan dan pencarian dalam sistem berbasis web. Mahasiswa dapat memahami pemanfaatan algoritma *Merge Sort* dan *Sequential Search* dalam mendukung efisiensi pengelolaan data, khususnya dalam konteks kegiatan praktikum, serta menjadi referensi khususnya bagi mahasiswa di bidang Informatika yang dapat membantu proses pembelajaran.

3. Tim Praktikum Fisika Dasar Fakultas Teknik Universitas Mulawarman

Hasil penelitian ini diharapkan dapat memberikan kontribusi dalam pengelolaan data praktikan pada sistem praktikum berbasis web, khususnya dalam mendukung proses pengurutan dan pencarian data. Sistem yang dikembangkan memanfaatkan penerapan algoritma pada lapisan aplikasi untuk meningkatkan efisiensi proses pengolahan data serta mendukung pengelolaan data nilai, presensi, dan administrasi praktikum secara lebih terstruktur.

1.6 Kontribusi Penelitian

Penelitian ini diharapkan dapat memberikan kontribusi yang relevan dalam bidang yang diteliti. Adapun kontribusi yang dihasilkan dalam penelitian ini dapat diuraikan sebagai berikut.

1. Kontribusi Implementatif

Penelitian ini menghasilkan sebuah sistem praktikum berbasis web yang mendukung pengelolaan data praktikan secara terpusat pada Praktikum Fisika Dasar Fakultas Teknik Universitas Mulawarman, sehingga proses pengurutan dan pencarian data dapat dilakukan secara lebih terstruktur dibandingkan dengan pengelolaan data menggunakan *spreadsheet* terpisah.

2. Kontribusi Metodologis

Penelitian ini menyajikan pendekatan evaluasi kinerja algoritma *Merge Sort* dan *Sequential Search* pada aplikasi web dengan mempertimbangkan perbedaan karakteristik pengolahan data di lapisan basis data dan lapisan aplikasi, khususnya pada sistem yang memerlukan pemrosesan data secara dinamis.

3. Kontribusi Teoritis

Penelitian ini memberikan gambaran mengenai pengaruh ukuran *dataset* terhadap kinerja algoritma pengurutan dan pencarian pada sistem praktikum berbasis web dengan arsitektur PHP–MySQL. Hasil implementasi sistem dan analisis kinerja algoritma dalam penelitian ini diharapkan dapat menjadi bahan rujukan awal bagi pengembangan dan penelitian lanjutan di bidang akademik, khususnya pada sistem pengelolaan data praktikum.

BAB II

TINJAUAN PUSTAKA

2.1 Penelitian Terkait

Kajian mengenai penelitian-penelitian terkait yang telah dilakukan sebelumnya diperlukan dalam rangka mendukung penelitian ini. Daftar penelitian terkait antara lain:

1. **Algoritma yang digunakan:** *Merge Sort* dan *Quick Sort*. **Temuan Penelitian:** Penelitian ini melakukan studi komparatif antara dua algoritma *Divide and Conquer*, yaitu *Merge Sort* dan *Quick Sort*, untuk menganalisis kinerja dalam berbagai ukuran data. Hasil penelitian menunjukkan bahwa *Quick Sort* unggul dalam kecepatan, sedangkan *Merge Sort* memiliki stabilitas lebih tinggi dengan kompleksitas waktu $O(n \log n)$. Studi ini menegaskan pentingnya pemilihan algoritma berdasarkan volume data yang diolah (Esau Taiwo et al., 2020).
2. **Algoritma yang digunakan:** *Selection Sort* dan *Sequential Search*. **Temuan Penelitian:** Penelitian ini merancang aplikasi penjualan berbasis web pada toko Mba Tik yang menggunakan *Selection Sort* untuk proses pengurutan dan *Sequential Search* untuk pencarian produk. Implementasi kedua algoritma tersebut meningkatkan efisiensi pengolahan data transaksi karena pencarian dan pengurutan dapat dilakukan dengan kompleksitas rendah tanpa bergantung pada fungsi bawaan. Hasil penelitian membuktikan bahwa penggunaan algoritma sederhana dapat meningkatkan kecepatan sistem pada skala data menengah (Saputra & Kusniyati, 2023).
3. **Algoritma yang digunakan:** *Merge Sort*. **Temuan Penelitian:** Penelitian ini melakukan analisis performa algoritma *Merge Sort* dalam pengolahan data mahasiswa untuk menentukan efisiensi dalam pengurutan berdasarkan nilai akademik. Penelitian menunjukkan bahwa *Merge Sort* memiliki stabilitas tinggi dengan waktu pemrosesan yang konsisten meskipun ukuran data meningkat. Hasilnya, *Merge Sort* dinilai sesuai diterapkan pada sistem berbasis pendidikan yang memerlukan pengurutan data besar secara *real-time* (Aditya et al., 2025).

4. **Algoritma yang digunakan:** *Sequential Search*. **Temuan Penelitian:** Penelitian ini mengembangkan aplikasi pencarian arsip pegawai berbasis web dengan menerapkan algoritma *Sequential Search* untuk menemukan *file* dokumen berdasarkan Nomor Induk Kependudukan (NIK) pegawai. Hasil penelitian menunjukkan bahwa metode ini bekerja efektif untuk jumlah data menengah karena mampu menelusuri seluruh elemen data secara linier tanpa memerlukan struktur data kompleks. Sistem ini juga mempermudah proses pencarian dokumen internal secara cepat dan akurat (Melati et al., 2022).
5. **Algoritma yang digunakan:** *Sequential Search*. **Temuan Penelitian:** Penelitian ini mengimplementasikan algoritma *Sequential Search* dalam aplikasi pencarian dokumen digital berbasis web untuk mempercepat proses penelusuran *file* e-dokumen. Penelitian menunjukkan bahwa algoritma ini mampu bekerja dengan baik pada data yang tidak terurut, serta lebih efisien dibanding metode pencarian manual karena tidak memerlukan struktur indeks tambahan. Implementasi ini mendukung efisiensi sistem arsip digital di lingkungan institusi (Diana & Putra, 2022).
6. **Algoritma yang digunakan:** *Merge Sort*. **Temuan Penelitian:** Penelitian ini membandingkan tiga algoritma pengurutan, yaitu *Insertion Sort*, *Selection Sort*, dan *Merge Sort*, dengan parameter waktu eksekusi serta penggunaan memori terhadap jumlah data yang berbeda. Hasil penelitian menunjukkan bahwa algoritma *Merge Sort* memiliki waktu eksekusi paling cepat dalam mengurutkan data besar—dengan rata-rata 108,59 ms untuk 3000 data—meskipun membutuhkan memori lebih besar dibanding algoritma lainnya (21,44 MB). Dalam skala data kecil, perbedaan ketiga algoritma masih sulit terlihat, tetapi pada dataset besar (>1000 data), perbedaan efisiensi menjadi signifikan. Penelitian ini menyimpulkan bahwa *Merge Sort* unggul dalam kecepatan pemrosesan untuk data besar, sementara penggunaan memorinya relatif lebih tinggi. Temuan tersebut memperkuat relevansi penggunaan *Merge Sort* dalam sistem berbasis web yang memerlukan pengurutan data skala besar secara cepat dan stabil (Anggreani et al., 2020).

7. **Algoritma yang digunakan:** *Sequential Search*. **Temuan Penelitian:** Penelitian ini mengimplementasikan *Sequential Search* pada aplikasi *e-commerce* penjualan mobil berbasis web. Hasil implementasi menunjukkan bahwa *Sequential Search* mudah diintegrasikan ke dalam antarmuka web, bekerja andal pada *dataset* menengah, dan memberikan pengalaman pencarian langsung tanpa pengindeksan tambahan. Implementasi ini menegaskan kelayakan *Sequential Search* sebagai metode pencarian sederhana untuk aplikasi web yang datanya dinamis (Reswan et al., 2024).
8. **Algoritma yang digunakan:** *Sequential Search*. **Temuan Penelitian:** Penelitian ini mengimplementasikan *Sequential Search* pada modul pencarian data cuti pegawai melaporkan peningkatan kecepatan akses informasi dan kemudahan pengembangan *backend* ketika *dataset* berukuran sedang. Hasil penelitian ini menunjukkan bahwa *Sequential Search* cocok untuk aplikasi web institusional yang melayani pencarian *ad-hoc* tanpa kebutuhan *indexing* kompleks. Temuan tersebut relevan untuk modul pencarian data praktikan berdasarkan nama, NIM, ataupun kelompok praktikan pada sistem praktikum berbasis web (Ridwan et al., 2025).
9. **Algoritma yang digunakan:** *Sequential Search*. **Temuan Penelitian:** Penelitian ini membuat aplikasi kamus istilah informatika berbasis web yang menggunakan algoritma *Sequential Search* untuk memaksimalkan proses pencarian. Hasilnya aplikasi menjadi lebih responsif dan memudahkan pengguna dalam mencari istilah informatika (Wiguna, 2020).
10. **Algoritma yang digunakan:** *Sequential Search*. **Temuan Penelitian:** Penelitian ini mengembangkan sistem berbasis web yang menggunakan algoritma pencarian linear untuk memantau harga komoditas pangan. Sistem tersebut menggantikan metode manual dan terbukti efisien dalam konteks web. Dalam sistem komoditas harga pangan berbasis web di Perusahaan Umum Daerah Pasar Kota Medan, penerapan *Sequential Search* menghasilkan waktu pencarian rata-rata yang rendah (sekitar puluhan milidetik) untuk *dataset* ratusan (Datin et al., 2024).
11. **Algoritma yang digunakan:** Optimasi indeks serta perancangan struktur basis data pada MySQL. **Temuan Penelitian:** Penelitian ini menganalisis dampak

penerapan berbagai strategi optimasi basis data, terutama penggunaan indeks dan penyesuaian struktur tabel, terhadap kinerja pengolahan data pada sistem manajemen basis data MySQL. Pengujian dilakukan melalui skenario pencarian dan seleksi data dengan variasi tingkat kompleksitas *query* dan ukuran *dataset*. Hasil penelitian menunjukkan bahwa penggunaan indeks, baik indeks tunggal maupun indeks gabungan, berkontribusi dalam meningkatkan efisiensi proses pencarian dengan mengurangi waktu respons sistem, khususnya pada volume data yang besar. Tanpa indeks yang memadai, sistem cenderung melakukan pemindaian seluruh tabel (*full table scan*) yang dapat meningkatkan latensi dan beban pemrosesan. Sebaliknya, melalui optimasi indeks, MySQL dapat memanfaatkan struktur data internal berbasis *B-tree* untuk mempercepat akses terhadap data. Temuan ini menunjukkan bahwa optimasi pada tingkat basis data dapat berperan sebagai acuan kinerja (*baseline*) yang penting, serta relevan untuk dibandingkan dengan pendekatan pencarian dan pengurutan data yang diterapkan pada lapisan aplikasi dalam pengembangan sistem berbasis web (Györödi et al., 2021).

Perbedaan dengan Penelitian Sebelumnya

Berdasarkan referensi yang terkait, maka didapatkan perbedaan penelitian yang terdahulu sebagai dasar penelitian ini. Penelitian sebelumnya telah menerapkan *Merge Sort* dan *Sequential Search* pada berbagai konteks, seperti sistem penjualan (Saputra & Kusniyati, 2023), arsip digital dan dokumen pegawai (Melati et al., 2022; Diana & Putra, 2022), *e-commerce* (Reswan et al., 2024), kamus informatika (Wiguna, 2020), serta pemantauan harga pangan (Datin et al., 2024). Penelitian lain juga membahas performa *Merge Sort* pada pengolahan data dan membandingkannya dengan algoritma pengurutan lain (Esau Taiwo et al., 2020; Anggreani et al., 2020; Aditya et al., 2025). Berbeda dengan penelitian terdahulu, penelitian ini menerapkan *Merge Sort* dan *Sequential Search* secara bersamaan pada lapisan aplikasi dalam sistem administrasi Praktikum Fisika Dasar Fakultas Teknik Universitas Mulawarman. Sistem menggunakan kedua algoritma tersebut untuk mengurutkan dan mencari kembali data yang telah dimuat ke memori aplikasi, sehingga proses pengolahan tidak selalu bergantung pada *query* berulang ke basis data. Pendekatan ini mendukung respons antarmuka dan menjaga konsistensi pengelolaan data

yang sebelumnya tersebar dalam beberapa *file*. Penelitian ini juga membandingkan kinerja *Merge Sort* dan *Sequential Search* dengan MySQL *ORDER BY* dan *WHERE/LIKE* sebagai *baseline* berdasarkan waktu eksekusi, penggunaan memori, serta skalabilitas terhadap variasi ukuran *dataset*. Perbedaan tersebut memberikan kontribusi praktis dalam menentukan strategi pengolahan data antara lapisan aplikasi dan basis data pada sistem administrasi akademik berbasis web.

2.2 Teori Pendukung

Berikut beberapa teori pendukung yang menjadi dasar dalam penelitian ini. Teori-teori ini digunakan untuk memperkuat landasan konseptual penelitian serta memastikan bahwa rancangan sistem yang dibuat memiliki dasar ilmiah yang relevan dan sesuai dengan kebutuhan pengelolaan data secara efektif.

2.2.1 Sistem Berbasis Web

Sistem berbasis web adalah sebuah aplikasi yang dibangun menggunakan teknologi internet sehingga pengguna dapat mengakses fitur dan layanan sistem melalui peramban tanpa perlu instalasi perangkat lunak khusus, serta memungkinkan pengelolaan data secara terpusat dan *real-time*. Implementasi sistem berbasis web dalam konteks pendidikan telah terbukti meningkatkan efisiensi dan efektivitas proses administrasi karena data dapat diakses dari berbagai lokasi, menyediakan informasi yang konsisten, serta mempercepat proses pencatatan dan pelaporan dibandingkan cara manual (Lontaan & Sinadia, 2024).

Penggunaan sistem berbasis web dalam lingkungan pendidikan dan administrasi akademik mampu meningkatkan efisiensi pengelolaan data, mengurangi kesalahan *input*, serta mempercepat proses pencarian (Bagus Sadewa et al., 2024). Dengan demikian, sistem berbasis web menjadi pendekatan yang sesuai untuk membangun aplikasi praktikum yang mengelola data praktikan. Sistem ini memungkinkan penyimpanan terpusat, standarisasi format data, serta kemudahan integrasi mekanisme pengurutan dan pencarian dari algoritma pengurutan dan pencarian, sehingga mendukung tujuan peningkatan efisiensi dan akurasi pengolahan data praktikum.

2.2.2 Pengelolaan Data Praktikum

Pengelolaan data praktikum merupakan bagian penting dari administrasi akademik yang mencakup pencatatan kehadiran, penilaian, serta evaluasi keterlibatan mahasiswa dalam kegiatan praktikum. Ketika masih dilakukan secara manual atau melalui *spreadsheet*, proses ini rentan terhadap kesalahan *input*, redudansi data, serta lambatnya pencarian dan rekapitulasi informasi. Penelitian perancangan sistem pengelolaan praktikum berbasis web menunjukkan bahwa penggunaan aplikasi web terpusat memungkinkan seluruh data praktikan dikelola dalam satu basis data yang terintegrasi, sehingga proses pencarian data, *update* nilai, serta monitoring kegiatan dapat dilakukan secara lebih cepat dan akurat (Gunawan et al., 2023). Selain itu, sistem berbasis web mampu menyediakan antarmuka yang mudah diakses oleh asisten dan pengelola untuk memantau data praktikan secara *real-time*, meningkatkan efisiensi kerja administrasi praktikum dan mengurangi beban kerja serta potensi kesalahan yang biasa terjadi pada pengelolaan manual.

2.2.3 Algoritma *Sorting* – *Merge Sort*

Merge Sort adalah algoritma pengurutan yang menerapkan pendekatan *divide and conquer* dengan membagi data menjadi bagian-bagian yang lebih kecil, kemudian digabung kembali dalam urutan yang benar (Esau Taiwo et al., 2020). Algoritma ini menunjukkan kinerja yang konsisten pada berbagai ukuran data karena kompleksitas waktu pada kasus terbaik, rata-rata, dan terburuk sama-sama berada pada $O(n \log n)$ yang berarti bersifat stabil karena tidak mengubah urutan elemen yang bernilai sama. *Merge Sort* juga memiliki performa yang baik pada ukuran *array* kecil maupun besar, sehingga cocok digunakan untuk mengurutkan data dalam jumlah besar secara efisien. Selain itu, *Merge Sort* memerlukan ruang memori tambahan sebesar $O(n)$ untuk membantu proses penggabungan data (Liu Peike, 2024). Penggunaan memori tambahan ini dapat dimanfaatkan dalam konteks sistem praktikum karena sebanding dengan jumlah data yang diurutkan dan memberikan keuntungan berupa hasil pengurutan yang stabil serta performa yang konsisten. Dengan karakteristik ini, *Merge Sort* sangat cocok untuk mengurutkan data praktikan seperti NIM, nama, dan nilai dalam sistem praktikum yang mengelola sejumlah besar praktikan dari berbagai program studi.

Algoritma *Merge Sort* dalam penelitian ini diimplementasikan secara sistematis untuk menggambarkan tahapan proses pengurutan data mulai dari pembagian *array* hingga tahap penggabungan kembali secara terurut. Penyajian *pseudocode* di bawah ini bertujuan untuk memberikan gambaran mengenai alur kerja algoritma secara terstruktur, sehingga memudahkan pemahaman terhadap mekanisme pembagian data (*divide*) dan proses penggabungan (*conquer*) dalam konteks pengolahan data praktikum.

1. *Pseudocode Merge Sort* (rekursif)

```
FUNCTION MergeSort (A)
  n ← length(A)

  IF n ≤ 1 THEN
    RETURN A
  END IF

  mid ← n DIV 2

  Left  ← A[0 ... mid - 1]
  Right ← A[mid ... n - 1]

  Left  ← MergeSort(Left)
  Right ← MergeSort(Right)

  RETURN Merge(Left, Right)
END FUNCTION
```

Source Code 2.1 Alur Merge Sort

2. *Pseudocode Proses Merge*

```
FUNCTION Merge(Left, Right)
  CREATE empty array Result
  i ← 0
  j ← 0

  WHILE i < length(Left) AND j < length(Right) DO
    IF Left[i] ≤ Right[j] THEN
      ADD Left[i] TO Result
      i ← i + 1
    ELSE
      ADD Right[j] TO Result
      j ← j + 1
    END IF
  END WHILE

  WHILE i < length(Left) DO
    ADD Left[i] TO Result
    i ← i + 1
  END WHILE
```

Source Code 2.2 Alur Proses Merge

```

WHILE j < length(Right) DO
    ADD Right[j] TO Result
    j ← j + 1
END WHILE

RETURN Result
END FUNCTION

```

Source Code 2.2 Alur Proses *Merge* (Lanjutan)

Pseudocode tersebut menunjukkan bahwa *Merge Sort* membagi *array* menjadi dua bagian secara rekursif hingga tersisa satu elemen, kemudian menggabungkannya kembali melalui proses perbandingan terurut. Struktur ini memastikan hasil pengurutan yang stabil dengan kompleksitas waktu $O(n \log n)$.

2.2.4 Analisis Kompleksitas Waktu Algoritma *Merge Sort*

Kompleksitas waktu algoritma *Merge Sort* ditentukan oleh proses pembagian *array* secara rekursif serta tahap penggabungan kembali *sub-array* yang telah terurut. Setiap proses pembagian menghasilkan dua bagian dengan ukuran relatif seimbang, sehingga kedalaman rekursi mengikuti pola logaritmik terhadap jumlah data. Setelah tahap pembagian selesai, proses *merge* dilakukan dengan membandingkan elemen dari masing-masing *sub-array* secara sistematis hingga terbentuk susunan data yang terurut. Struktur kerja ini menyebabkan jumlah operasi yang dilakukan tetap konsisten pada berbagai kondisi input data (Liu Peike, 2024).

Berbeda dengan beberapa algoritma pengurutan yang performanya sangat dipengaruhi oleh susunan data awal, *Merge Sort* mempertahankan kompleksitas waktu yang sama pada kondisi terbaik, rata-rata, maupun terburuk karena seluruh elemen tetap harus melalui proses pembagian dan penggabungan. Karakteristik tersebut menjadikan *Merge Sort* sebagai algoritma yang stabil dan dapat diprediksi, khususnya ketika digunakan pada sistem yang mengelola data dalam jumlah besar dan memerlukan konsistensi performa.

Tabel 2.1 Kompleksitas Waktu Algoritma *Merge Sort*

Kondisi	Kompleksitas Waktu	Karakteristik
<i>Best Case</i>	$O(n \log n)$	Pembagian array berlangsung seimbang dan proses merge berjalan optimal

Tabel 2.1 Kompleksitas Waktu Algoritma *Merge Sort* (Lanjutan)

Kondisi	Kompleksitas Waktu	Karakteristik
<i>Average Case</i>	$O(n \log n)$	Distribusi data umum tetap melalui seluruh tahapan <i>divide dan merge</i>
<i>Worst Case</i>	$O(n \log n)$	Seluruh elemen diproses tanpa dipengaruhi oleh susunan data awal

Kompleksitas waktu algoritma *Merge Sort* pada Tabel 2.1 menunjukkan kestabilan performa karena tidak mengalami degradasi kompleksitas meskipun ukuran data meningkat. Konsistensi ini menjadi keunggulan utama terutama dalam konteks sistem akademik yang memerlukan hasil pengurutan yang akurat dan berulang pada setiap periode pengolahan data. Meskipun demikian, *Merge Sort* membutuhkan ruang memori tambahan sebesar $O(n)$ untuk mendukung proses penggabungan data. Kebutuhan memori ini merupakan konsekuensi dari strategi *divide and conquer*, namun tetap sebanding dengan keuntungan berupa waktu eksekusi yang stabil. Oleh karena itu, penggunaan *Merge Sort* dalam sistem praktikum berbasis web dapat membantu menjaga keteraturan data praktikan sekaligus memastikan performa pengurutan tetap stabil ketika *volume* data bertambah.

2.2.5 Algoritma *Searching – Sequential Search*

Sequential Search atau pencarian linier merupakan algoritma pencarian yang bekerja dengan membandingkan elemen data satu per satu secara berurutan hingga data yang dicari ditemukan atau seluruh data telah diperiksa dengan kompleksitas waktu $O(n)$. Pada kondisi terburuk, yaitu ketika data yang dicari berada pada posisi terakhir, algoritma harus memeriksa seluruh data yang ada. Meskipun demikian, untuk skala data praktikum yang berkisar antara 500–1.000 entri, waktu pencarian yang dihasilkan masih berada dalam batas yang dapat dimanfaatkan. Algoritma ini sesuai diterapkan pada sistem dengan data yang bersifat dinamis karena tidak memerlukan kondisi awal berupa data terurut maupun struktur indeks tambahan. *Sequential Search* efektif digunakan pada aplikasi berbasis web yang membutuhkan proses pencarian sederhana dan fleksibel terhadap perubahan data (Wiguna, 2020). Dalam sistem praktikum berbasis web, pencarian data praktikan seperti NIM atau nama dilakukan pada data yang dapat berubah setiap periode praktikum, sehingga penggunaan *Sequential Search* mendukung proses pencarian yang

konsisten, mudah diimplementasikan, dan terintegrasi langsung dengan logika pengolahan data sistem.

Algoritma *Sequential Search* dalam penelitian ini diimplementasikan untuk menggambarkan alur pencarian data secara sistematis pada lapisan aplikasi. Penyajian *pseudocode* di bawah ini bertujuan memberikan representasi mengenai tahapan pencarian mulai dari inisialisasi indeks hingga proses perbandingan data, sehingga mekanisme penelusuran elemen dapat dipahami secara terstruktur dalam konteks pengolahan data praktikum.

```
FUNCTION SequentialSearch(Array, key)
  n ← length(Array)
  i ← 0

  WHILE i < n DO

    IF Array[i] = key THEN
      RETURN i          // data ditemukan pada indeks ke-i
    END IF

    i ← i + 1
  END WHILE

  RETURN -1             // data tidak ditemukan
END FUNCTION
```

Source Code 2.3 Alur Sequential Search

Pseudocode tersebut menunjukkan bahwa proses pencarian dilakukan dengan menelusuri setiap elemen *array* secara berurutan hingga data ditemukan atau seluruh elemen selesai diperiksa. Pendekatan ini memungkinkan sistem melakukan pencarian secara langsung tanpa bergantung pada struktur indeks, sehingga relevan untuk diterapkan pada *application-layer processing* yang bertujuan mengurangi ketergantungan terhadap *query* berulang ke *server* serta meningkatkan respons antarmuka pengguna. Dengan implementasi yang sederhana dan fleksibel, *Sequential Search* dapat menjadi alternatif untuk sistem praktikum berbasis web dengan *dataset* kecil hingga menengah yang bersifat dinamis.

2.2.6 Analisis Kompleksitas Waktu Algoritma *Sequential Search*

Analisis kompleksitas waktu *Sequential Search* didasarkan pada jumlah perbandingan elemen yang harus dilakukan hingga data target ditemukan atau seluruh

data selesai diperiksa. *Sequential Search* menelusuri setiap elemen secara linier sehingga waktu eksekusinya berbanding lurus dengan jumlah data yang diproses. Karakteristik ini menyebabkan performa *Sequential Search* sangat dipengaruhi oleh posisi elemen target di dalam array. Dalam skenario terbaik, pencarian dapat dihentikan segera setelah elemen pertama diperiksa. Namun pada kondisi lain, *Sequential Search* harus terus melakukan perbandingan hingga mencapai posisi tertentu atau sampai seluruh data ditelusuri. Meskipun memiliki kompleksitas linear, pendekatan ini tetap relevan untuk *dataset* kecil hingga menengah (Wiguna, 2020).

Tabel 2.2 Kompleksitas Waktu Algoritma *Sequential Search*

Kondisi	Kompleksitas Waktu	Karakteristik
<i>Best Case</i>	$O(1)$	Data ditemukan pada pemeriksaan pertama
<i>Average Case</i>	$O(n)$	Data berada pada posisi acak sehingga sebagian elemen harus diperiksa
<i>Worst Case</i>	$O(n)$	Data berada di posisi terakhir atau tidak ditemukan

Tabel 2.2 menunjukkan kompleksitas waktu algoritma *Sequential Search* dengan karakteristeiknya. *Sequential Search* memiliki keunggulan berupa mekanisme pencarian yang sederhana tanpa prasyarat data terurut maupun struktur indeks tambahan (Wiguna, 2020). Hal ini memungkinkan algoritma diimplementasikan langsung pada lapisan aplikasi untuk melakukan pencarian terhadap data yang sering diperbarui. Dalam konteks *application-layer processing*, penggunaan *Sequential Search* dapat membantu mengurangi ketergantungan terhadap permintaan *query* berulang ke *server*, terutama ketika data telah dimuat ke dalam memori aplikasi. Pendekatan ini berkontribusi pada peningkatan respons antarmuka pengguna karena proses pencarian dapat dilakukan secara langsung tanpa menunggu komunikasi ulang dengan basis data.

2.2.7 Pengujian Kinerja Algoritma

Pengujian kinerja algoritma pengurutan dan pencarian dilakukan melalui eksekusi berulang pada skenario yang sama agar hasil pengukuran lebih stabil dan objektif. Penelitian ini menjalankan 50 iterasi utama pada setiap kombinasi metode, ukuran *dataset*, dan skenario pengujian. Jumlah tersebut digunakan untuk memperoleh nilai

mean, *median*, dan *standard deviation* yang lebih konsisten serta mengurangi pengaruh fluktuasi antar-eksekusi. Pengulangan sekitar 50 kali dinilai memadai untuk menghasilkan data statistik yang lebih stabil dan meminimalkan perbedaan hasil akibat variasi awal pengujian (Plevris & Solorzano, 2022). Jumlah pengulangan yang terlalu sedikit dapat menghasilkan kesimpulan yang bias dan sulit direproduksi, sehingga 50 iterasi dipilih sebagai keseimbangan antara reliabilitas hasil dan efisiensi komputasi (Werner et al., 2024).

Sebelum pengujian utama, sistem menjalankan 10 iterasi *warm-up* yang tidak digunakan dalam perhitungan statistik. Tahap ini bertujuan menstabilkan kondisi awal sistem dan mengurangi pengaruh pemuatan data, optimisasi *runtime*, serta *caching* awal. Pengukuran utama dilakukan setelah kondisi sistem lebih stabil sehingga hasil lebih mewakili kondisi operasional (Traini et al., 2023). Penggunaan *warm-up* juga dapat mengurangi bias akibat fluktuasi awal proses eksekusi (Laaber et al., 2024). Pengujian juga menggunakan beberapa ukuran *dataset* agar perubahan waktu eksekusi, penggunaan memori, jumlah iterasi, dan jumlah perbandingan dapat diamati seiring bertambahnya jumlah data (Pujiono et al., 2025).

Data hasil pengujian dianalisis menggunakan statistik deskriptif berupa *mean*, *median*, dan *standard deviation*. Sistem menerapkan metode Interquartile Range (IQR) untuk mendeteksi nilai ekstrem berdasarkan batas $Q1 - 1,5 \times IQR$ dan $Q3 + 1,5 \times IQR$. Data yang berada di luar batas tersebut ditandai sebagai *outlier* dan tidak digunakan dalam perhitungan statistik akhir, sedangkan data mentah tetap disimpan sebagai bagian dari hasil pengujian (Magar et al., 2024). Data bersih kemudian digunakan untuk membandingkan kinerja *Merge Sort* dengan MySQL *ORDER BY* serta *Sequential Search* dengan MySQL *WHERE/LIKE*. Perbandingan dilakukan menggunakan ukuran *dataset* dan skenario yang sama agar perbedaan waktu eksekusi serta penggunaan memori setiap metode dapat dianalisis secara konsisten.

2.2.8 Database Indexing dan Query Optimization

Database indexing merupakan teknik optimasi yang digunakan untuk mempercepat proses pencarian data pada sistem manajemen basis data relasional seperti MySQL (Sahar, 2024). MySQL umumnya menerapkan struktur *B-Tree Index* untuk

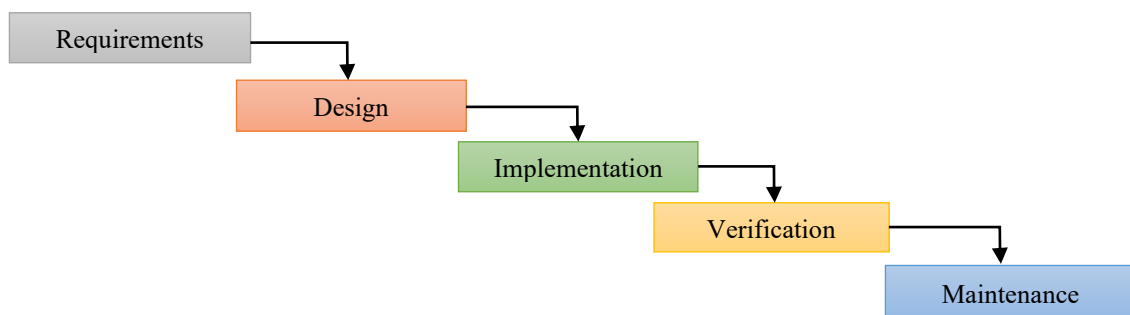
mengurangi pemindaian seluruh tabel sehingga menurunkan waktu respons, terutama saat volume data meningkat. Optimasi ini menjadi baseline performa karena menunjukkan pengaruh optimasi *database layer* terhadap efisiensi sistem (Györödi et al., 2021).

Meskipun *indexing* efektif pada *database layer*, optimasi performa tidak hanya bergantung pada sisi *server*. Pengelolaan data dan pemrosesan pada lapisan aplikasi dapat meningkatkan efisiensi saat data telah dimuat ke memori, serta mengurangi frekuensi *query* berulang (Pujiono et al., 2025). Kombinasi optimasi basis data dan pemrosesan *application layer* digunakan untuk menyeimbangkan kecepatan akses dengan penggunaan sumber daya. *Benchmarking* menunjukkan adanya *trade-off* antara strategi *server-side* dan *application-side*, sehingga pemilihan metode harus disesuaikan dengan karakteristik *dataset* serta pola akses sistem (Hadiewijaya & Wasito, 2024).

Pada penelitian ini, *Sequential Search* dan *Merge Sort* diterapkan pada lapisan aplikasi PHP sebagai pelengkap mekanisme pencarian dan pengurutan MySQL. Penerapan ini memberikan kontrol pemrosesan data di memori, mengurangi *query* berulang, meningkatkan responsivitas pada *dataset* kecil hingga menengah, serta mendukung evaluasi dan penyesuaian algoritma sesuai kebutuhan sistem praktikum.

2.2.9 Model Waterfall

Penelitian ini menggunakan model *Waterfall* dalam pengembangan sistem karena kebutuhan pengelolaan praktikum bersifat terstruktur, stabil, dan dapat direncanakan secara sistematis (Khan & Mahadik, 2022). Model ini menerapkan tahapan linier berurutan, yaitu analisis kebutuhan, perancangan, implementasi, pengujian, dan pemeliharaan, sebagaimana ditunjukkan pada Gambar 2.1. Pendekatan ini dipilih karena memberikan alur kerja yang jelas, serta memudahkan evaluasi sehingga sistem dapat digunakan secara optimal dan berkelanjutan pada periode berikutnya (Sallu et al., 2023).



Gambar 2.1 Model Waterfall

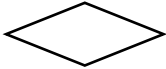
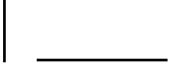
2.3 Perancangan Sistem

Perancangan sistem pada penelitian ini digunakan sebagai landasan dalam menggambarkan struktur dan alur kerja sistem sebelum tahap implementasi. Tahapan ini bertujuan untuk menyajikan model sistem yang bersifat visual dan terstruktur agar proses pengolahan, penyimpanan, serta distribusi data dapat dirancang secara sistematis dan sesuai dengan kebutuhan pengguna.

2.3.1 *Entity Relationship Diagram (ERD)*

Entity Relationship Diagram (ERD) merupakan model perancangan basis data yang digunakan untuk menggambarkan entitas, atribut, serta hubungan antarentitas dalam suatu sistem (Pulungan et al., 2022). Tabel 2.3 menjelaskan beberapa simbol ERD. ERD disusun dengan pendekatan *top-down* dengan mengidentifikasi data-data utama sebagai entitas dan memodelkan keterkaitan antar entitas tersebut dalam bentuk diagram konseptual. Penerapan ERD bertujuan untuk memberikan gambaran struktur data secara jelas sebelum sistem diimplementasikan, sehingga perancangan basis data menjadi lebih sistematis dan terorganisasi. Melalui ERD, perancang sistem dapat mengurangi terjadinya redundansi data, menjaga konsistensi serta integritas data, dan memudahkan pengembangan serta pemeliharaan basis data, sebagaimana diterapkan pada perancangan aplikasi berbasis web (Putri & Setiawan, 2020).

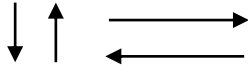
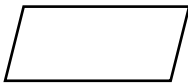
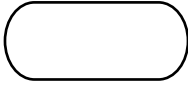
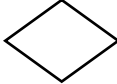
Tabel 2.3 Simbol ERD

Simbol	Nama	Keterangan
	Entitas	Jenis entitas dapat berupa suatu elemen lingkungan, sumber daya atau transaksi <i>field-field</i> yang dipergunakan dalam aplikasi program.
	Hubungan atau Relasi	Menunjukkan nama relasi antar satu entitas dengan entitas lainnya.
	Atribut	Atribut memberikan informasi lebih rinci tentang jenis entitas.
	Garis Relasi	Menunjukkan hubungan atau keterkaitan antar entitas.

2.3.2 Flowchart

Flowchart (*Flow Direction Symbols*) merupakan diagram yang memvisualisasikan urutan langkah dalam suatu proses secara terstruktur sehingga alurnya mudah diikuti. Setiap tahap digambarkan menggunakan simbol-simbol standar seperti *connecting line*, *processing*, *input and output*, *terminator*, *decision*, *connector*, serta *predefined process*.

Tabel 2.4 Simbol Flowchart

Simbol	Nama	Fungsi
	<i>Connecting Line</i>	Garis ini menghubungkan satu proses ke proses berikutnya.
	<i>Processing</i>	Simbol ini menunjukkan proses dalam sistem.
	<i>Input and Output</i>	Simbol ini merepresentasikan kegiatan <i>input</i> dan <i>output</i> data.
	<i>Terminator</i>	Simbol ini menandai titik awal maupun akhir suatu proses.
	<i>Decision</i>	Simbol keputusan ini memilih percabangan berdasarkan kondisi tertentu.
	<i>Connector</i>	Simbol ini menghubungkan bagian flowchart yang terpisah.
	<i>On-Page Connector</i>	Simbol ini menghubungkan bagian proses <i>Flowchart</i> yang berbeda.

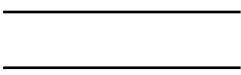
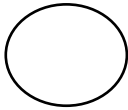
Penyajian visual ini membantu memastikan rangkaian proses tersusun dengan jelas sebelum diimplementasikan. Tabel 2.4 menunjukkan beberapa simbol yang umum digunakan dalam *flowchart*, beserta nama dan fungsinya (Smrti et al., 2023).

2.3.3 Data Flow Diagram (DFD)

Data Flow Diagram (DFD) merupakan alat pemodelan terstruktur yang digunakan untuk merepresentasikan aliran data dalam suatu sistem, mulai dari sumber eksternal, proses pengolahan, hingga penyimpanan data. DFD menyajikan gambaran visual mengenai bagaimana data mengalir dan ditransformasikan antar proses, sehingga

membantu analis sistem dalam memahami mekanisme kerja sistem secara menyeluruh. Struktur hierarkis pada DFD memungkinkan sistem diuraikan ke dalam beberapa tingkat abstraksi, yang memudahkan identifikasi hubungan fungsional antar proses serta proses kritis dalam pengolahan data. Selain itu, penggunaan DFD terbukti efektif dalam tahap analisis dan perancangan sistem karena mampu memberikan representasi alur data yang logis, terstruktur, dan mudah dipahami oleh pengembang maupun pemangku kepentingan.

Tabel 2.5 Simbol DFD

Simbol	Nama	Keterangan
	<i>External Entities</i>	Entitas eksternal digambarkan dalam bentuk persegi yang merepresentasikan individu atau kelompok di luar sistem.
	<i>Data Store</i>	<i>Data store</i> ditampilkan dalam bentuk persegi panjang terbuka yang berfungsi sebagai tempat penyimpanan data.
	<i>Process</i>	Proses digambarkan dalam bentuk lingkaran yang menunjukkan aktivitas pengolahan, yaitu transformasi <i>input</i> menjadi <i>output</i> .
	<i>Data Flow</i>	<i>Data flow</i> merepresentasikan perpindahan data di dalam sistem dengan menekankan hubungan antara proses, <i>data store</i> , dan entitas eksternal.

DFD juga berperan sebagai fundamental guna memperoleh pemahaman yang lebih komprehensif terhadap dinamika aliran data dalam sistem (Aleryani, 2024). Tabel 2.5 merupakan beberapa simbol DFD yang umum digunakan.

2.4 Perancangan *Website*

Perancangan *website* dilakukan sebagai dasar pengembangan sistem praktikum berbasis web yang terpusat dan terintegrasi. Perancangan ini berfungsi untuk memastikan bahwa *website* mampu mendukung pengelolaan data secara terstruktur, konsisten, dan aman sesuai dengan kebutuhan sistem yang dikembangkan.

2.4.1 *Framework Laravel*

Laravel adalah *framework* PHP berbasis *Model–View–Controller* (MVC) yang menyediakan struktur pengembangan aplikasi web secara terorganisir. PHP (*Hypertext*

Preprocessor) merupakan bahasa pemrograman *open source* yang dirancang khusus untuk pengembangan web, dapat disisipkan ke dalam HTML, dan dieksekusi di sisi server sehingga hasilnya dikirimkan ke *browser* sebagai HTML. Mekanisme ini memudahkan pengembang dalam membangun halaman web dinamis (Nova, 2025).

HTML (*HyperText Markup Language*) berfungsi sebagai bahasa markup untuk menyusun struktur dasar halaman web. Dalam Laravel, HTML dihasilkan melalui *Blade templating engine* sehingga pemisahan antara logika dan tampilan tetap terjaga, sekaligus memungkinkan penyusunan antarmuka yang rapi, responsif, dan mudah dipelihara agar dapat ditampilkan secara konsisten pada berbagai perangkat (Rahmawati & Sumarsono, 2024). Tampilan diatur menggunakan CSS (*Cascading Style Sheets*), yaitu bahasa desain visual yang mengontrol aspek estetika seperti warna, tipografi, jarak, dan layout. Pemisahan antara struktur dan tampilan memungkinkan pengelolaan desain yang konsisten tanpa mengubah HTML dasar, serta mendukung pengembangan sistem web yang responsif dan *user-friendly* dengan pengalaman pengguna yang lebih optimal (Tanjung et al., 2025). Interaktivitas sistem diperkuat oleh JavaScript yang memungkinkan penambahan fitur dinamis seperti animasi, validasi formulir, dan pembaruan data tanpa memuat ulang halaman. Karena berjalan pada sisi klien, JavaScript mampu meningkatkan kecepatan respons dan kualitas interaksi pengguna. JavaScript juga dapat digunakan pada sisi *server* melalui platform seperti Node.js, sehingga menjadi komponen penting dalam pengembangan aplikasi web yang menuntut antarmuka responsif dan fungsionalitas tinggi (Raehan et al., 2025).

Pada sisi *Model*, Laravel menggunakan *Eloquent Object-Relational Mapping (ORM)* untuk mengelola interaksi basis data secara objektif sehingga data praktikan dapat diakses, diubah, dan difilter dengan lebih efisien. Bagian *View* memanfaatkan *Blade Template Engine* untuk menyajikan halaman dinamis dan terstruktur, sementara *Controller* jembatan atau koordinasi antara *Model* dan *View*, termasuk untuk pemanggilan fungsi *Merge Sort* untuk pengurutan dan *Sequential Search* untuk pencarian. Laravel menyediakan mekanisme *routing*, *middleware*, validasi *input*, serta sistem migrasi *database* yang mempermudah pengembangan dan pemeliharaan sistem praktikum antar periode. Dengan arsitektur yang jelas, Laravel mendukung sistem tetap terstruktur (Rahmawati & Sumarsono, 2024).

2.4.2 MySQL Database

MySQL berperan sebagai sistem manajemen basis data untuk menyimpan seluruh informasi praktikan seperti NIM, nama, kelompok, kehadiran, nilai *pretest*, asistensi, dan nilai akhir. Struktur tabel dirancang agar mendukung proses pengurutan dan pencarian, serta kompatibel dengan *Eloquent Object-Relational Mapping (ORM)* pada Laravel. MySQL dipilih karena stabil, cepat, dan mudah terintegrasi dengan Laravel melalui konfigurasi koneksi basis data bawaan (Lontaan & Sinadia, 2024). Untuk mendukung kinerja sistem, struktur basis data disusun secara terorganisir dengan memperhatikan konsistensi dan keterkaitan antar tabel agar proses pencarian dan pengurutan data berjalan dengan baik serta tetap fleksibel terhadap perubahan kebutuhan sistem. Migrasi *database* Laravel mempermudah pembentukan struktur tabel yang konsisten pada setiap lingkungan pengembangan maupun *deployment*. Dengan adanya migrasi, struktur *database* dapat diperbarui dan diatur secara sistematis seiring perkembangan sistem praktikum.

2.4.3 Pengujian Black Box

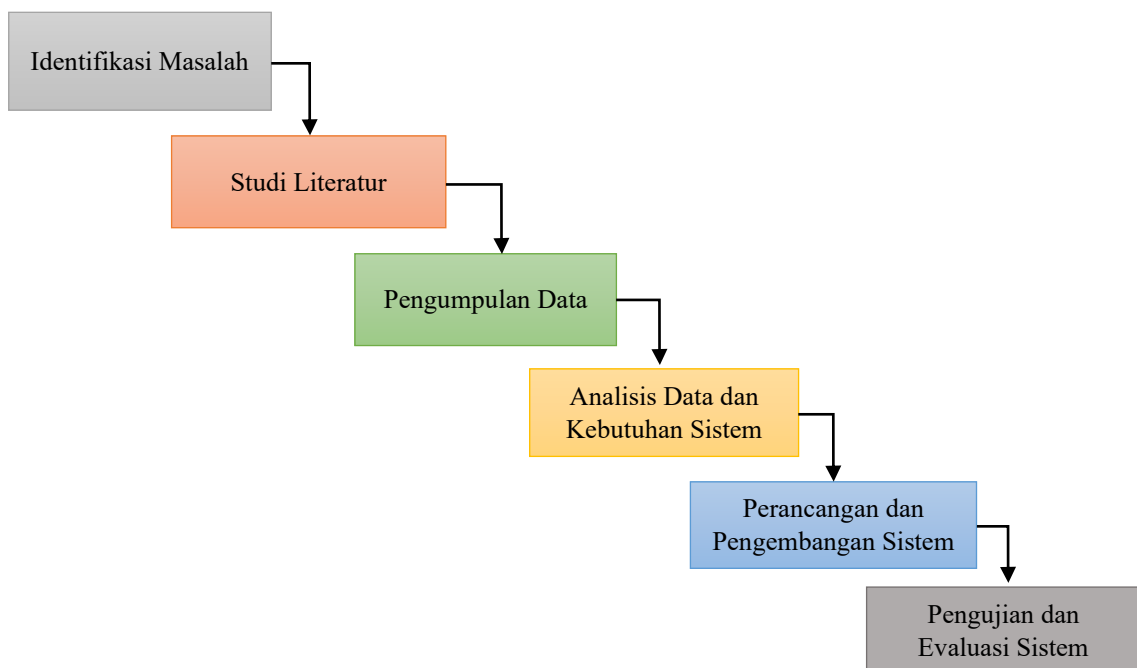
Pengujian *Black Box* merupakan metode pengujian perangkat lunak yang mengevaluasi kesesuaian fungsi sistem terhadap kebutuhan pengguna dengan berfokus pada masukan dan keluaran tanpa melihat struktur kode program. Pengujian ini bertujuan memastikan setiap fitur berjalan sesuai spesifikasi melalui skenario uji berdasarkan perilaku sistem. Salah satu teknik yang digunakan adalah *Equivalence Partitioning*, yaitu membagi data masukan ke dalam kelompok *valid* dan tidak *valid* agar proses pengujian lebih sistematis dan efisien. Metode ini membantu mendeteksi kesalahan fungsional pada sistem berbasis web sejak tahap awal pengembangan (Yulianti et al., 2023).

BAB III

METODOLOGI PENELITIAN

3.1 Tahapan Pelaksanaan Penelitian

Pelaksanaan penelitian ini dirancang secara bertahap untuk menggambarkan alur kegiatan penelitian sejak penemuan permasalahan hingga penerapan sistem. Urutan tahapan tersebut disajikan pada Gambar 3.1 yang mengacu pada kerangka penelitian yang telah disusun, uraian setiap tahap adalah sebagai berikut.



Gambar 3.1 Tahapan Pelaksanaan Penelitian

1. Identifikasi Masalah

Tahap identifikasi masalah dilakukan untuk memahami kondisi nyata pengelolaan data Praktikum Fisika Dasar di Fakultas Teknik Universitas Mulawarman. Proses ini dilakukan melalui observasi langsung dan wawancara dengan koordinator praktikum serta asisten praktikum guna mengetahui alur pencatatan, penyimpanan, pencarian, dan rekapitulasi data praktikan yang masih menggunakan beberapa file *spreadsheet* terpisah, sehingga menimbulkan keterlambatan proses kerja serta potensi terjadinya ketidakkonsistenan data.

2. Studi Literatur

Pada tahap ini dilakukan penelusuran berbagai referensi ilmiah yang relevan dengan sistem berbasis web, pengelolaan basis data, metode pengembangan perangkat lunak, serta algoritma *Merge Sort* dan *Sequential Search*. Literatur yang diperoleh digunakan sebagai landasan teori dan acuan ilmiah dalam proses perancangan, implementasi, dan evaluasi sistem.

3. Pengumpulan Data

Pengumpulan data dilakukan melalui observasi, wawancara, dan dokumentasi terhadap data Praktikum Fisika Dasar. Data yang diperoleh mencakup identitas praktikan, data presensi, nilai kemampuan praktikum, nilai laporan, nilai asistensi, serta rekapitulasi nilai akhir yang selanjutnya dimanfaatkan sebagai bahan analisis kebutuhan sistem dan pengujian algoritma.

4. Analisis Data dan Kebutuhan Sistem

Data yang telah terkumpul kemudian dianalisis untuk mengidentifikasi kebutuhan sistem, baik dari sisi fungsi maupun kinerja. Hasil analisis ini menjadi dasar perancangan solusi berupa sistem praktikum berbasis web yang mampu mengelola data secara terintegrasi serta menyediakan fitur pengurutan dan pencarian data.

5. Perancangan dan Pengembangan Sistem

Tahap ini meliputi perancangan basis data menggunakan ERD, pemodelan alur proses dengan DFD dan *Flowchart*, serta perancangan antarmuka. Seluruh rancangan diwujudkan dalam bentuk sistem praktikum berbasis web yang menerapkan algoritma *Merge Sort* untuk pengurutan data dan *Sequential Search* untuk pencarian data pada lapisan aplikasi.

6. Pengujian dan Evaluasi Sistem

Sistem yang telah dibangun kemudian diuji untuk memastikan setiap fungsi berjalan sesuai dengan kebutuhan pengguna. Pengujian dilakukan menggunakan metode *black-box testing* serta evaluasi kinerja algoritma dengan mengukur waktu eksekusi, penggunaan memori, dan respons sistem pada beberapa variasi ukuran data.

3.2 Pengumpulan Data

Pengumpulan data merupakan salah satu tahapan penting dalam penelitian yang bertujuan untuk memperoleh informasi yang relevan dan akurat sesuai dengan permasalahan yang telah dirumuskan. Data yang diperoleh digunakan sebagai dasar dalam perancangan dan pengembangan sistem berbasis web serta sebagai bahan analisis untuk mendukung penerapan algoritma yang digunakan dalam penelitian ini. Proses pengumpulan data dilakukan secara sistematis agar hasil penelitian memiliki validitas dan dapat dipertanggungjawabkan secara ilmiah.

Adapun teknik pengumpulan data yang diterapkan dalam penelitian ini adalah sebagai berikut:

1. **Observasi**

Observasi dilakukan dengan cara mengamati secara langsung objek penelitian, yaitu proses pengelolaan data Praktikum Fisika Dasar di lingkungan Fakultas Teknik Universitas Mulawarman. Kegiatan observasi ini bertujuan untuk mengetahui alur kerja yang berjalan saat ini, mengidentifikasi permasalahan yang muncul, serta memahami kebutuhan pengguna secara nyata. Observasi merupakan teknik pengumpulan data yang dilakukan secara sistematis untuk memperoleh gambaran faktual mengenai kondisi lapangan.

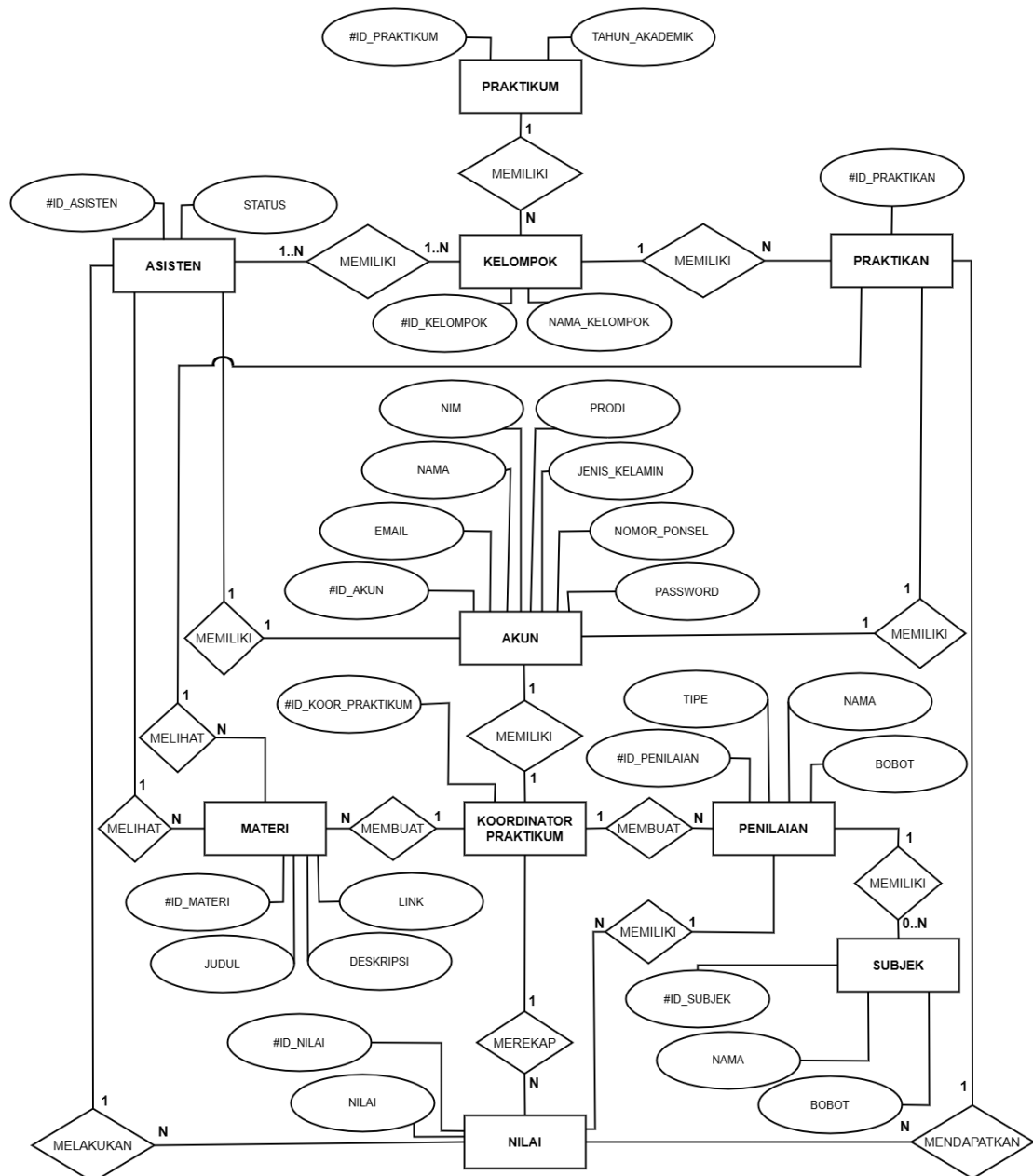
2. **Wawancara**

Wawancara dilakukan sebagai metode pengumpulan data dengan cara tanya jawab secara langsung antara peneliti dan narasumber yang dianggap memiliki keterkaitan dengan objek penelitian. Narasumber dalam penelitian ini adalah koordinator yang menjadi penanggungjawab Praktikum Fisika Dasar. Wawancara bertujuan untuk memperoleh informasi yang lebih mendalam mengenai proses pencatatan, pengurutan, dan pencarian data, serta kebutuhan terhadap sistem berbasis web yang diharapkan dapat meningkatkan efektivitas pengelolaan data praktikum.

3.3 Perancangan Data

Perancangan data dalam penelitian ini difokuskan pada penyusunan struktur basis data yang mendukung pengelolaan data praktikan secara terpusat, konsisten, dan efisien pada sistem praktikum berbasis web. Tahapan ini dilakukan dengan mengidentifikasi

entitas utama, atribut, serta hubungan antar entitas yang merepresentasikan data Praktikum Fisika Dasar, kemudian memodelkannya menjadi *Entity Relationship Diagram* (ERD).



Gambar 3.2 ERD Sistem Praktikum Fisika Dasar

ERD digunakan untuk menggambarkan struktur logis data seperti data akun praktikan, asisten, dan koordinator praktikum, kelompok praktikum, materi, nilai, penilaian, serta subjek penilaian sehingga hubungan antar data dapat didefinisikan secara jelas sebelum diimplementasikan ke dalam basis data relasional. Penerapan ERD

membantu mengurangi redundansi data, menjaga integritas data, serta mempermudah proses pengembangan dan pemeliharaan sistem.

ERD sistem praktikum Fisika Dasar pada Gambar 3.2 yang telah dirancang terdiri dari beberapa entitas utama yang saling berelasi untuk mendukung pengelolaan data praktikum secara terintegrasi.

1. Entitas Akun berfungsi sebagai penyimpan data autentikasi pengguna sistem dengan atribut utama berupa *id_akun* (PK), *email*, *nama*, *nim*, *prodi*, *jenis_kelamin*, *nomor_ponsel*, dan *password* yang disimpan dalam bentuk *hash*. Entitas ini menjadi pusat akses karena satu akun dapat dimiliki oleh pengguna dengan peran berbeda, yaitu praktikan, asisten, dan koordinator praktikum, sehingga kontrol akses dapat dikelola secara lebih terstruktur.
2. Entitas Praktikan menyimpan data identitas mahasiswa yang mengikuti praktikum Fisika Dasar yang telah melakukan proses registrasi melalui sistem dengan mengisi data akun dan melengkapi data pribadi praktikan dengan atribut *id_praktikan* (PK). Setiap praktikan terhubung dengan satu Akun dan tergabung ke dalam satu Kelompok, sehingga relasi yang terbentuk bersifat *one-to-many*, di mana satu kelompok dapat memiliki banyak praktikan.
3. Entitas Asisten digunakan untuk menyimpan data asisten praktikum yang bertugas mendampingi kegiatan praktikum Fisika Dasar. Entitas ini memiliki atribut *id_asisten* (PK) dan *status*. Asisten dapat menangani satu atau lebih kelompok praktikum, dan sebaliknya satu kelompok dapat didampingi oleh satu atau lebih asisten sesuai pembagian tugas yang telah ditetapkan, sehingga relasi yang terbentuk antara Entitas Asisten dan Entitas Kelompok bersifat *many-to-many*.
4. Entitas Kelompok berperan sebagai pengelompokan praktikan dalam satu periode praktikum dengan atribut utama *id_kelompok* (PK) dan *nama_kelompok*. Entitas ini berelasi dengan entitas Praktikum, Asisten, dan Praktikan, sehingga mendukung pengelolaan dan keterkaitan data dalam sistem praktikum secara terstruktur.
5. Entitas Praktikum menyimpan informasi pelaksanaan praktikum berdasarkan periode akademik dengan atribut *id_praktikum* (PK) dan *tahun_akademik*. Setiap periode praktikum memiliki beberapa kelompok, sehingga relasi antara praktikum

dan kelompok bersifat *one-to-many*. Struktur ini memungkinkan sistem mengelola data tahun akademik tanpa tumpang tindih data.

6. Entitas Koordinator Praktikum merepresentasikan pihak yang memiliki wewenang dalam pengelolaan kegiatan praktikum dengan atribut *id_koor_praktikum* (PK) yang terhubung dengan entitas Akun. Koordinator bertanggung jawab dalam membuat materi serta menyusun komponen penilaian, sehingga koordinator praktikum memiliki relasi dengan entitas Materi dan Penilaian, menunjukkan peran koordinator dalam melakukan rekap data penilaian praktikan.
7. Entitas Penilaian digunakan untuk menyimpan seluruh komponen evaluasi yang digunakan dalam proses penilaian praktikan dengan atribut *id_penilaian* (PK), tipe, nama, dan bobot. Entitas ini berelasi dengan entitas Nilai, Subjek dan Koordinator Praktikum, yang menunjukkan bahwa setiap praktikan memperoleh satu set data Nilai oleh asisten yang kemudian direkap menjadi penilaian lengkap oleh koordinator praktikum.
8. Entitas Subjek merepresentasikan rincian indikator dari setiap komponen penilaian dengan atribut *id_subjek* (PK), nama, dan bobot. Relasi antara Penilaian dan Subjek bersifat *one-to-many*, di mana satu komponen penilaian dapat memiliki beberapa subjek agar proses evaluasi lebih terstruktur dan terukur.
9. Entitas Nilai digunakan untuk menyimpan hasil evaluasi praktikan dengan atribut *id_nilai* (PK) dan nilai. Setiap nilai berkaitan dengan Subjek penilaian dan diperoleh oleh Praktikan, sementara proses rekapitulasi dilakukan oleh Koordinator Praktikum. Relasi ini memungkinkan sistem menampilkan hasil penilaian secara komprehensif serta mendukung proses transparansi penilaian akademik.

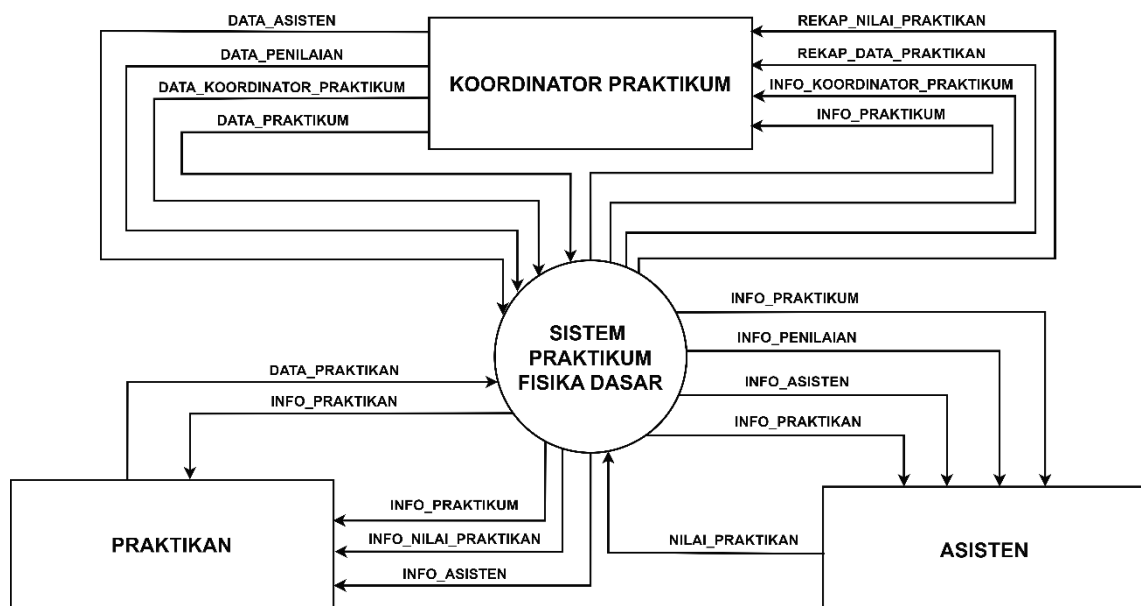
3.4 Perancangan Proses

Perancangan proses pada penelitian ini difokuskan pada pemodelan alur interaksi dan aliran data dalam Sistem Praktikum Fisika Dasar berbasis web guna memastikan bahwa setiap fungsi sistem dapat berjalan secara terstruktur, konsisten, dan sesuai kebutuhan pengguna. Tahapan ini dilakukan dengan memanfaatkan beberapa model pemodelan proses, yaitu *Flowchart* dan *Data Flow Diagram* (DFD) yang masing-masing memiliki peran dalam merepresentasikan cara kerja sistem dari sudut pandang yang

berbeda. DFD digunakan untuk memodelkan aliran data antar proses dan entitas secara logis, sedangkan *Flowchart* digunakan untuk menggambarkan urutan langkah proses secara sistematis. Penerapan pemodelan proses ini bertujuan untuk meminimalkan kesalahan perancangan, meningkatkan kejelasan fungsi sistem, serta mempermudah proses implementasi dan pengembangan lanjutan, khususnya pada sistem yang melibatkan banyak peran pengguna dan kompleksitas data yang tinggi.

3.4.1 Data Flow Diagram (DFD)

Perancangan *Data Flow Diagram* (DFD) pada Sistem Praktikum Fisika Dasar menggambarkan alur utama pertukaran data antara sistem dengan entitas eksternal yang terlibat, yaitu Koordinator Praktikum, Asisten, dan Praktikan.



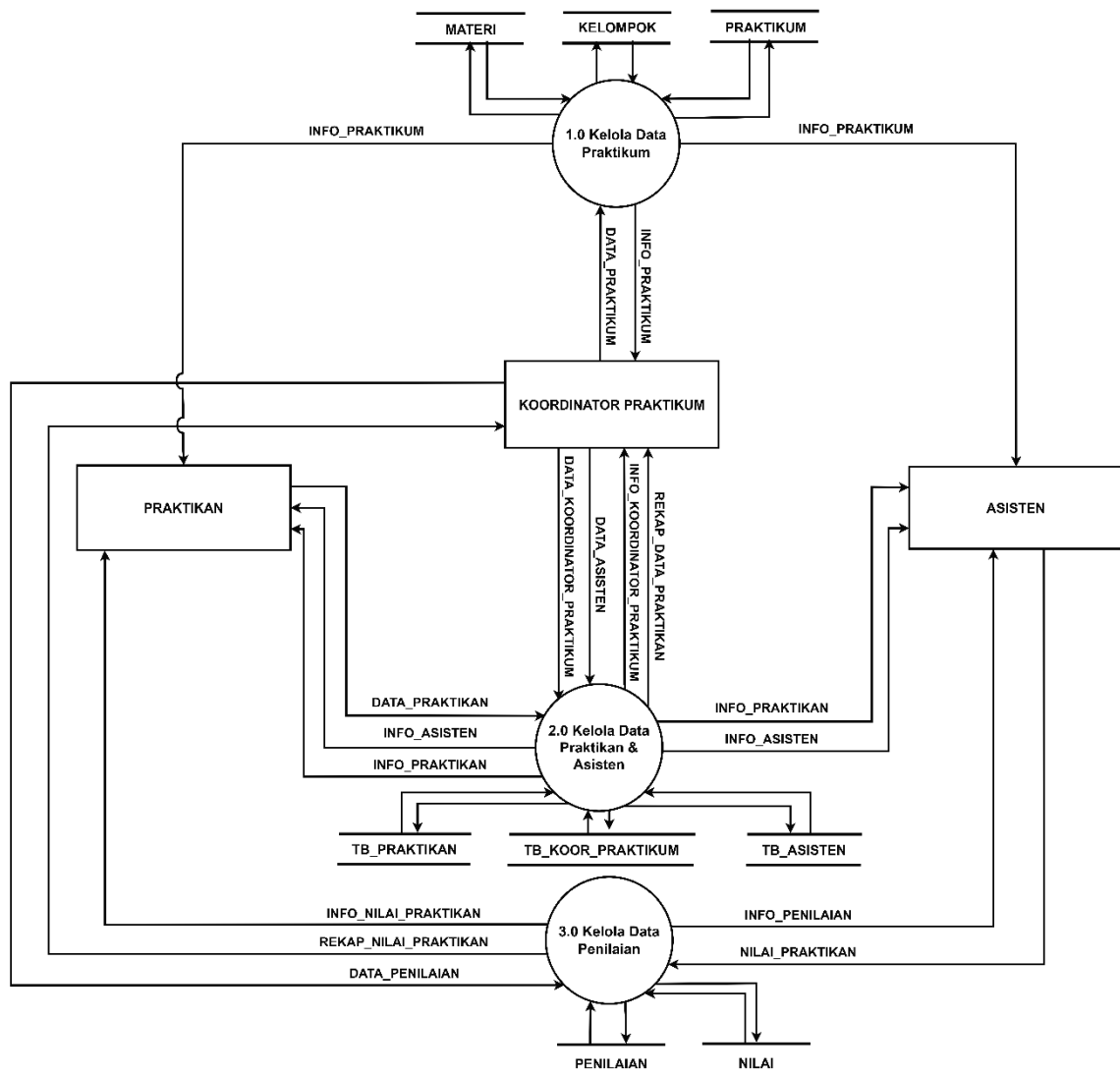
Gambar 3.3 DFD Level 0 Sistem Praktikum Fisika Dasar

Pada Gambar 3.3 DFD level 0, sistem digambarkan sebagai satu proses utama yang mengelola seluruh aktivitas Praktikum Fisika Dasar, mulai dari data pengguna hingga rekapitulasi nilai praktikum.

1. Koordinator Praktikum berinteraksi dengan sistem dalam lingkup pengelolaan yang bersifat menyeluruh. Koordinator mengelola data asisten, data koordinator praktikum, data penilaian, serta data praktikum. Selain itu, koordinator juga melakukan proses rekap data praktikan dan rekap nilai praktikan sebagai bagian dari *monitoring* dan

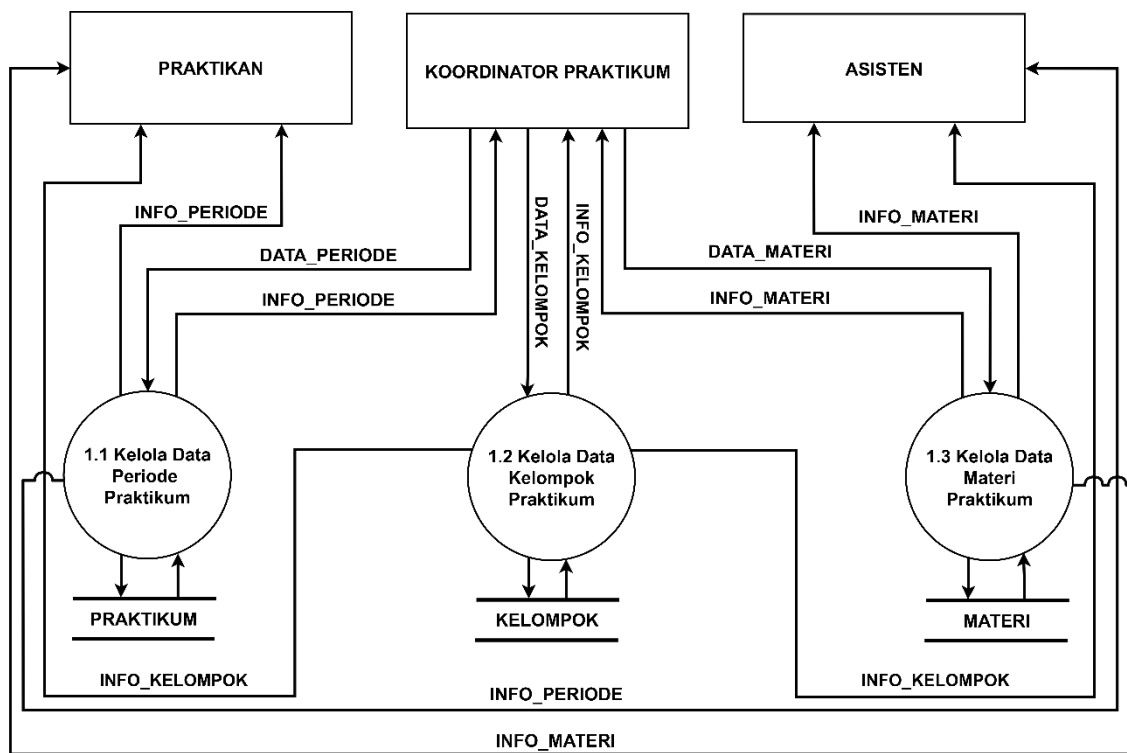
evaluasi pelaksanaan kegiatan praktikum. Data yang dihasilkan dari proses ini disimpan dalam basis data sistem dan dapat diakses kembali sesuai kebutuhan.

2. Asisten mengakses informasi praktikum, penilaian, asisten, dan praktikan sebagai pendukung pelaksanaan praktikum. Asisten juga menginput nilai praktikan berdasarkan hasil evaluasi yang kemudian digunakan dalam proses rekapitulasi oleh koordinator.
3. Praktikan melakukan registrasi dan autentikasi, mengelola data diri, serta mengakses informasi terkait praktikum dan hasil penilaian praktikum. Sistem berfungsi sebagai media penyedia informasi yang terpusat sehingga praktikan dapat memantau perkembangan hasil praktikum secara mandiri.



Gambar 3.4 DFD Level 1 Sistem Praktikum Fisika Dasar

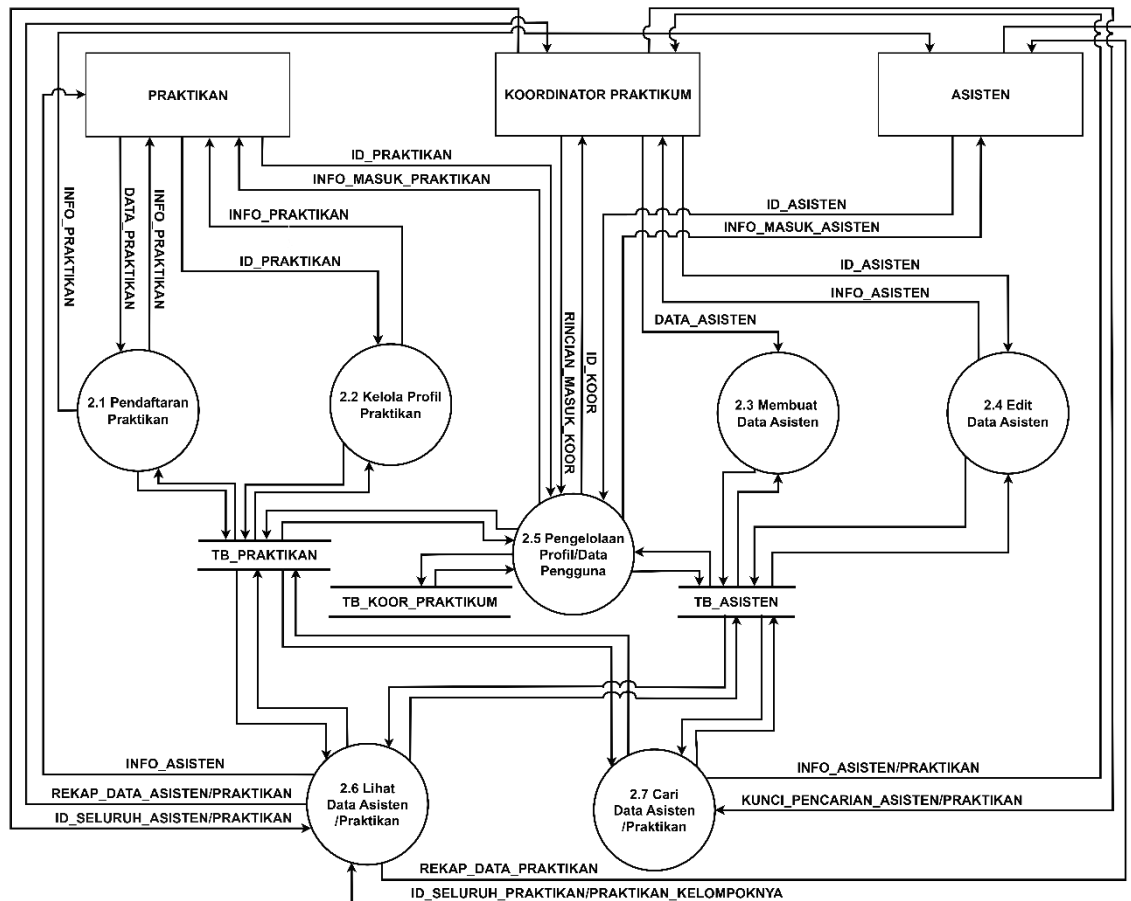
DFD Level 1 pada Gambar 3.4 merupakan penguraian dari proses utama pada DFD Level 0 menjadi beberapa proses yang lebih rinci untuk menggambarkan fungsi sistem secara lebih spesifik. Pada level ini, Sistem Praktikum Fisika Dasar dibagi menjadi beberapa proses inti, yaitu pengelolaan data praktikum, pengelolaan data praktikan dan asisten, serta pengelolaan data penilaian. Setiap proses menunjukkan bagaimana data dari entitas eksternal diterima oleh sistem, diproses sesuai kebutuhan, kemudian disimpan dalam basis data sebelum menghasilkan informasi yang dapat diakses kembali oleh pengguna sesuai perannya. Dengan pemecahan proses ini, alur kerja sistem dapat dipahami secara lebih jelas dan terstruktur.



Gambar 3.5 DFD Level 2 pada Proses 1 Sistem Praktikum Fisika Dasar

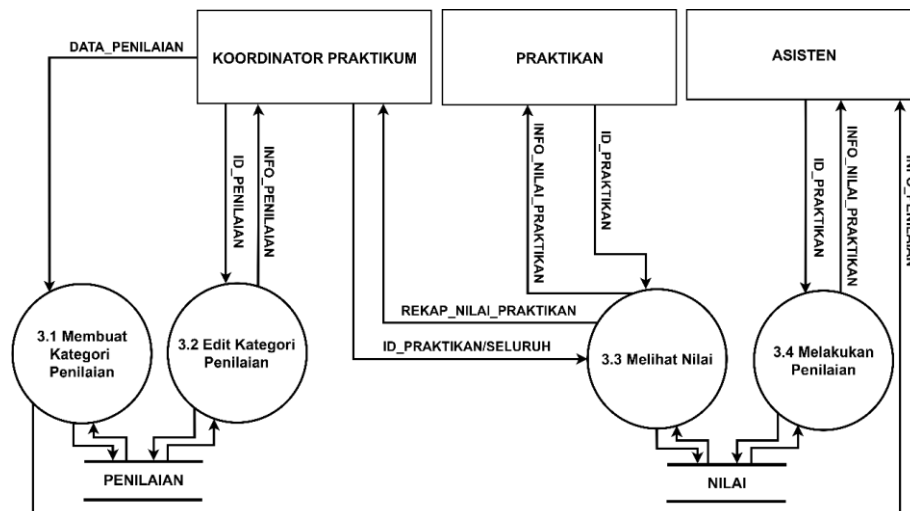
DFD Level 2 pada Gambar 3.5 merupakan penguraian dari proses utama pada DFD Level 1 menjadi beberapa proses yang lebih rinci untuk menggambarkan fungsi sistem secara lebih spesifik. DFD level 2 pada proses 2 menggambarkan pengelolaan data praktikan dan asisten secara lebih rinci. Proses ini mencakup pendaftaran praktikan, perubahan profil praktikan, pembuatan dan perubahan data asisten, proses masuk dengan akun, melihat data praktikan dan asisten, serta pencarian data. Setiap proses berinteraksi

dengan entitas pengguna dan basis data sesuai kebutuhan pengelolaan informasi. Alur proses tersebut ditunjukkan pada Gambar 3.6.



Gambar 3.6 DFD Level 2 pada Proses 2 Sistem Praktikum Fisika Dasar

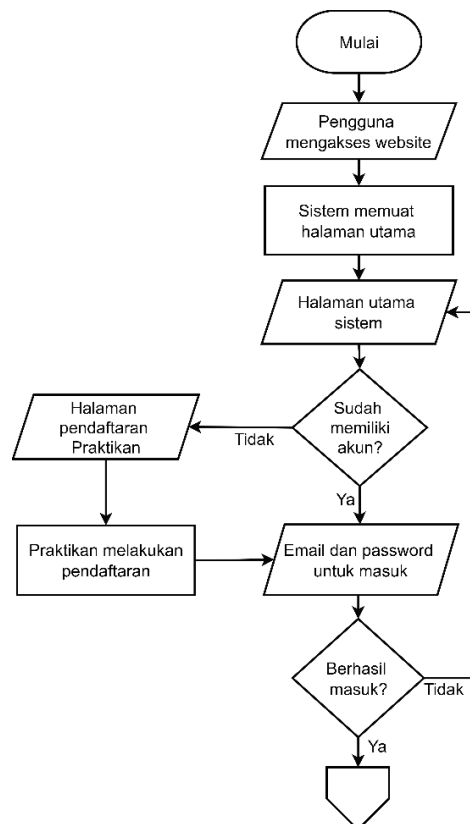
Proses pengelolaan data penilaian melibatkan koordinator praktikum, asisten, dan praktikan sesuai dengan hak akses masing-masing. Koordinator bertugas mengelola kategori penilaian dan melakukan rekapitulasi nilai berdasarkan data yang telah diberikan oleh asisten. Asisten memberikan nilai kepada praktikan berdasarkan hasil kegiatan praktikum dan komponen penilaian yang telah ditentukan. Sementara itu, praktikan dapat mengakses informasi nilai yang telah diperoleh sebagai hasil dari kegiatan praktikum yang telah diikuti. Setiap data penilaian dikelola sesuai dengan peran pengguna untuk menjaga keteraturan dan keamanan informasi. Data yang diberikan oleh asisten menjadi dasar dalam proses rekapitulasi dan penyajian nilai kepada praktikan. Pengelolaan secara terstruktur juga memudahkan proses pemantauan dan pengecekan data penilaian oleh koordinator.



Gambar 3.7 DFD Level 2 pada Proses 3 Sistem Praktikum Fisika Dasar

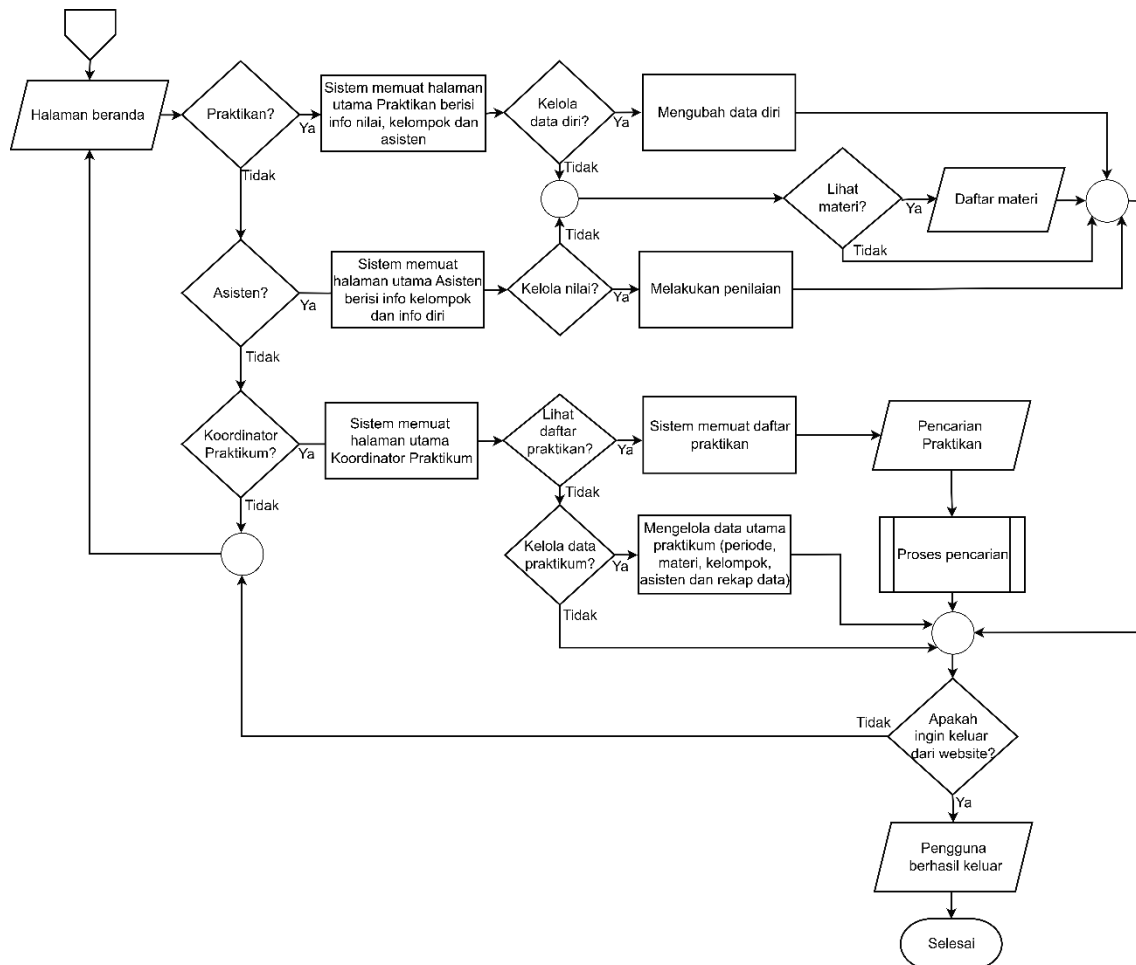
Sistem dapat mendukung proses penilaian yang lebih terorganisasi, transparan, dan mudah diakses oleh setiap pengguna sesuai dengan hak aksesnya. Alur pengelolaan data penilaian tersebut ditunjukkan pada Gambar 3.7.

3.4.2 Flowchart Sistem



Gambar 3.8 Flowchart Pendaftaran dan Masuk Sistem Praktikum Fisika Dasar

Flowchart sistem menunjukkan urutan proses pada Sistem Praktikum Fisika Dasar, mulai dari akses *website* hingga pengguna masuk ke halaman utama. Pengguna yang telah memiliki akun dapat masuk menggunakan email dan *password*, sedangkan praktikan yang belum memiliki akun harus melakukan pendaftaran terlebih dahulu. Setelah berhasil masuk, sistem mengarahkan pengguna ke halaman beranda sesuai perannya. Alur pendaftaran dan proses masuk ditunjukkan pada Gambar 3.8.

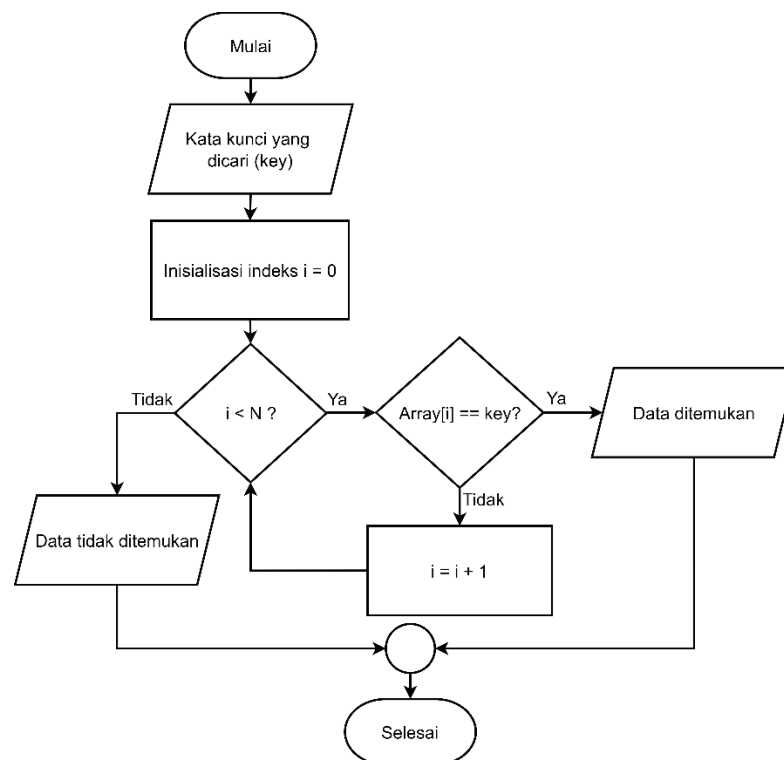


Gambar 3.9 Flowchart Sistem Praktikum Fisika Dasar

Setelah berhasil masuk, sistem menyediakan fitur sesuai dengan peran pengguna. Praktikan dapat mengelola data diri serta melihat informasi praktikum, nilai, kelompok, asisten, dan materi. Asisten dapat melihat dan mencari data praktikum, mengelola penilaian, serta mengakses materi. Koordinator Praktikum dapat mengelola data periode, materi, kelompok, asisten, praktikan, dan rekap data. Seluruh proses dapat dilakukan selama pengguna berada di dalam sistem dan berakhir ketika pengguna keluar dari website. Alur penggunaan Sistem Praktikum Fisika Dasar ditunjukkan pada Gambar 3.9.

3.4.3 Flowchart Sequential Search

Poses pencarian data menggunakan *Sequential Search* dilakukan dengan menelusuri elemen *array* secara berurutan dan membandingkannya dengan *key* hingga ditemukan kecocokan atau seluruh elemen diperiksa. Jika kecocokan ditemukan, data dinyatakan ditemukan, sedangkan jika pencarian mencapai akhir *array* tanpa kecocokan, data dinyatakan tidak ditemukan. Alur proses *Sequential Search* tersebut ditunjukkan pada Gambar 3.10.



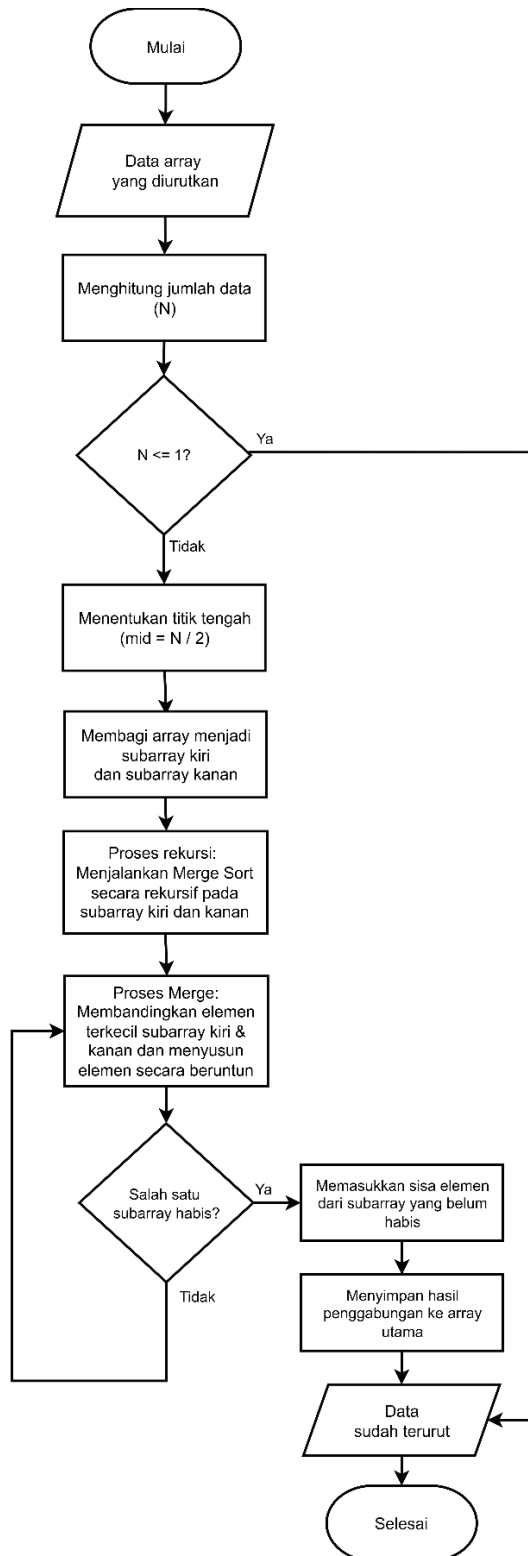
Gambar 3.10 Flowchart Sequential Search

Penerapan *Sequential Search* memungkinkan proses pencarian data dilakukan secara sederhana dan sistematis sesuai urutan elemen yang tersedia. Proses ini mendukung pencarian data pada sistem praktikum dengan memastikan setiap elemen diperiksa hingga data ditemukan atau seluruh elemen telah ditelusuri.

3.4.4 Flowchart Merge Sort

Proses pengurutan data menggunakan *Merge Sort* menerapkan pendekatan *divide and conquer* dengan membagi data menjadi *subarray* yang lebih kecil secara rekursif hingga mencapai kondisi dasar. Selanjutnya, setiap *subarray* digabungkan kembali

dengan membandingkan dan menyusun elemennya secara berurutan hingga menghasilkan *array* yang telah terurut. Alur proses *Merge Sort* tersebut ditunjukkan pada Gambar 3.11.

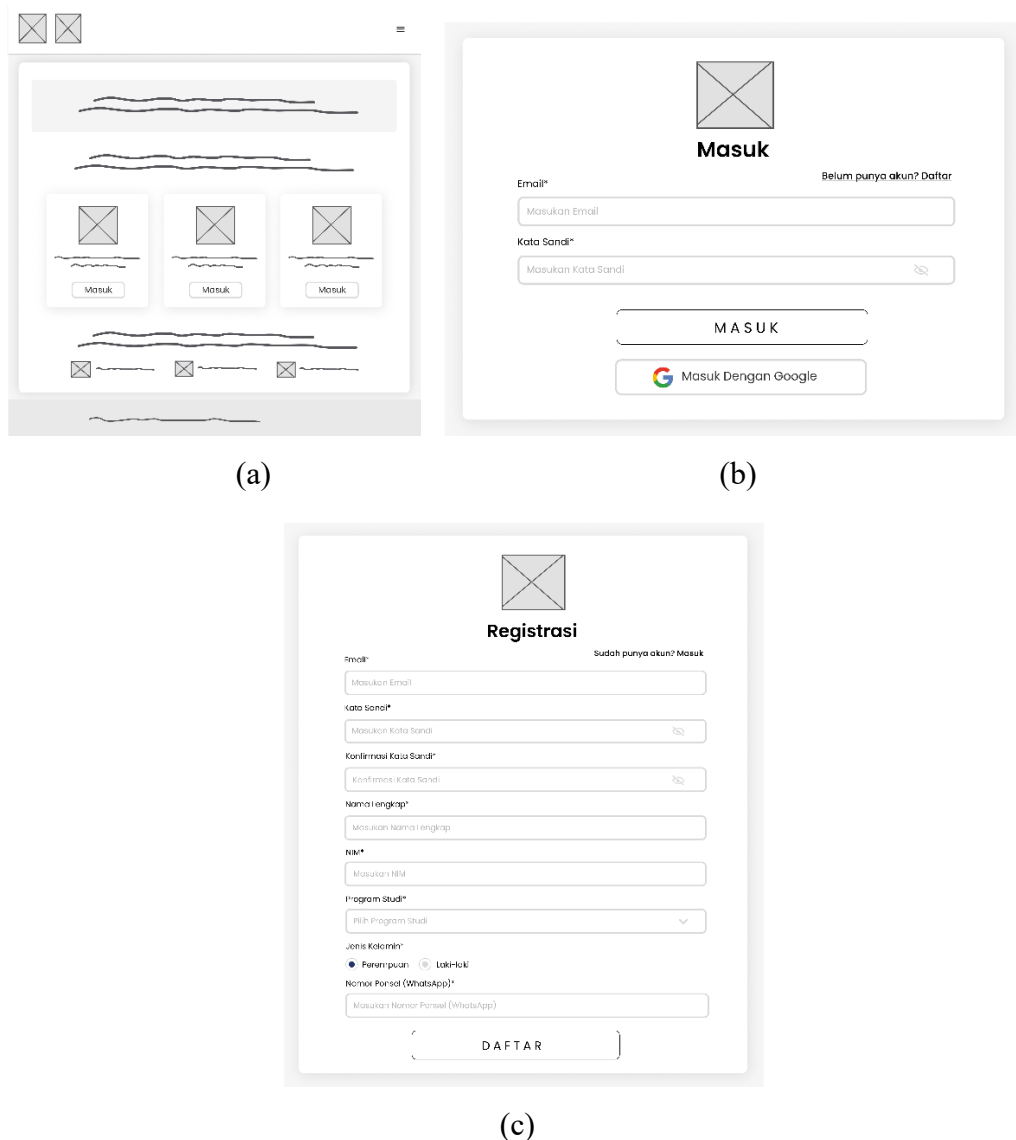


Gambar 3.11 *Flowchart Merge Sort*

3.5 Perancangan Tampilan

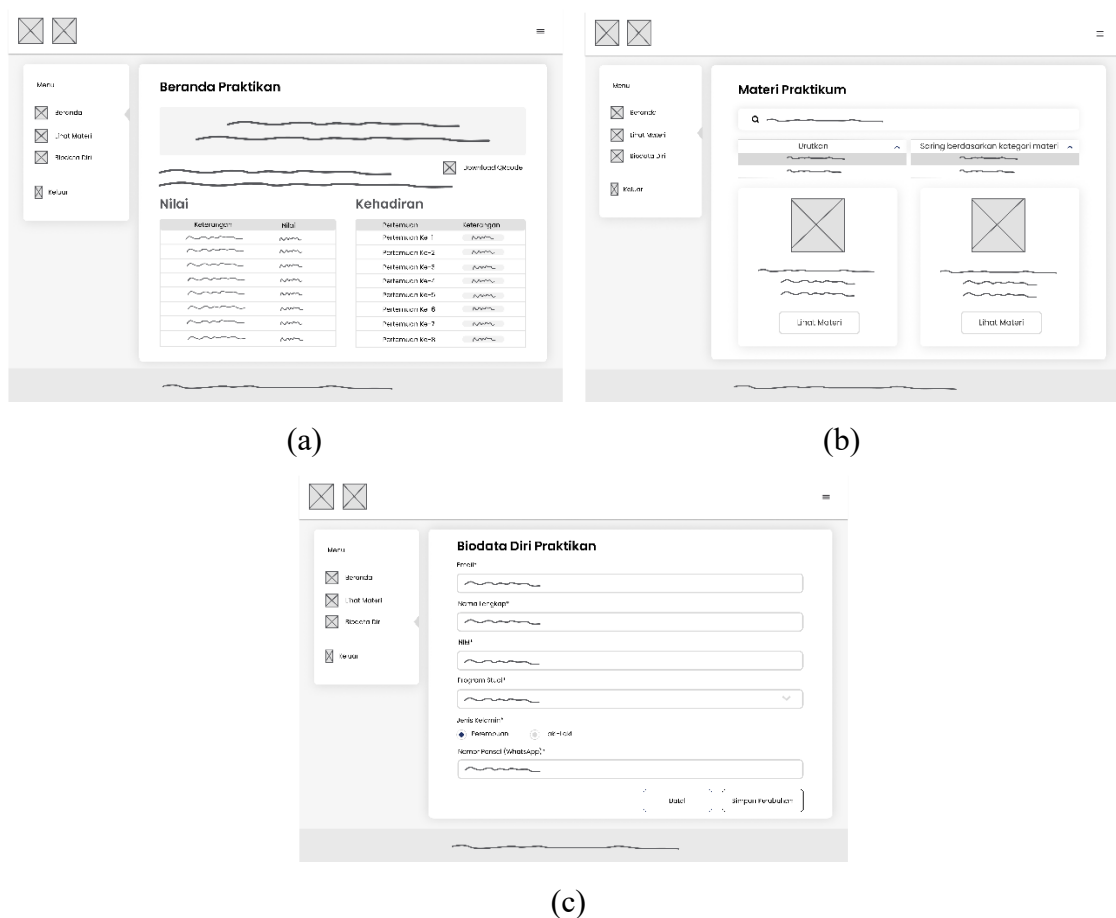
Perancangan tampilan Sistem Praktikum Fisika Dasar menghasilkan antarmuka yang mudah digunakan, informatif, dan sesuai dengan hak akses praktikan, asisten, serta koordinator praktikum. *Wireframe* menggambarkan susunan komponen dan tata letak sebelum sistem dikembangkan.

Halaman awal mencakup halaman utama, halaman masuk, dan halaman registrasi. Halaman utama memuat informasi praktikum, halaman masuk mengarahkan pengguna sesuai perannya, sedangkan halaman registrasi digunakan praktikan untuk mendaftarkan identitas. Rancangan tersebut ditunjukkan pada Gambar 3.12.



Gambar 3.12 Wireframe Halaman Awal Sistem: (a) halaman utama, (b) halaman masuk, dan (c) halaman registrasi praktikan

Tampilan praktikan dirancang untuk membantu pengguna mengakses informasi praktikum dan mengelola data pribadinya. Halaman beranda menampilkan nama praktikan, kelompok, asisten pembimbing, serta akses menuju materi dan biodata. Halaman materi menyajikan daftar bahan praktikum yang dilengkapi judul, deskripsi singkat, dan tautan materi. Halaman biodata menampilkan identitas praktikan serta menyediakan fitur perubahan data tertentu agar informasi pengguna tetap sesuai. Rancangan tampilan praktikan ditunjukkan pada Gambar 3.13.



Gambar 3.13 Wireframe Halaman Praktikan: (a) halaman beranda, (b) halaman materi praktikum, dan (c) halaman biodata diri

Tampilan asisten dirancang untuk mendukung pemantauan kegiatan praktikum, pencatatan presensi, dan pengelolaan nilai praktikan. Halaman beranda menampilkan ringkasan kelompok, jumlah pertemuan, statistik kehadiran, dan rata-rata nilai. Halaman presensi digunakan untuk mencatat serta memantau kehadiran praktikan pada setiap pertemuan. Halaman materi menyediakan daftar bahan praktikum yang dapat digunakan

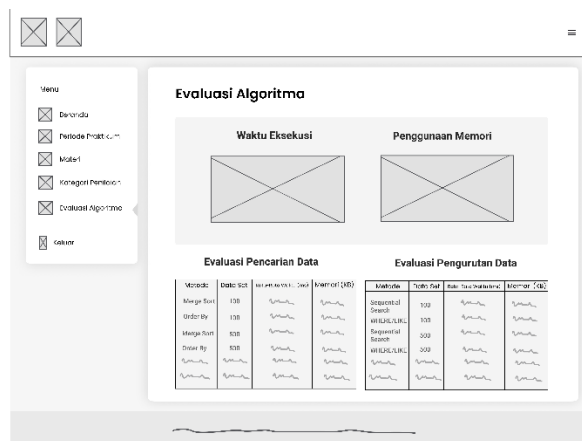
sebagai pedoman pelaksanaan kegiatan. Halaman penilaian menampilkan daftar nilai praktikan, sedangkan halaman beri nilai menyediakan formulir pengisian nilai berdasarkan kategori dan subjek penilaian yang telah ditentukan. Rancangan tampilan asisten ditunjukkan pada Gambar 3.14.



Gambar 3.14 Wireframe Halaman Asisten: (a) halaman beranda, (b) halaman presensi praktikan, (c) halaman materi praktikum, (d) halaman penilaian, dan (e) halaman beri nilai

Tampilan utama koordinator praktikum dirancang untuk mendukung pengelolaan data dan pengaturan sistem secara menyeluruh. Halaman beranda menjadi akses awal menuju menu pengelolaan praktikum. Halaman periode digunakan untuk mengelola tahun akademik dan semester praktikum. Halaman materi digunakan untuk mengatur judul, deskripsi, kategori, dan tautan bahan praktikum. Halaman kategori penilaian menyediakan pengaturan jenis penilaian, subjek penilaian, serta bobot terhadap nilai akhir. Halaman evaluasi algoritma menyajikan grafik dan tabel hasil pengujian kinerja *Merge Sort*, *Sequential Search*, *MySQL ORDER BY*, dan *MySQL WHERE/LIKE*. Rancangan tampilan utama koordinator praktikum ditunjukkan pada Gambar 3.15.

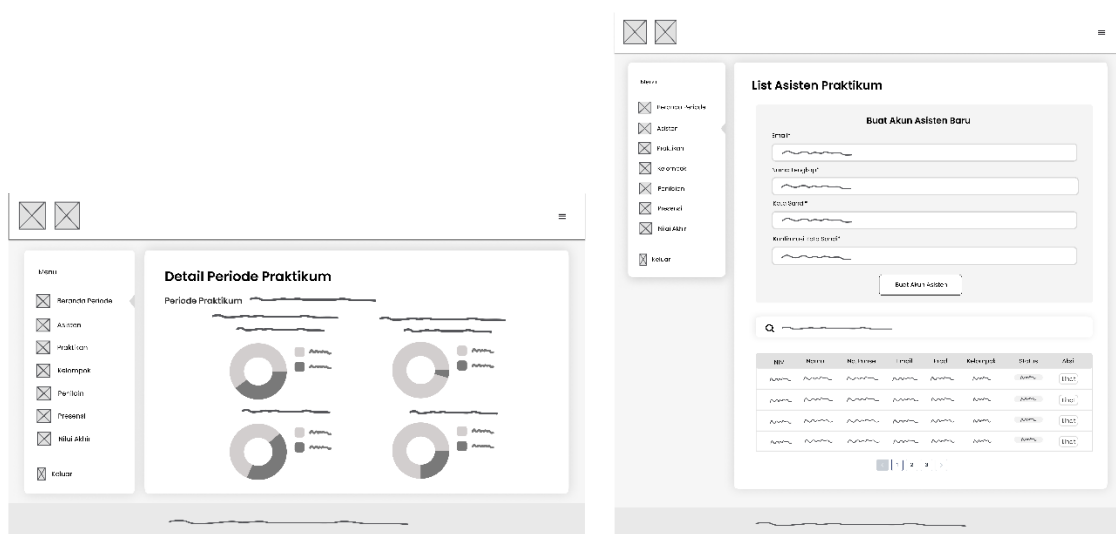




(e)

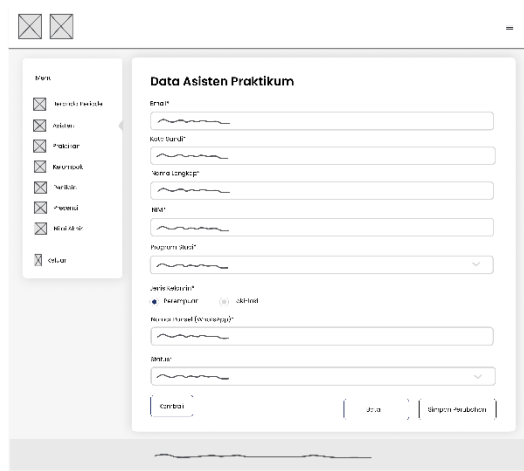
Gambar 3.15 Wireframe Halaman Koordinator: (a) halaman beranda, (b) halaman periode praktikum, (c) halaman materi praktikum, (d) halaman kategori penilaian, dan (e) halaman evaluasi algoritma

Tampilan pengelolaan data periode digunakan koordinator untuk mengatur pengguna dan kelompok dalam satu periode praktikum. Halaman detail periode menampilkan jumlah kelompok, praktikan, serta ringkasan hasil pelaksanaan praktikum. Halaman asisten digunakan untuk mengelola akun dan informasi asisten, sedangkan halaman data praktikan menampilkan seluruh praktikan yang terdaftar dalam sistem. Halaman kelompok dan detail kelompok untuk mengatur anggota kelompok serta asisten pembimbing. Rancangan pengelolaan data periode praktikum ditunjukkan pada Gambar 3.16.

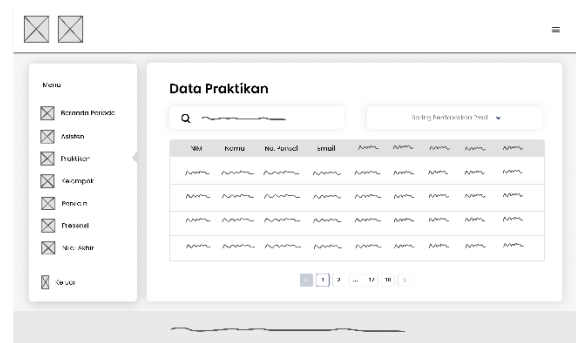


(a)

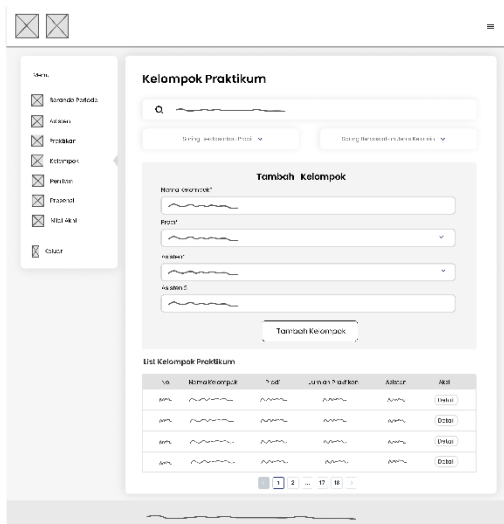
(b)



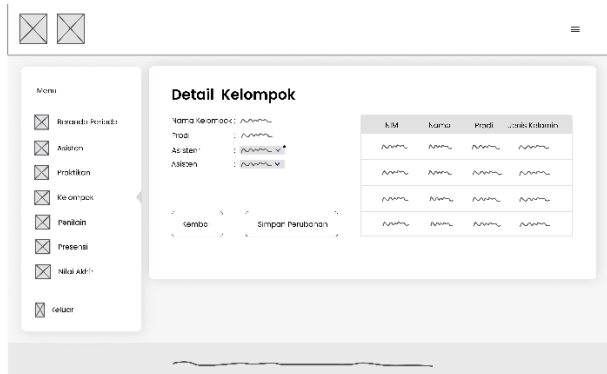
(c)



(d)



(e)



(f)

Gambar 3.16 Wireframe Pengelolaan Data Periode Praktikum: (a) halaman detail periode, (b) halaman asisten, (c) halaman data asisten, (d) halaman data praktikan, (e) halaman kelompok praktikum, dan (f) halaman detail kelompok praktikum

Tampilan pemantauan hasil praktikum membantu koordinator melihat hasil penilaian dan kehadiran praktikan secara menyeluruh. Halaman penilaian menampilkan nilai praktikan berdasarkan kategori yang telah ditentukan. Halaman presensi menyajikan rekap kehadiran pada setiap pertemuan, sedangkan halaman nilai akhir menampilkan hasil akhir praktikan dan menyediakan fitur pengunduhan laporan dalam format PDF dan Excel. Rancangan pemantauan hasil praktikum ditunjukkan pada Gambar 3.17.



Gambar 3.17 Wireframe Pemantauan Hasil Praktikum: (a) halaman penilaian, (b) halaman presensi praktikan, dan (c) halaman nilai akhir praktikan

Rancangan tampilan sistem disusun berdasarkan kebutuhan setiap peran pengguna. Praktikan memperoleh akses terhadap informasi praktikum dan data pribadi, asisten mengelola presensi serta penilaian, sedangkan koordinator mengelola seluruh data praktikum dan hasil evaluasi algoritma. *Wireframe* tersebut menjadi acuan dalam pengembangan antarmuka agar tampilan sistem tetap terarah, konsisten, dan sesuai dengan kebutuhan fungsional.

3.6 Perancangan Pengujian

Perancangan pengujian dalam penelitian ini bertujuan untuk memastikan bahwa sistem yang dibangun dapat berjalan dengan baik serta mengevaluasi performa algoritma yang digunakan. Proses pengujian terdiri dari dua bagian, yaitu pengujian fungsional sistem dan pengujian kinerja algoritma. Pengujian fungsional dilakukan menggunakan metode *Black Box Testing* untuk memastikan setiap fitur sistem bekerja sesuai dengan

kebutuhan pengguna. Sementara itu, pengujian kinerja algoritma dilakukan untuk mengetahui performa algoritma pengurutan dan pencarian dengan melihat parameter seperti waktu eksekusi, penggunaan memori, serta respons sistem saat memproses data dengan jumlah yang berbeda.

3.6.1 Pengujian Sistem

Pengujian sistem dilakukan untuk memastikan seluruh fitur yang tersedia pada sistem dapat berfungsi dengan baik sesuai dengan kebutuhan pengguna. Metode yang digunakan adalah *Black Box Testing*, yaitu metode pengujian yang berfokus pada hubungan antara data masukan dan keluaran tanpa melihat struktur atau kode program di dalam sistem.

Berikut merupakan pengujian sistem yang dilakukan menggunakan metode *Black Box Testing*:

1. Pengujian Masuk (*Login*)

Pengujian masuk pada Tabel 3.1 dilakukan untuk memastikan bahwa sistem mampu memverifikasi kredensial pengguna secara benar sesuai dengan peran yang dimiliki, serta menolak akses apabila data yang dimasukkan tidak benar atau tidak lengkap.

Tabel 3.1 Pengujian Masuk

No	Bagian Pengujian	Nama Pengujian	Hasil yang Diharapkan	Hasil Pengujian
1	Halaman utama (<i>landing page</i>)	Menampilkan informasi praktikum dan tombol masuk sesuai peran	Informasi mengenai praktikum ditampilkan serta tersedia tiga tombol masuk sesuai peran pengguna	Sesuai/Tidak
2	<i>Login</i> Praktikan	Masuk dengan email dan kata sandi benar	Sistem memverifikasi kredensial dan mengarahkan ke halaman utama praktikan	Sesuai/Tidak
3	<i>Login</i> Asisten	Masuk asisten dengan akun <i>valid</i>	Sistem mengarahkan ke halaman utama asisten	Sesuai/Tidak
4	<i>Login</i> Koordinator	Masuk koordinator dengan akun <i>valid</i>	Sistem mengarahkan ke halaman utama koordinator	Sesuai/Tidak
5	<i>Login</i> Umum	Email salah, kata sandi benar	Sistem menampilkan pesan kesalahan <i>login</i>	Sesuai/Tidak
		Email benar, kata sandi salah	Sistem menolak <i>login</i> dan menampilkan notifikasi kesalahan	Sesuai/Tidak

2. Pengujian Registrasi Praktikan

Pengujian registrasi praktikan pada Tabel 3.2 dilakukan untuk memastikan bahwa sistem mampu memproses pendaftaran akun baru secara benar dengan menerapkan validasi data sesuai ketentuan. Pengujian ini bertujuan untuk mencegah kesalahan *input*, duplikasi akun, serta memastikan data praktikan tersimpan dengan lengkap dan konsisten.

Tabel 3.2 Pengujian Registrasi Praktikan

No	Bagian Pengujian	Nama Pengujian	Hasil yang Diharapkan	Hasil Pengujian
1	Registrasi Praktikan	<i>Input</i> semua data <i>valid</i>	Akun praktikan berhasil dibuat dan tersimpan di sistem	Sesuai/Tidak
		<i>Password</i> dan ulangi <i>password</i> tidak sama	Sistem menolak registrasi dan menampilkan pesan kesalahan	Sesuai/Tidak
		Email dan NIM sudah terdaftar	Sistem menolak registrasi dan menampilkan notifikasi	Sesuai/Tidak
		Salah satu <i>field</i> kosong	Sistem menampilkan peringatan <i>input</i> wajib diisi	Sesuai/Tidak
		Nomor ponsel bukan angka	Sistem menolak <i>input</i> dan menampilkan pesan validasi	Sesuai/Tidak

3. Pengujian Fitur Praktikan

Pengujian fitur praktikan pada Tabel 3.3 dilakukan untuk memastikan bahwa seluruh fungsi yang dapat diakses oleh praktikan berjalan sesuai dengan rancangan antarmuka, meliputi akses informasi nilai, kelompok praktikum, materi, serta pengelolaan profil pribadi.

Tabel 3.3 Pengujian Fitur Praktikan

No	Bagian Pengujian	Nama Pengujian	Hasil yang Diharapkan	Hasil Pengujian
1	Beranda	Lihat nilai akhir	Nilai akhir tampil	Sesuai/Tidak
		Lihat nilai kehadiran	Persentase kehadiran tampil	Sesuai/Tidak
		Lihat rata-rata nilai	Nilai rata-rata tampil	Sesuai/Tidak
		Lihat rincian nilai	Nilai rinci untuk setiap tes bisa di tampilan	Sesuai/Tidak
		Lihat ringkasan data diri asisten	Nama, NIM, nomor ponsel, dan identitas tampil	Sesuai/Tidak
		Lihat ringkasan data kelompok	Nama kelompok dan jumlah praktikan tampil	Sesuai/Tidak

Tabel 3.3 Pengujian Fitur Praktikan (Lanjutan)

No	Bagian Pengujian	Nama Pengujian	Hasil yang Diharapkan	Hasil Pengujian
1	Beranda	Lihat dan unduh <i>QR Code</i>	<i>QR Code</i> dapat dipindai dan diunduh	Sesuai/Tidak
2	Data Diri	Lihat data diri	Data tampil lengkap	Sesuai/Tidak
		Edit data diri	Perubahan tersimpan	Sesuai/Tidak
3	Materi	Buka materi	Sistem membuka <i>link</i> Drive	Sesuai/Tidak
		Cari materi	Materi ditemukan	Sesuai/Tidak
		Saring materi	Materi tersaring berdasarkan kategori	Sesuai/Tidak

4. Pengujian Fitur Asisten

Pengujian fitur asisten pada Tabel 3.4 dilakukan untuk memastikan bahwa asisten dapat mengelola kegiatan praktikum sesuai kewenangannya, meliputi pemantauan kelompok, pengelolaan presensi, serta proses penilaian.

Tabel 3.4 Pengujian Fitur Asisten

No	Bagian Pengujian	Nama Pengujian	Hasil yang Diharapkan	Hasil Pengujian
1	Beranda	Lihat ringkasan data diri asisten	Nama, NIM, nomor ponsel, dan identitas tampil	Sesuai/Tidak
		Lihat ringkasan data kelompok	Nama kelompok dan jumlah praktikan tampil	Sesuai/Tidak
		Lihat <i>pie chart</i> pertemuan	Persentase pertemuan tampil	Sesuai/Tidak
		Lihat persentase rata-rata nilai	Rata-rata nilai tampil	Sesuai/Tidak
		Lihat jumlah alpa	Jumlah alpa tampil	Sesuai/Tidak
		Lihat praktikan belum tes	Data praktikan tampil	Sesuai/Tidak
2	Presensi	Input presensi	Data tersimpan	Sesuai/Tidak
		Saring berdasarkan pertemuan	Data tersaring	Sesuai/Tidak
		<i>Scan QR Code</i>	<i>QR Code</i> dapat terpindai	Sesuai/Tidak
3	Penilaian	Lihat nilai	Data nilai tampil	Sesuai/Tidak
		Saring berdasarkan jenis tes	Data tes tersaring	Sesuai/Tidak
4	Beri Nilai	<i>Input</i> nilai	Nilai tersimpan	Sesuai/Tidak
5	Materi	Buka materi	Sistem membuka <i>link</i> Drive	Sesuai/Tidak
		Cari materi	Materi ditemukan	Sesuai/Tidak
		Saring materi	Materi tersaring berdasarkan kategori	Sesuai/Tidak

5. Pengujian Koordinator

Pengujian fitur koordinator pada Tabel 3.5 dilakukan untuk memastikan bahwa seluruh fungsi manajemen praktikum dapat berjalan sesuai dengan peran koordinator sebagai pengelola utama sistem, termasuk pengaturan periode, data pengguna, kategori penilaian, serta rekapitulasi nilai.

Tabel 3.5 Pengujian Koordinator

No	Bagian Pengujian	Nama Pengujian	Hasil yang Diharapkan	Hasil Pengujian
1	Beranda	Lihat dan unduh <i>QR Code</i>	<i>QR Code</i> dapat dipindai dan diunduh	Sesuai/Tidak
2	Data Diri	Lihat data diri	Data tampil lengkap	Sesuai/Tidak
		Edit data diri	Perubahan tersimpan	Sesuai/Tidak
3	Beranda Periode	Lihat jumlah kelompok	Jumlah kelompok tampil	Sesuai/Tidak
		Lihat ringkasan periode	Jumlah praktikan dan asisten tampil	Sesuai/Tidak
		Lihat jumlah pertemuan	Jumlah pertemuan tampil	Sesuai/Tidak
		Lihat grafik kelulusan	Grafik kelulusan tampil	Sesuai/Tidak
4	Materi	Tambah materi (<i>link Drive</i>)	Materi tersimpan dan dapat dibuka	Sesuai/Tidak
		Edit materi	Perubahan tersimpan	Sesuai/Tidak
		Hapus materi	Materi terhapus dari sistem	Sesuai/Tidak
		Buka materi	Sistem membuka <i>link Drive</i>	Sesuai/Tidak
		Cari materi	Materi ditemukan	Sesuai/Tidak
		Saring materi	Materi tersaring berdasarkan kategori	Sesuai/Tidak
5	Koordinator – Data Asisten	Tambah data asisten	Data asisten tersimpan dan ditampilkan	Sesuai/Tidak
		Mengubah data asisten	Data tersimpan dan memengaruhi akses sistem	Sesuai/Tidak
6	Koordinator – Data Praktikan	Lihat data praktikan	Data praktikan ditampilkan	Sesuai/Tidak
7	Koordinator – Data Kelompok	Tambah kelompok praktikum	Data kelompok tersimpan dan tampil sesuai konfigurasi	Sesuai/Tidak
		Lihat detail kelompok dan kelola asisten dalam kelompok	Detail kelompok ditampilkan dan data asisten dapat tersimpan	Sesuai/Tidak
8	Koordinator – Penilaian	Pilih jenis tes	Data nilai tampil sesuai tes terpilih	Sesuai/Tidak
		Saring dan urutkan nilai	Data nilai tersaring dan terurut	Sesuai/Tidak

Tabel 3.5 Pengujian Koordinator (Lanjutan)

No	Bagian Pengujian	Nama Pengujian	Hasil yang Diharapkan	Hasil Pengujian
9	Koordinator – Presensi Praktikan	Lihat presensi per pertemuan	Data presensi tampil	Sesuai/Tidak
		Saring berdasarkan pertemuan	Data presensi sesuai pertemuan	Sesuai/Tidak
10	Kategori Penilaian	Tambah kategori nilai	Kategori tersimpan	Sesuai/Tidak
		<i>Input</i> bobot nilai	Sistem memvalidasi total bobot	Sesuai/Tidak
		Total bobot > 100%	Sistem menolak dan meminta perbaikan	Sesuai/Tidak
		Edit bobot	Nilai bobot diperbarui	Sesuai/Tidak
11	Koordinator – Rekap Nilai	Lihat rekap nilai akhir	Data nilai akhir tampil	Sesuai/Tidak
		Unduh nilai akhir (PDF atau Excel)	<i>File</i> berhasil diunduh	Sesuai/Tidak

6. Pengujian Navigasi dan Tombol Umum

Pengujian navigasi dan tombol umum pada Tabel 3.6 dilakukan untuk memastikan bahwa seluruh komponen navigasi sistem bekerja dengan baik, termasuk pengelolaan sesi pengguna dan pembatasan hak akses berdasarkan peran.

Tabel 3.6 Pengujian Navigasi dan Tombol Umum

No	Bagian Pengujian	Nama Pengujian	Hasil yang Diharapkan	Hasil Pengujian
1	Sistem	Tombol keluar (<i>logout</i>)	Sistem mengakhiri sesi dan kembali ke halaman utama sistem (<i>landing page</i>)	Sesuai/Tidak
2	Sistem	Navigasi antar halaman	Perpindahan halaman berjalan tanpa kesalahan	Sesuai/Tidak
3	Sistem	Pembatasan hak akses peran	Pengguna tidak dapat mengakses halaman peran lain	Sesuai/Tidak
4	Sistem	Tombol kembali (<i>back</i>)	Halaman tetap menampilkan data yang sesuai	Sesuai/Tidak
5	Sistem	Akses halaman tanpa <i>login</i>	Sistem mengarahkan ke halaman utama (<i>landing page</i>)	Sesuai/Tidak
6	Sistem	<i>Refresh</i> halaman setelah <i>login</i>	<i>Session</i> tetap aktif	Sesuai/Tidak
7	Hak Akses	Pengguna mencoba membuka halaman yang tidak sesuai <i>role</i>	Sistem menolak akses	Sesuai/Tidak

7. Pengujian Algoritma *Sequential Search*

Pengujian algoritma *Sequential Search* pada Tabel 3.7 dilakukan untuk memastikan bahwa sistem mampu mencari data praktikan sesuai kriteria yang ditentukan. Selain itu, pengujian ini juga bertujuan untuk mengevaluasi respons sistem terhadap berbagai kondisi pencarian, termasuk data yang tersedia, data yang tidak ditemukan, *input* kosong, *dataset* kosong, serta kasus dengan hanya satu data. Dengan demikian, perilaku sistem dapat dipastikan tetap konsisten dan tangguh terhadap *edge cases* (kasus batas) yang sering muncul dalam penggunaan nyata.

Tabel 3.7 Pengujian Algoritma *Sequential Search*

No	Bagian Pengujian	Nama Pengujian	Hasil yang Diharapkan	Hasil Pengujian
1	<i>Sequential Search</i>	Cari praktikan yang ada	Data ditemukan dan ditampilkan	Sesuai/Tidak
		Cari praktikan yang tidak ada	Sistem menampilkan "Data tidak ditemukan"	Sesuai/Tidak
		Search dengan <i>keyword</i> kosong	Sistem menampilkan semua data	Sesuai/Tidak
		Pencarian pada <i>dataset</i> kosong (0 <i>record</i>)	Sistem menampilkan pesan "Data tidak ditemukan"	Sesuai/Tidak
		Pencarian pada <i>dataset</i> dengan 1 <i>record</i>	Data tersebut ditemukan jika sesuai, atau pesan tidak ditemukan jika tidak cocok	Sesuai/Tidak
		Pencarian dengan data duplikat	Semua data duplikat yang sesuai ditampilkan	Sesuai/Tidak

8. Pengujian Algoritma *Merge Sort*

Pengujian algoritma *Merge Sort* pada Tabel 3.8 dilakukan untuk memverifikasi bahwa sistem mengurutkan data secara benar dan stabil sesuai urutan yang diinginkan (*ascending* maupun *descending*). Pengujian ini mencakup berbagai kondisi data, seperti data acak, data sudah terurut, data kosong, hanya satu *record*, serta data dengan nilai duplikat. Memastikan algoritma tetap berfungsi dengan baik, tidak *error*, dan menjaga keakuratan hasil meskipun menghadapi *edge cases* sehingga algoritma *Merge Sort* dapat digunakan sebagai metode pengurutan utama dalam sistem.

Tabel 3.8 Pengujian Algoritma *Merge Sort*

No	Bagian Pengujian	Nama Pengujian	Hasil yang Diharapkan	Hasil Pengujian
1	<i>Merge Sort</i>	Urutkan data acak <i>ascending</i>	Data terurut dari kecil ke besar	Sesuai/Tidak
		Urutkan data acak <i>descending</i>	Data terurut dari besar ke kecil	Sesuai/Tidak
		Urutkan data yang sudah terurut	Data tetap terurut, tidak error	Sesuai/Tidak
		Urutkan data dengan nilai duplikat	Data terurut, duplikat tetap ada (<i>stable sort</i>)	Sesuai/Tidak
		Urutkan <i>dataset</i> kosong (0 <i>record</i>)	Tidak error, hasil tetap kosong atau menampilkan pesan sesuai	Sesuai/Tidak
		Urutkan <i>dataset</i> dengan hanya 1 <i>record</i>	Data tetap sama (tidak berubah), tidak <i>error</i>	Sesuai/Tidak

3.6.2 Evaluasi Kinerja Algoritma

Metode evaluasi kinerja dalam penelitian ini digunakan untuk menilai performa algoritma *Merge Sort* dan *Sequential Search* yang diterapkan pada sistem praktikum berbasis web dalam pengelolaan data praktikan Fisika Dasar Fakultas Teknik Universitas Mulawarman. Evaluasi kinerja dilakukan secara terkontrol agar hasil pengukuran bersifat objektif, konsisten, dan dapat diulang pada kondisi yang sama. Aspek yang dianalisis meliputi waktu eksekusi, penggunaan memori, jumlah operasi internal, dan perubahan kinerja berdasarkan ukuran dataset. Evaluasi kinerja dalam penelitian ini dilakukan secara terstruktur berdasarkan:

a. Objek Evaluasi

Objek evaluasi mencakup proses pengurutan dan pencarian data praktikan pada lapisan aplikasi. *Merge Sort* digunakan untuk mengurutkan data berdasarkan program studi dan NIM, sedangkan *Sequential Search* digunakan untuk mencari data berdasarkan NIM atau nama praktikan. Penelitian ini juga menggunakan MySQL *ORDER BY* sebagai *baseline* pengurutan serta MySQL *WHERE/LIKE* sebagai *baseline* pencarian. Perbandingan tersebut digunakan untuk mengetahui perbedaan kinerja antara pemrosesan data pada lapisan aplikasi dan basis data.

b. Parameter Evaluasi

Penelitian ini menggunakan beberapa parameter untuk mengukur efisiensi setiap metode. Parameter evaluasi mencakup waktu eksekusi, penggunaan memori, dan operasi internal algoritma. Parameter yang digunakan dalam pengujian disajikan pada Tabel 3.9.

Tabel 3.9 Parameter Evaluasi Algoritma

No.	Metode	Parameter Evaluasi	Keterangan
1	<i>Merge Sort</i>	Waktu eksekusi	Mengukur durasi proses pengurutan dalam milidetik menggunakan <code>microtime(true)</code> .
		Jumlah perbandingan	Menghitung proses perbandingan (<code>comparison</code>) antar elemen selama pembagian dan penggabungan data.
		Jumlah proses penggabungan data	Menghitung penggabungan data (<i>merge</i>) yang dicatat setiap kali dua bagian <i>array</i> digabungkan kembali dalam proses pengurutan.
		Penggunaan memori	Mengukur penggunaan memori selama proses pengurutan menggunakan fungsi pengukuran memori pada PHP <code>memory_get_peak_usage()</code> .
2	<i>Sequential Search</i>	Waktu eksekusi	Mengukur durasi proses pencarian dalam milidetik menggunakan <code>microtime(true)</code> .
		Jumlah iterasi	Menghitung banyaknya elemen yang diperiksa selama proses pencarian.
		Jumlah perbandingan	Menghitung proses perbandingan antara kata kunci dan elemen data.
		Penggunaan memori	Mengukur penggunaan memori (<code>peak memory usage</code>) selama proses pencarian.
3	<i>MySQL baseline</i>	Waktu eksekusi	Mengukur durasi pemrosesan MySQL <code>ORDER BY</code> dan <code>WHERE/LIKE</code> .
		Penggunaan memori	Mengukur penggunaan memori ketika <i>query</i> dijalankan dan hasil dikembalikan ke aplikasi.
		Jumlah hasil	Menghitung jumlah data yang dikembalikan oleh <i>query</i> .
		Hasil <code>EXPLAIN</code>	Menunjukkan cara MySQL menjalankan <i>query</i> , termasuk penggunaan indeks dan perkiraan jumlah baris yang diperiksa.

Sistem mencatat seluruh parameter pada setiap iterasi berdasarkan metode, ukuran *dataset*, dan skenario pengujian. Pencatatan tersebut memungkinkan perubahan waktu eksekusi, penggunaan memori, jumlah iterasi, dan jumlah perbandingan dianalisis secara terstruktur.

c. Skenario *Dataset* dan Variasi Ukuran Data.

Penelitian ini menggunakan data praktikan yang tersimpan dalam basis data MySQL dan data sintetis yang dibuat secara terkontrol. Data praktikan digunakan sebagai dasar pengembangan serta pengujian fungsi sistem, sedangkan data sintetis digunakan dalam evaluasi kinerja agar ukuran dan kondisi data dapat diatur secara konsisten.

Ukuran *dataset* sintetis terdiri atas 100, 500, 1.000, dan 5.000 entri. Variasi tersebut digunakan untuk mengamati perubahan kinerja setiap metode ketika jumlah data bertambah. Sistem menggunakan dataset yang sama pada *Merge Sort* dan MySQL *ORDER BY*, serta pada *Sequential Search* dan MySQL *WHERE/LIKE*, agar hasil perbandingan diperoleh dari kondisi pengujian yang setara.

Pengujian *Merge Sort* menggunakan empat skenario yang mewakili kondisi pengurutan pada sistem. Skenario tersebut meliputi data yang sudahurut, data yang memiliki nilai duplikat, data acak yang diurutkan secara menaik, dan data acak yang diurutkan secara menurun. Skenario pengujian *Merge Sort* disajikan pada Tabel 3.10.

Tabel 3.10 Skenario Pengujian *Merge Sort*

No.	Skenario	Karakteristik Data	Tujuan Pengujian
1	Sudah Urut	Data telah tersusun secara menaik berdasarkan kunci pengurutan	Mengukur kinerja <i>Merge Sort</i> pada data yang telah terstruktur
2	Duplikat	Dataset memiliki beberapa nilai pengurutan yang sama	Menilai kemampuan algoritma mempertahankan data duplikat selama pengurutan
3	<i>Random Asc</i>	Data disusun secara acak dan diurutkan secara menaik	Mengukur kinerja pengurutan pada kondisi data umum
4	<i>Random Desc</i>	Data disusun secara acak dan diurutkan secara menurun	Mengukur kinerja algoritma pada arah pengurutan yang berlawanan

Pengujian algoritma *Sequential Search* pada Tabel 3.11 dilakukan dengan mempertimbangkan beberapa kemungkinan posisi elemen yang dicari dalam *dataset*. Variasi ini digunakan untuk menggambarkan kondisi pencarian yang berbeda yang dapat memengaruhi jumlah iterasi serta waktu eksekusi algoritma. Karena *Sequential Search* tidak memerlukan data dalam kondisi terurut, pengujian dapat dilakukan pada dataset dengan berbagai bentuk susunan data. Hal ini memungkinkan kinerja algoritma dianalisis pada kondisi data yang berbeda.

Tabel 3.11 Skenario Pengujian *Sequential Search*

No.	Kondisi Input Data	Karakteristik Data	Tujuan Pengujian
1	<i>Best Case</i>	Elemen yang dicari berada pada posisi pertama	Mengukur performa optimal <i>Sequential Search</i> dengan jumlah iterasi minimum
2	<i>Average Case</i>	Elemen yang dicari berada pada posisi acak dalam <i>dataset</i>	Merepresentasikan kondisi pencarian umum dan mengamati performa rata-rata algoritma
3	<i>Worst Case</i>	Elemen yang dicari berada pada posisi terakhir atau tidak ditemukan	Mengukur jumlah iterasi maksimum yang diperlukan algoritma

Data sintetis digunakan agar setiap skenario dapat dibuat dan dijalankan kembali dengan kondisi yang sama. Pendekatan tersebut menjaga konsistensi pengujian serta memudahkan perbandingan hasil antarukuran dataset.

d. Prosedur Pengujian Algoritma.

Penelitian ini membagi pengujian kinerja menjadi pengujian pengurutan dan pencarian. Setiap kombinasi metode, ukuran dataset, dan skenario menjalani 50 iterasi utama. Jumlah pengulangan tersebut digunakan untuk memperoleh nilai *mean*, *median*, dan *standard deviation* yang lebih stabil. Sistem menjalankan 10 iterasi *warm-up* sebelum 50 iterasi utama dilakukan. Hasil *warm-up* tidak digunakan dalam perhitungan statistik akhir karena tahap tersebut berfungsi mengurangi pengaruh kondisi awal, seperti pemuatan data, inisialisasi proses, dan *caching*. Setiap metode diuji menggunakan jumlah iterasi dan kondisi *dataset* yang sama agar hasil perbandingan tetap konsisten.

Sistem menganalisis hasil pengujian utama menggunakan statistik deskriptif berupa *mean*, *median*, dan *standard deviation*. Sistem juga menerapkan metode *Interquartile Range* (IQR) untuk mendeteksi hasil pengujian yang berada di luar batas kewajaran. Perhitungan IQR menggunakan rumus berikut:

$$\begin{aligned} \text{IQR} &= Q3 - Q1 \\ \text{Batas bawah} &= Q1 - 1.5 \times \text{IQR} \\ \text{Batas atas} &= Q3 + 1.5 \times \text{IQR} \end{aligned}$$

Hasil yang berada di bawah batas bawah atau di atas batas atas ditandai sebagai *outlier* dan tidak digunakan dalam perhitungan statistik akhir. Data mentah tetap disimpan

sebagai dokumentasi hasil pengujian, sedangkan data bersih setelah proses IQR digunakan dalam tabel statistik dan grafik perbandingan kinerja.

e. Prosedur Pengujian Algoritma *Merge Sort*.

Pengujian *Merge Sort* dilakukan pada setiap ukuran *dataset* dan skenario pengurutan dengan kondisi awal yang disiapkan secara konsisten agar waktu eksekusi, penggunaan memori, jumlah perbandingan, dan ketepatan hasil dapat dicatat pada setiap iterasi. Rangkuman prosedur pengujian *Merge Sort* disajikan pada Tabel 3.12. Sistem menjalankan pengujian *Merge Sort* melalui langkah-langkah berikut:

1. Sistem memuat *dataset* sintetis sesuai ukuran data yang sedang diuji.
2. Sistem membuat salinan data agar setiap iterasi menggunakan kondisi awal yang sama.
3. Sistem menyiapkan susunan data berdasarkan skenario sudahurut, duplikat, *Random Asc*, atau *Random Desc*.
4. Sistem menginisialisasi ulang penghitung jumlah perbandingan.
5. Sistem mencatat waktu dan penggunaan memori sebelum pengurutan dimulai.
6. Sistem menjalankan *Merge Sort* dan mencatat setiap proses perbandingan.
7. Sistem mencatat waktu akhir dan penggunaan memori setelah pengurutan selesai.
8. Sistem memeriksa ketepatan hasil pengurutan dan menyimpan hasil pengujian ke tabel *log*.

Tabel 3.12 Prosedur Pengujian Algoritma *Merge Sort*

No.	Ukuran <i>Dataset</i>	Kondisi Input Data	Parameter Evaluasi	Hasil
1	100	Sudah Urut, Duplikat, <i>Random Asc</i> , <i>Random Desc</i>	Waktu eksekusi, jumlah perbandingan, penggunaan memori	Tercatat
2	500	Sudah Urut, Duplikat, <i>Random Asc</i> , <i>Random Desc</i>	Waktu eksekusi, jumlah perbandingan, penggunaan memori	Tercatat
3	1.000	Sudah Urut, Duplikat, <i>Random Asc</i> , <i>Random Desc</i>	Waktu eksekusi, jumlah perbandingan, penggunaan memori	Tercatat
4	5.000	Sudah Urut, Duplikat, <i>Random Asc</i> , <i>Random Desc</i>	Waktu eksekusi, jumlah perbandingan, penggunaan memori	Tercatat

Tabel 3.12 menunjukkan bahwa seluruh ukuran dataset diuji menggunakan empat skenario pengurutan yang sama. Keceragaman tersebut digunakan agar perubahan waktu

eksekusi dan penggunaan memori dapat dibandingkan berdasarkan ukuran serta kondisi data.

f. Prosedur Pengujian *Sequential Search*.

Pengujian *Sequential Search* dilakukan pada setiap ukuran dataset dan skenario pencarian dengan kata kunci yang telah ditentukan agar waktu eksekusi, penggunaan memori, jumlah iterasi, jumlah perbandingan, dan ketepatan hasil dapat dicatat secara konsisten. Rangkuman prosedur pengujian *Sequential Search* disajikan pada Tabel 3.13. Sistem menjalankan pengujian *Sequential Search* melalui langkah-langkah berikut:

1. Sistem memuat dataset sintetis sesuai ukuran yang sedang diuji.
2. Sistem menentukan kata kunci berdasarkan skenario *Keyword* Ada, Tidak Ada, Kosong, atau Duplikat.
3. Sistem menginisialisasi ulang penghitung jumlah iterasi dan jumlah perbandingan.
4. Sistem mencatat waktu dan penggunaan memori sebelum pencarian dimulai.
5. Sistem memeriksa setiap data secara berurutan berdasarkan NIM atau nama.
6. Sistem mencatat jumlah iterasi dan perbandingan selama proses pencarian.
7. Sistem mencatat waktu akhir dan penggunaan memori setelah proses selesai.
8. Sistem memeriksa ketepatan hasil pencarian dan menyimpan hasil pengujian ke tabel *log*.

Tabel 3.13 Prosedur Pengujian Algoritma *Sequential Search*

No.	Ukuran Dataset	Skenario Pencarian	Parameter Evaluasi	Hasil
1	100	<i>Best, Average, Worst</i>	Waktu eksekusi, jumlah iterasi, jumlah perbandingan, penggunaan memori	Tercatat
2	500	<i>Best, Average, Worst</i>	Waktu eksekusi, jumlah iterasi, jumlah perbandingan, penggunaan memori	Tercatat
3	1.000	<i>Best, Average, Worst</i>	Waktu eksekusi, jumlah iterasi, jumlah perbandingan, penggunaan memori	Tercatat
4	5.000	<i>Best, Average, Worst</i>	Waktu eksekusi, jumlah iterasi, jumlah perbandingan, penggunaan memori	Tercatat

Tabel 3.13 menunjukkan bahwa *Sequential Search* diuji menggunakan skenario dan jumlah pengulangan yang sama pada setiap ukuran *dataset*. Prosedur tersebut digunakan untuk melihat pengaruh penambahan data terhadap waktu pencarian, penggunaan memori, jumlah iterasi, dan jumlah perbandingan. Pengujian dilakukan secara konsisten agar hasil pada setiap ukuran dataset dapat dibandingkan secara objektif.

3.6.3 Teknik Pengukuran

Penelitian ini menggunakan pendekatan pengukuran pada sisi *server* untuk memperoleh data performa algoritma secara lebih objektif dan konsisten. Proses pengukuran difokuskan pada dua indikator utama, yaitu waktu eksekusi dan penggunaan memori selama algoritma dijalankan. Seluruh hasil pengukuran kemudian dicatat secara terstruktur ke dalam basis data agar proses evaluasi dapat dilakukan kembali serta memungkinkan analisis kuantitatif terhadap performa sistem.

1. Pengukuran *Server-Side* (PHP / Laravel). Pengukuran pada sisi *server* dilakukan untuk mendapatkan data kinerja algoritma secara langsung dari proses komputasi inti sistem, sehingga hasil pengukuran tidak dipengaruhi oleh faktor eksternal seperti tampilan antarmuka pengguna maupun kondisi jaringan. Parameter yang diamati dalam pengukuran ini meliputi:
 - a) Waktu eksekusi algoritma, yang diperoleh dengan mencatat waktu sebelum dan sesudah algoritma dijalankan menggunakan fungsi `microtime(true)`, kemudian selisih waktu tersebut dikonversikan ke dalam satuan milidetik.
 - b) Penggunaan memori, yang diukur melalui fungsi `memory_get_peak_usage(true)` untuk mengetahui konsumsi memori maksimum selama proses eksekusi algoritma berlangsung.
 - c) Operasi internal algoritma, yaitu pencatatan jumlah perbandingan pada algoritma *Merge Sort* serta jumlah iterasi dan perbandingan pada algoritma *Sequential Search*, yang direkam menggunakan variabel penghitung internal.
 - d) Waktu eksekusi *query baseline*, yang diukur dengan mencatat waktu sebelum dan sesudah pemanggilan fungsi `DB::select()`. Hasilnya kemudian dianalisis menggunakan perintah seperti `EXPLAIN` untuk membandingkan kinerja algoritma pada sisi aplikasi dengan mekanisme pemrosesan standar pada SQL.
2. Alat Bantu dan Otomatisasi Pengujian. Penelitian ini memanfaatkan beberapa alat bantu serta mekanisme otomatisasi untuk memastikan proses pengujian berjalan secara konsisten dan terkontrol. Otomatisasi diterapkan guna meminimalkan kesalahan manual serta mempercepat proses pengumpulan data selama eksperimen berlangsung.

- a) Skrip otomatis berbasis *Artisan Command* atau PHPUnit digunakan untuk menjalankan berbagai skenario pengujian secara berulang.
- b) Seluruh hasil pengukuran dicatat ke dalam `performance_logs` yang memuat beberapa atribut, seperti `timestamp`, `dataset_size`, `sort_time`, `search_time`, dan `memory_usage`. Data hasil pengujian juga dapat diekspor ke dalam berkas CSV untuk keperluan dokumentasi serta memungkinkan proses replikasi pengujian di masa mendatang.

3.6.4 Lingkungan Pengujian

Pengujian sistem dilakukan pada lingkungan lokal untuk memastikan proses pengukuran kinerja algoritma dan sistem berlangsung secara konsisten, terkontrol, serta meminimalkan pengaruh faktor eksternal. Penggunaan lingkungan lokal memungkinkan latensi jaringan dapat dihindari sehingga hasil pengujian lebih mencerminkan performa komputasi sistem yang sebenarnya. Selain itu, spesifikasi perangkat keras dan perangkat lunak yang digunakan dijaga tetap sama selama proses pengujian agar hasil evaluasi dapat direplikasi secara akurat.

1. Spesifikasi Perangkat Keras: Sistem dijalankan pada sebuah laptop dengan sistem operasi Windows 11 64-bit, *prosesor* 12th Gen Intel(R) Core(TM) i5, memori RAM 12 GB, serta media penyimpanan SSD yang digunakan untuk menjalankan *server* lokal dan basis data selama proses pengujian berlangsung.
2. Tumpukan Perangkat Lunak: Lingkungan pengujian menggunakan web server lokal berbasis Apache, bahasa pemrograman PHP versi 8.2.12, *framework* Laravel 12.57.0, serta sistem manajemen basis data MySQL versi 8.x. Akses ke sistem dilakukan melalui *browser Google Chrome* pada perangkat desktop sebagai media utama pengujian sisi klien.
3. Kondisi Jaringan dan Beban Sistem: Seluruh pengujian dilakukan pada lingkungan *localhost* sehingga tidak dipengaruhi oleh latensi jaringan. Selama proses pengujian berlangsung, sistem dijalankan dalam kondisi normal tanpa adanya aplikasi berat lain yang berjalan di latar belakang agar hasil pengukuran tidak terpengaruh oleh beban komputasi tambahan.

BAB IV

HASIL DAN PEMBAHASAN

Hasil penelitian ini menyajikan implementasi sistem pengelolaan data praktikan berbasis web yang dikembangkan berdasarkan perancangan yang telah dijelaskan pada bab sebelumnya. Pembahasan mencakup proses pengelolaan data praktikan, penerapan algoritma *Merge Sort* dan *Sequential Search*, implementasi baseline MySQL, tampilan sistem, serta hasil pengujian fungsional dan evaluasi kinerja algoritma. Sistem praktikum dirancang untuk mengelola data praktikan, asisten, periode praktikum, kelompok, materi, jadwal, presensi, kategori penilaian, dan nilai akhir dalam satu basis data yang terintegrasi. Pengujian kinerja algoritma dilakukan menggunakan variasi jumlah data sebanyak 100, 500, 1.000, dan 5.000 entri guna menganalisis perubahan waktu eksekusi, penggunaan memori, serta jumlah operasi internal algoritma pada berbagai kondisi data.

4.1 Pengolahan Data

Pengolahan data menjelaskan mekanisme sistem dalam menyimpan, mengelola, dan memanfaatkan data praktikan selama proses pelaksanaan praktikum. Sebelumnya, data praktikan tersimpan pada beberapa berkas *spreadsheet* yang terpisah, seluruh data tersebut diintegrasikan ke dalam basis data MySQL sehingga proses pencarian, pengurutan, dan rekapitulasi dapat lebih mudah dilakukan. Pembahasan pada bagian ini meliputi data praktikan yang digunakan oleh sistem, data sintetis yang dimanfaatkan untuk pengujian algoritma, serta tabel *log* yang merekam hasil evaluasi *Merge Sort*, *Sequential Search*, dan *baseline MySQL*.

4.1.1 Data Praktikan pada Sistem

Sistem menyimpan data praktikan ke dalam basis data MySQL melalui tabel pengguna dan tabel praktikan. Data pengguna memuat informasi utama seperti nama, email, NIM, program studi, jenis kelamin, nomor ponsel, kata sandi, dan peran pengguna, sedangkan data praktikan memuat informasi tambahan seperti data kelompok, sesi, dan periode praktikum. Data praktikan digunakan sebagai data utama dalam proses pengelolaan praktikum pada sistem berbasis web. Data tersebut terhubung dengan asisten,

jadwal, materi, presensi, kategori penilaian, dan nilai akhir sehingga proses pencatatan, pengelolaan, dan rekapitulasi dapat dilakukan secara lebih terstruktur dibandingkan penggunaan beberapa file *spreadsheet* terpisah.

Tabel 4.1 Prosedur Pengujian Algoritma *Sequential Search*

Tahun Akademik	Nama Praktikum	Jumlah Praktikan	Keterangan
2025	Praktikum Fisika Dasar 1	382	Data praktikan tahun akademik 2025
	Praktikum Fisika Dasar 2	156	

Tabel 4.1 menyajikan distribusi data praktikan berdasarkan jenis praktikum pada tahun akademik 2025 sebagai dasar pengembangan sistem. Praktikum Fisika Dasar 1 memiliki 382 praktikan, sementara Praktikum Fisika Dasar 2 mencatat 156 praktikan. Keseluruhan data yang berjumlah 538 praktikan tersebut menunjukkan bahwa sistem perlu mengelola data dalam skala menengah secara terstruktur, terpusat, dan mudah diakses.

4.1.2 Data Sintetis untuk Pengujian Algoritma

Data sintetis berperan sebagai dataset pengujian untuk menjaga keseragaman jumlah data pada setiap skenario evaluasi algoritma. Sistem membentuk data tersebut menggunakan pola NIM numerik yang disusun secara berurutan sehingga menyerupai format NIM praktikan. Penyusunan pola tersebut memungkinkan sistem mengatur posisi data target selama proses pengujian *Sequential Search*, yang mencakup kondisi *best case*, *average case*, *worst case* ketika data ditemukan, dan *worst case* ketika data tidak ditemukan.

Sistem menggunakan *dataset* sintetis yang sama untuk menguji kinerja *Merge Sort*, *Sequential Search*, dan *baseline* MySQL agar seluruh metode dievaluasi pada kondisi data yang identik. Pengujian memanfaatkan *dataset* berukuran 100, 500, 1.000, dan 5.000 entri sehingga perubahan performa algoritma dapat diamati seiring dengan bertambahnya jumlah data. Proses pengujian memanfaatkan data *benchmark* yang dibuat secara khusus sehingga data praktikan asli tetap terjaga dan tidak terpengaruh selama evaluasi berlangsung. Sistem menyimpan data *benchmark* tersebut pada tabel yang sama

dengan data praktikan, tetapi memberikan penanda berupa pola email *eval-benchmark-*. Penanda tersebut memudahkan sistem dalam memuat, menghapus, maupun membangun kembali dataset pengujian tanpa mengganggu data operasional praktikan.

Tabel 4.2 Prosedur Pengujian Algoritma *Sequential Search*

Jumlah Data	Jenis Data	Tujuan Pengujian
100	Sintetis	Menguji performa pada data kecil
500	Sintetis	Menguji performa pada data menengah awal
1.000	Sintetis	Menguji performa pada data menengah
5.000	Sintetis	Menguji skalabilitas pada data lebih besar

Tabel 4.2 menyajikan variasi ukuran *dataset* yang digunakan dalam proses pengujian algoritma. Sistem menjalankan setiap ukuran dataset secara berulang untuk mengurangi ketergantungan hasil pada satu kali proses eksekusi. Pengulangan tersebut menghasilkan nilai rata-rata, *median*, standar deviasi, serta distribusi data pengujian yang lebih stabil.

4.1.3 Penyimpanan *Log Hasil Pengujian*

Sistem menyimpan hasil pengujian *Merge Sort*, *Sequential Search*, dan *baseline MySQL* pada tabel *performance_logs*. Data yang dicatat mencakup ukuran *dataset*, skenario pengujian, waktu eksekusi, penggunaan memori, jumlah iterasi, jumlah perbandingan, jumlah hasil pencarian, status validitas, *warm-up*, dan *outlier*. Sistem juga menyimpan hasil *EXPLAIN*, yaitu informasi tentang cara MySQL menjalankan *query*, seperti indeks yang digunakan dan perkiraan jumlah baris yang diperiksa. Hasil *EXPLAIN* membantu mengetahui apakah MySQL dapat menemukan data dengan cepat atau harus memeriksa banyak data satu per satu.

Tabel 4.3 Penyimpanan *Log Hasil Pengujian*

No.	Tabel Log	Data yang Disimpan	Fungsi
1	<code>log_sortings</code>	jumlah data, field pengurutan, arah pengurutan, waktu pemrosesan, jumlah perbandingan, penggunaan memori, dan waktu pengujian	Mencatat hasil khusus pengujian Merge Sort
2	<code>log_searches</code>	jumlah data, field pencarian, kata kunci, waktu pemrosesan, jumlah iterasi, jumlah perbandingan, penggunaan memori, dan waktu pengujian,	Mencatat hasil khusus pengujian Sequential Search

Tabel 4.3 Penyimpanan Log Hasil Pengujian (Lanjutan)

No.	Tabel Log	Data yang Disimpan	Fungsi
3	<code>performance_logs</code>	kode batch, ukuran dataset, metode, skenario, waktu eksekusi, penggunaan memori, jumlah iterasi, jumlah perbandingan, jumlah hasil, status validitas, status warm-up, status outlier, dan hasil EXPLAIN	Menggabungkan hasil Merge Sort, Sequential Search, dan baseline MySQL sebagai data perbandingan

Tabel 4.3 menyajikan struktur penyimpanan *log* hasil pengujian yang diterapkan dalam sistem. Tabel *log_sortings* mencatat hasil pengujian *Merge Sort*, sedangkan tabel *log_searches* menyimpan hasil pengujian *Sequential Search*. Tabel *performance_logs* menghimpun hasil kedua algoritma serta *baseline* MySQL menggunakan *ORDER BY* dan *WHERE/LIKE* sebagai dasar perbandingan kinerja. Pembagian fungsi tersebut memudahkan proses analisis waktu eksekusi, penggunaan memori, jumlah iterasi, jumlah perbandingan, validitas hasil, serta data *outlier* berdasarkan setiap metode pengujian.

4.2 Implementasi Proses Algoritma

Sistem menerapkan *Merge Sort* untuk mengurutkan data praktikan berdasarkan NIM, sedangkan *Sequential Search* digunakan untuk mencari data berdasarkan NIM. Sistem juga menggunakan MySQL *ORDER BY* dan *WHERE/LIKE* sebagai *baseline* pembanding. Proses pengujian memakai *dataset* sintetis berukuran 100, 500, 1.000, dan 5.000 entri, lalu sistem mencatat waktu eksekusi, penggunaan memori, jumlah iterasi, jumlah perbandingan, validitas hasil, serta *outlier* pada tabel *performance_logs*.

4.2.1 Implementasi Merge Sort

Sistem menggunakan *Merge Sort* untuk mengurutkan data praktikan berdasarkan program studi dan NIM. Proses pengurutan menempatkan program studi secara alfabetis, kemudian mengurutkan praktikan dalam program studi yang sama berdasarkan tiga angka terakhir NIM. Pengguna dapat memilih arah pengurutan menaik atau menurun melalui parameter `sort_direction`.

Method *praktikan()* mengambil seluruh data yang telah melewati proses penyaringan berdasarkan periode praktikum, program studi, kelompok, dan sesi. Sistem selanjutnya menjalankan *Sequential Search* apabila pengguna memasukkan kata kunci

pencarian. *Merge Sort* kemudian mengurutkan hasil pencarian sebelum sistem membaginya ke dalam halaman. Pemanggilan proses pencarian dan pengurutan tersebut ditunjukkan pada *Source Code 4.1*.

```
$semuaPraktikans = $query->get();

$hasilSearch = $this->sequentialSearchPraktikanKoordinator(
    $semuaPraktikans,
    $request->search
);

$hasilSort = $this->mergeSortPraktikanByNim(
    $hasilSearch,
    $sortDirection
);

$praktikans = $this->paginateCollectionKoordinator(
    $hasilSort,
    $request,
    400
);
```

Source Code 4.1 Pemanggilan Sequential Search dan Merge Sort

Source Code 4.1 menunjukkan urutan pengolahan data praktikan pada lapisan aplikasi. Sistem mengambil data dari basis data, menjalankan proses pencarian, mengurutkan hasil pencarian, lalu membagi data yang telah diurutkan ke dalam halaman. Urutan tersebut memastikan bahwa data yang ditampilkan telah sesuai dengan kata kunci dan arah pengurutan yang dipilih pengguna.

Fungsi *mergeSortPraktikanByNim()* menjalankan proses pembagian data secara rekursif. Fungsi tersebut menghentikan proses ketika jumlah data hanya terdiri atas satu elemen. Sistem membagi kumpulan data yang lebih besar menjadi bagian kiri dan kanan, mengurutkan kedua bagian tersebut, lalu menggabungkannya kembali. Implementasi proses pembagian data ditunjukkan pada *Source Code 4.2*.

```
private function mergeSortPraktikanByNim(
    $praktikans,
    string $direction = 'asc'
) {
    $praktikans = collect($praktikans)->values();

    if ($praktikans->count() <= 1) {
        return $praktikans;
    }

    $mid = intval($praktikans->count() / 2);
```

Source Code 4.2 Proses Pembagian Data pada Merge Sort

```

    $left = $this->mergeSortPraktikanByNim(
        $praktikans->slice(0, $mid)->values(),
        $direction
    );

    $right = $this->mergeSortPraktikanByNim(
        $praktikans->slice($mid)->values(),
        $direction
    );

    return $this->mergePraktikanByNim(
        $left,
        $right,
        $direction
    );
}

```

Source Code 4.2 Proses Pembagian Data pada *Merge Sort* (Lanjutan)

Fungsi tersebut menentukan posisi tengah menggunakan *intdiv()*. Sistem membentuk bagian kiri dari indeks awal sampai sebelum posisi tengah, sedangkan bagian kanan memuat data dari posisi tengah sampai indeks terakhir. Proses yang sama terus berjalan sampai setiap bagian hanya memiliki satu data.

Fungsi *mergePraktikanByNim()* menggabungkan bagian kiri dan kanan dalam kondisi terurut. Sistem membandingkan elemen pertama dari kedua bagian, memilih elemen yang memenuhi ketentuan pengurutan, lalu memasukkannya ke dalam hasil. Proses berlanjut sampai seluruh data dari kedua bagian selesai digabungkan. Implementasi proses penggabungan ditunjukkan pada *Source Code 4.3*.

```

private function mergePraktikanByNim(
    $left,
    $right,
    string $direction = 'asc'
) {
    $result = collect();

    $i = 0;
    $j = 0;

    while ($i < $left->count() && $j < $right->count()) {
        if ($this->ambilKiriUntukSortingPraktikan(
            $left[$i],
            $right[$j],
            $direction
        )) {
            $result->push($left[$i]);
            $i++;
        } else {

```

Source Code 4.3 Proses Penggabungan Data pada *Merge Sort*

```

        $result->push($right[$j]);
        $j++;
    }
}

while ($i < $left->count()) {
    $result->push($left[$i]);
    $i++;
}

while ($j < $right->count()) {
    $result->push($right[$j]);
    $j++;
}

return $result->values();
}

```

Source Code 4.3 Proses Penggabungan Data pada Merge Sort (Lanjutan)

Fungsi *ambilKiriUntukSortingPraktikan()* menentukan elemen yang lebih dahulu dimasukkan ke dalam hasil penggabungan. Sistem membandingkan program studi secara alfabetis. Sistem kemudian membandingkan tiga angka terakhir NIM apabila kedua praktikan berasal dari program studi yang sama. Arah pengurutan menaik menghasilkan susunan nomor dari kecil ke besar, sedangkan arah menurun menghasilkan susunan dari besar ke kecil.

Ketiga *source code* tersebut menunjukkan alur utama *Merge Sort* pada sistem. *Source Code 4.1* menunjukkan pemanggilan algoritma, *Source Code 4.2* menunjukkan pembagian data secara rekursif, dan *Source Code 4.3* menunjukkan penggabungan kembali setiap bagian dalam kondisi terurut. Sistem menghasilkan keluaran *Merge Sort* melalui data sintetis yang disusun dalam kondisi acak. Tabel 4.4 menunjukkan contoh perubahan urutan data sebelum dan sesudah proses pengurutan berdasarkan NIM secara menaik.

Tabel 4.3 Contoh Input dan Output Merge Sort

No.	Data Input Sebelum Diurutkan	Output Setelah Merge Sort
1	9999000005 – Benchmark Praktikan 00005	9999000005 – Benchmark Praktikan 00005
2	9999000002 – Benchmark Praktikan 00002	9999000002 – Benchmark Praktikan 00002
3	9999000004 – Benchmark Praktikan 00004	9999000004 – Benchmark Praktikan 00004
4	9999000001 – Benchmark Praktikan 00001	9999000001 – Benchmark Praktikan 00001
5	9999000003 – Benchmark Praktikan 00003	9999000003 – Benchmark Praktikan 00003

Tabel 4.4 menunjukkan bahwa *Merge Sort* berhasil mengubah data yang sebelumnya tersusun secara acak menjadi data yang terurut berdasarkan NIM. Hasil tersebut membuktikan bahwa proses pembagian dan penggabungan data menghasilkan urutan yang sesuai dengan pilihan pengguna.

4.2.2 Implementasi *Sequential Search*

Sistem menerapkan *Sequential Search* untuk mencari data praktikan berdasarkan NIM atau nama. Algoritma tersebut memeriksa setiap data secara berurutan mulai dari elemen pertama sampai elemen terakhir. *Sequential Search* tidak mewajibkan data berada dalam kondisi terurut sebelum proses pencarian dijalankan.

Fungsi *sequentialSearchPraktikanKoordinator()* menerima kumpulan data praktikan dan kata kunci pencarian. Sistem mengembalikan seluruh data apabila pengguna tidak memasukkan kata kunci. Sistem mengubah kata kunci, NIM, dan nama menjadi huruf kecil agar pencarian tidak dipengaruhi oleh perbedaan penggunaan huruf kapital. Implementasi *Sequential Search* ditunjukkan pada *Source Code 4.4*.

```
private function sequentialSearchPraktikanKoordinator(
    $praktikans,
    ?string $keyword
) {
    $praktikans = collect($praktikans)->values();

    if (!$keyword) {
        return $praktikans;
    }
    $keyword = strtolower(trim($keyword));
    $hasil = collect();

    foreach ($praktikans as $praktikan) {
        $nim = strtolower(
            (string) ($praktikan->user->nim ?? '')
        );

        $nama = strtolower(
            (string) ($praktikan->user->name ?? '')
        );

        if (
            str_contains($nim, $keyword) ||
            str_contains($nama, $keyword)
        ) {
```

Source Code 4.4 Implementasi *Sequential Search* pada Data Praktikan

```

        $hasil->push($praktikan);
    }
}

return $hasil->values();
}

```

Source Code 4.4 Implementasi *Sequential Search* pada Data Praktikan (Lanjutan)

Source Code 4.4 menunjukkan bahwa sistem memeriksa NIM dan nama pada setiap data praktikan. Fungsi *str_contains()* menentukan keberadaan kata kunci pada kedua atribut tersebut. Sistem memasukkan setiap data yang sesuai ke dalam kumpulan hasil sehingga proses pencarian dapat menghasilkan satu atau beberapa data.

Sequential Search diterapkan pada data nilai akhir melalui fungsi *sequentialSearchNilaiAkhirKoordinator()*. Fungsi tersebut menggunakan prinsip yang sama, yaitu memeriksa NIM dan nama praktikan secara berurutan. Penerapan yang sama pada beberapa halaman menjaga konsistensi proses pencarian dalam sistem.

Pengujian *Sequential Search* menggunakan empat skenario yang sesuai dengan implementasi evaluasi, yaitu kata kunci ditemukan, kata kunci tidak ditemukan, kata kunci kosong, dan kata kunci yang menghasilkan beberapa data. Contoh *input* dan *output* pencarian ditunjukkan pada Tabel 4.5.

Tabel 4.4 Contoh *Input* dan *Output Sequential Search*

No.	Skenario	<i>Dataset Input</i>	Kata Kunci	<i>Output Sequential Search</i>
1	Kata kunci ditemukan	<i>Dataset</i> 100 entri	9999000050	Data ditemukan
2	Kata kunci tidak ditemukan	<i>Dataset</i> 100 entri	9999999999	Data tidak ditemukan
3	Kata kunci kosong	<i>Dataset</i> 100 entri	Kosong	Seluruh data ditampilkan
4	Kata kunci duplikat	<i>Dataset</i> 100 entri	<i>Benchmark</i> Praktikan	Beberapa data ditemukan

Tabel 4.5 menunjukkan bahwa *Sequential Search* dapat mengembalikan data ketika kata kunci tersedia, mengembalikan hasil kosong ketika kata kunci tidak ditemukan, dan menampilkan seluruh data ketika pengguna tidak memasukkan kata kunci. Sistem juga dapat menampilkan beberapa data apabila kata kunci yang sama terdapat pada lebih dari satu praktikan. Hasil tersebut menunjukkan bahwa algoritma dapat menangani berbagai kondisi pencarian sesuai dengan input yang diberikan pengguna.

4.2.3 Implementasi *Baseline* MySQL *ORDER BY* dan *WHERE/LIKE*

Sistem menggunakan MySQL sebagai *baseline* pembanding terhadap algoritma pada lapisan aplikasi. MySQL *ORDER BY* menjadi pembanding *Merge Sort* dalam proses pengurutan, sedangkan MySQL *WHERE/LIKE* menjadi pembanding *Sequential Search* dalam proses pencarian.

Sistem membedakan setiap metode pengujian melalui nilai pada kolom *method*. Nilai *merge_sort_application* menunjukkan pengujian *Merge Sort*, *sequential_search_application* menunjukkan pengujian *Sequential Search*, *mysql_order_by* menunjukkan *baseline* pengurutan MySQL, dan *mysql_where_like* menunjukkan *baseline* pencarian MySQL. Pemetaan label metode ditunjukkan pada *Source Code 4.5*.

```
$labelMetode = function ($method) {  
    return match ($method) {  
        'merge_sort_application' => 'Merge Sort',  
        'sequential_search_application' => 'Sequential Search',  
        'mysql_order_by' => 'MySQL ORDER BY',  
        'mysql_where_like' => 'MySQL WHERE/LIKE',  
        default => str_replace('_', ' ', $method),  
    };  
};
```

Source Code 4.5 Pemetaan Metode Pengujian

Tabel *performance_logs* menyimpan waktu eksekusi sesuai dengan jenis metode yang diuji. Kolom *sort_time* mencatat waktu *Merge Sort*, kolom *search_time* mencatat waktu *Sequential Search*, dan kolom *baseline_time* mencatat waktu *baseline* MySQL. Model *PerformanceLog* menentukan waktu eksekusi melalui atribut *execution_time* seperti yang ditunjukkan pada *Source Code 4.6*.

```
public function getExecutionTimeAttribute(): float  
{  
    return (float) (  
        $this->sort_time  
        ?? $this->search_time  
        ?? $this->baseline_time  
        ?? 0  
    );  
}
```

Source Code 4.6 Penentuan Waktu Eksekusi Pengujian

Sistem juga menyediakan kolom *explain_result* untuk menyimpan hasil *EXPLAIN* dari *query* MySQL. Hasil tersebut menunjukkan cara MySQL menjalankan proses

pengurutan atau pencarian, termasuk indeks yang digunakan dan perkiraan jumlah data yang diperiksa. Informasi tersebut membantu menunjukkan apakah MySQL dapat langsung menggunakan indeks atau perlu memeriksa banyak baris data. Skenario *baseline* MySQL yang digunakan sebagai pembanding ditunjukkan pada Tabel 4.6.

Tabel 4.5 Skenario *Baseline* MySQL

No.	Operasi	Metode pada Log	Bentuk Proses	Tujuan
1	Pengurutan	mysql_order_by	MySQL <i>ORDER BY</i> berdasarkan NIM	Membandingkan pengurutan MySQL dengan <i>Merge Sort</i>
2	Pencarian	mysql_where_like	MySQL <i>WHERE/LIKE</i> berdasarkan NIM atau nama	Membandingkan pencarian MySQL dengan <i>Sequential Search</i>

Tabel 4.6 menunjukkan dua metode *baseline* MySQL yang digunakan dalam evaluasi. MySQL *ORDER BY* menjadi pembanding proses pengurutan pada lapisan aplikasi, sedangkan MySQL *WHERE/LIKE* menjadi pembanding proses pencarian. Perbandingan tersebut menunjukkan perbedaan waktu eksekusi dan penggunaan memori antara pemrosesan pada lapisan aplikasi dan basis data.

4.2.4 Otomatisasi Pengujian Menggunakan *Artisan Command*

Sistem menjalankan pengujian *Merge Sort*, *Sequential Search*, MySQL *ORDER BY*, dan MySQL *WHERE/LIKE* secara otomatis melalui *Artisan Command*. Pengujian menggunakan *dataset* sintetis berukuran 100, 500, 1.000, dan 5.000 entri. Setiap kombinasi metode dan skenario menjalani 50 pengujian utama serta 10 proses *warm-up* agar pengukuran dilakukan secara konsisten. Perintah pengujian algoritma ditunjukkan pada *Source Code* 4.7.

```
php artisan benchmark:algoritma --fresh --runs=50 --warmup=10
```

Source Code 4.7 Perintah *Artisan Command* Pengujian Algoritma

Perintah `benchmark:algoritma` menjalankan seluruh proses evaluasi algoritma. Opsi `--fresh` menghapus hasil pengujian sebelumnya, opsi `--runs=50` menetapkan 50 pengujian utama pada setiap metode dan skenario, sedangkan opsi `--warmup=10` menjalankan 10 pengujian awal yang tidak digunakan dalam perhitungan statistik dan penyajian grafik.

Sistem menjalankan *Merge Sort*, *Sequential Search*, serta *baseline* MySQL pada ukuran *dataset* dan jumlah pengulangan yang sama. Sistem mencatat hasil khusus *Merge Sort* pada tabel `log_sortings` dan hasil khusus *Sequential Search* pada tabel `log_searches`. Sistem juga menyimpan seluruh hasil perbandingan algoritma dan *baseline* MySQL pada tabel `performance_logs`. *Baseline* MySQL menyimpan hasil MySQL *ORDER BY* dan *WHERE/LIKE* pada tabel `performance_logs` melalui kolom metode dan waktu *baseline*.

Sistem menyimpan hasil setiap pengujian pada tabel `performance_logs`. Sistem juga mencatat hasil khusus *Merge Sort* pada tabel `log_sortings` dan hasil khusus *Sequential Search* pada tabel `log_searches`. *Baseline* MySQL hanya disimpan pada tabel `performance_logs` melalui metode `mysql_order_by` dan `mysql_where_like`. Mekanisme penyimpanan hasil pengujian ditunjukkan pada *Source Code 4.8*.

```
$isWarmup = $run <= $warmup;

PerformanceLog::query()->create([
    'batch_code' => $batchCode,
    'timestamp' => now(),
    'dataset_size' => $size,
    'test_type' => $testType,
    'method' => $method,
    'scenario' => $scenario,
    'field_name' => $fieldName,
    'direction' => $direction,
    'keyword' => $keyword,
    'run_order' => $run,
    'is_warmup' => $isWarmup,
    'is_outlier' => false,

    'sort_time' => $method === 'merge_sort_application'
        ? $measurement['time_ms']
        : null,

    'search_time' => $method === 'sequential_search_application'
        ? $measurement['time_ms']
        : null,

    'baseline_time' => in_array($method, [
        'mysql_order_by',
        'mysql_where_like',
    ])
        ? $measurement['time_ms']
        : null,
```

Source Code 4.8 Pencatatan Hasil Pengujian dan Status *Warm-up*

```

        'memory_usage' => $measurement['memory_kb'] ?? 0,
        'iteration_count' => $measurement['iterations'] ?? 0,
        'comparison_count' => $measurement['comparisons'] ?? 0,
        'result_count' => $measurement['result_count'] ?? 0,
        'is_valid' => $measurement['is_valid'] ?? true,
        'explain_result' => $measurement['explain_result'] ?? null,
    ]);

    if (!$isWarmup && $method === 'merge_sort_application') {
        LogSorting::query()->create([
            'jumlah_data' => $size,
            'field_pengurutan' => $fieldName,
            'arah_pengurutan' => $direction,
            'waktu_pemrosesan_server' => $measurement['time_ms'],
            'jumlah_perbandingan' => $measurement['comparisons'] ?? 0,
            'penggunaan_memori' => $measurement['memory_kb'] ?? 0,
            'waktu_pengujian' => now(),
            'keterangan_pengujian' => $scenario,
        ]);
    }

    if (!$isWarmup && $method === 'sequential_search_application') {
        LogSearch::query()->create([
            'jumlah_data' => $size,
            'field_pencarian' => $fieldName,
            'kata_kunci_pencarian' => $keyword ?? '',
            'waktu_pemrosesan_server' => $measurement['time_ms'],
            'jumlah_iterasi' => $measurement['iterations'] ?? 0,
            'jumlah_perbandingan' => $measurement['comparisons'] ?? 0,
            'penggunaan_memori' => $measurement['memory_kb'] ?? 0,
            'waktu_pengujian' => now(),
            'keterangan_pengujian' => $scenario,
        ]);
    }
}

```

Source Code 4.8 Pencatatan Hasil Pengujian dan Status *Warm-up* (Lanjutan)

Source Code 4.8 menunjukkan proses penyimpanan hasil pengujian berdasarkan metode yang dijalankan. Tabel `performance_logs` menyimpan seluruh hasil *Merge Sort*, *Sequential Search*, *MySQL ORDER BY*, dan *MySQL WHERE/LIKE*. Kolom `sort_time`, `search_time`, dan `baseline_time` menyimpan waktu eksekusi sesuai metode pengujian. Sistem juga mencatat ukuran *dataset*, skenario, penggunaan memori, jumlah iterasi, jumlah perbandingan, jumlah hasil pencarian, validitas, status *warm-up*, dan hasil *EXPLAIN* MySQL.

Sistem menyimpan hasil utama *Merge Sort* secara khusus pada tabel `log_sortings`, sedangkan hasil utama *Sequential Search* disimpan pada tabel `log_searches`. Kondisi `!$isWarmup` mencegah data *warm-up* masuk ke kedua tabel tersebut. Tabel `performance_logs` tetap dapat mencatat data *warm-up* melalui nilai

is_warmup agar sistem dapat membedakannya dari pengujian utama ketika melakukan analisis.

Tabel 4.6 Hasil Eksekusi *Artisan Command* Pengujian Algoritma

No.	Tahap Pengujian	Proses yang Dijalankan	Hasil yang Dihasilkan
1	Konfigurasi <i>command</i>	Sistem membaca opsi <i>fresh</i> , <i>sizes</i> , <i>runs</i> , dan <i>warmup</i>	Parameter pengujian berhasil ditentukan
2	Persiapan <i>dataset</i>	Sistem menyiapkan <i>dataset</i> 100, 500, 1.000, dan 5.000 entri	Dataset sintetis tersedia untuk pengujian
3	Pelaksanaan pengujian	Sistem menjalankan <i>Merge Sort</i> , <i>Sequential Search</i> , MySQL <i>ORDER BY</i> , dan MySQL <i>WHERE/LIKE</i>	Seluruh metode berhasil diuji
4	Proses pengulangan	Sistem menjalankan 10 <i>warm-up</i> dan 50 pengujian utama	Data utama dan data <i>warm-up</i> dapat dibedakan
5	Penyimpanan <i>log</i>	Sistem menyimpan hasil berdasarkan metode pengujian	Hasil <i>Merge Sort</i> masuk ke <i>log_sortings</i> dan <i>performance_logs</i> , hasil <i>Sequential Search</i> masuk ke <i>log_searches</i> dan <i>performance_logs</i> , sedangkan <i>baseline</i> MySQL masuk ke <i>performance_logs</i>
6	Pengolahan hasil	Sistem mengecualikan <i>warm-up</i> dan <i>outlier</i> dari statistik	Data bersih tersedia untuk analisis
7	Penyajian hasil	Sistem membentuk statistik, grafik, dan berkas ekspor	Hasil evaluasi dapat dilihat dan diunduh

Tabel 4.7 menunjukkan bahwa *Artisan Command* mengatur proses pengujian mulai dari penentuan parameter, persiapan *dataset*, pelaksanaan setiap metode, pemisahan *warm-up*, penyimpanan *log*, hingga pengolahan hasil. Proses tersebut menghasilkan data evaluasi yang digunakan untuk membandingkan kinerja *Merge Sort*, *Sequential Search*, dan *baseline* MySQL.

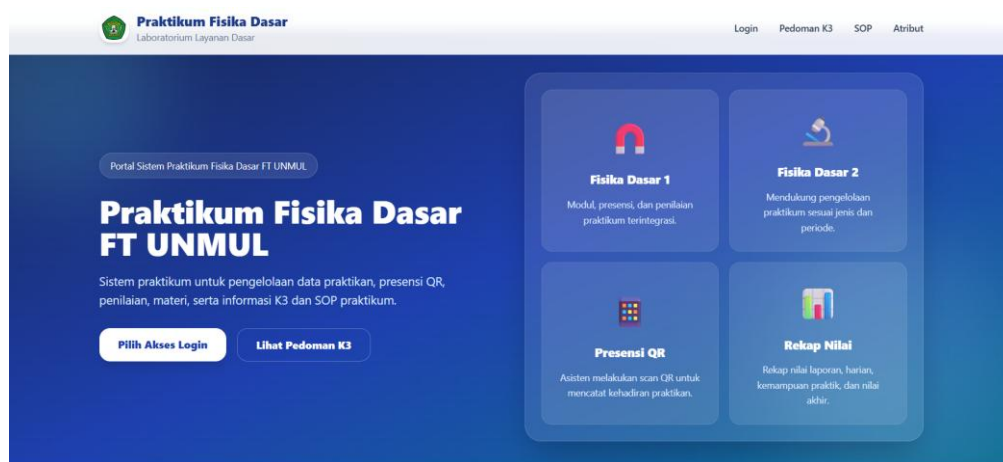
4.3 Penerapan Tampilan

Sistem menerapkan antarmuka berdasarkan rancangan *wireframe* yang telah dibuat pada tahap perancangan. Tampilan sistem dibagi sesuai hak akses praktikan, asisten, dan koordinator agar setiap pengguna memperoleh menu dan fitur yang sesuai dengan tugasnya. Sistem menyediakan halaman pendaftaran akun, pengelolaan data praktikan, presensi, materi, jadwal, penilaian, dan rekapitulasi nilai praktikum. Setiap

halaman dirancang agar pengguna dapat mengakses informasi dan menjalankan proses praktikum secara lebih terarah.

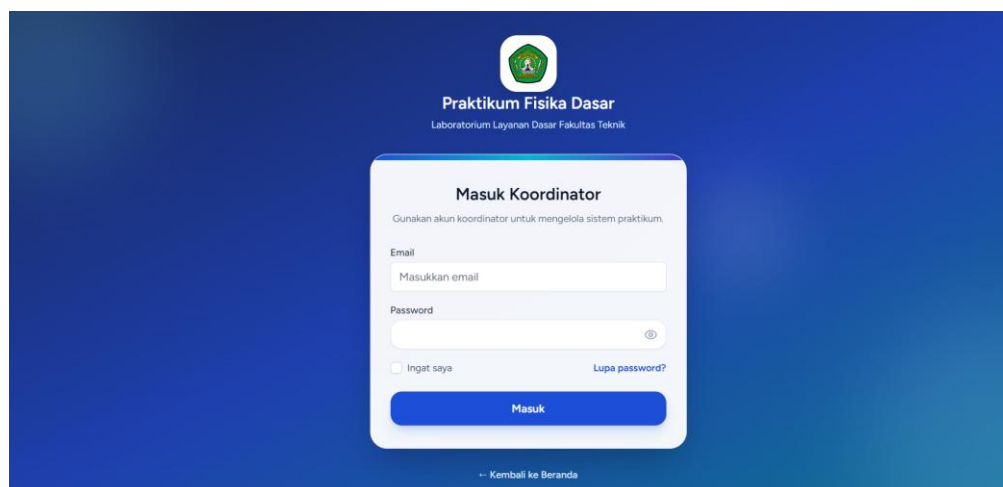
4.3.1 Tampilan Halaman Awal dan Autentikasi

Sistem menampilkan halaman awal yang memuat identitas Praktikum Fisika Dasar FT UNMUL, informasi layanan, serta akses menuju halaman masuk sesuai peran, pedoman K3, SOP, dan atribut praktikum. Tampilan halaman awal ditunjukkan pada Gambar 4.1.



Gambar 4.1 Tampilan Beranda Awal Sistem

Sistem menyediakan halaman masuk untuk memverifikasi email dan kata sandi pengguna. Pengguna yang berhasil masuk akan diarahkan menuju halaman sesuai hak akses akunnya. Tampilan halaman masuk ditunjukkan pada Gambar 4.2.



Gambar 4.2 Halaman Masuk ke Sistem

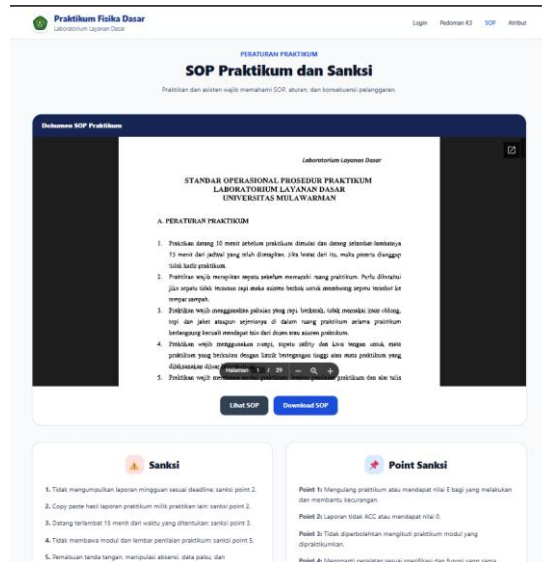
Sistem menyediakan halaman pendaftaran bagi praktikan yang belum memiliki akun. Praktikan dapat mendaftar menggunakan akun Google atau mengisi email dan kata sandi secara langsung. Tampilan halaman pendaftaran ditunjukkan pada Gambar 4.3.

Gambar 4.3 Halaman Pendaftaran Praktikan

Sistem menyediakan halaman Pedoman K3 yang memuat video keselamatan laboratorium dan dokumen pedoman K3. Pengguna dapat melihat atau mengunduh materi tersebut sebagai panduan sebelum mengikuti kegiatan praktikum. Tampilan halaman Pedoman K3 ditunjukkan pada Gambar 4.4.

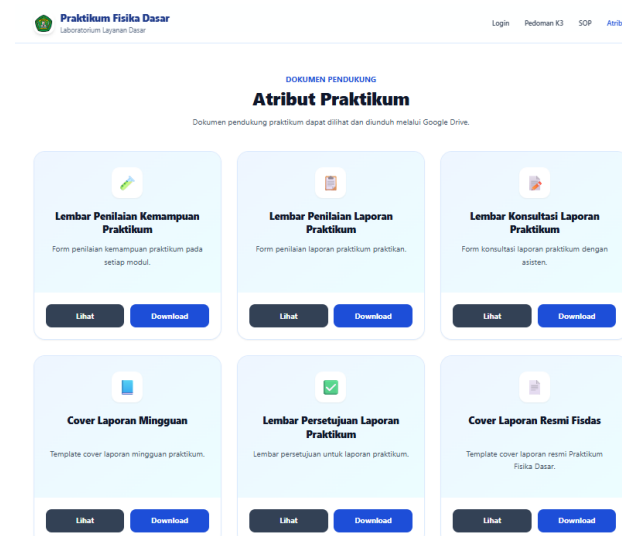
Gambar 4.4 Halaman Pedoman K3

Sistem menyediakan halaman SOP Praktikum yang memuat dokumen peraturan, tata tertib, dan ketentuan sanksi selama kegiatan praktikum. Pengguna dapat melihat atau mengunduh dokumen SOP melalui tombol yang tersedia. Tampilan halaman SOP Praktikum ditunjukkan pada Gambar 4.5.



Gambar 4.5 Halaman SOP Praktikum

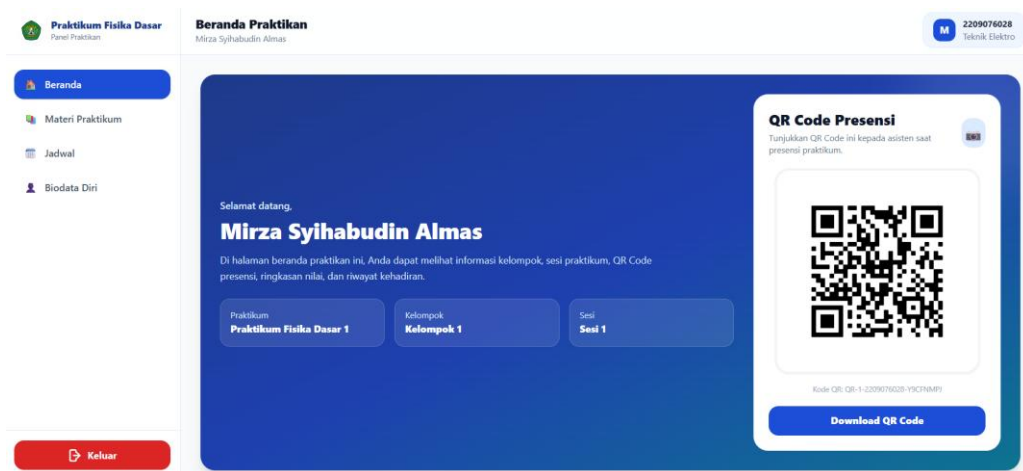
Sistem menyediakan halaman Atribut Praktikum yang memuat dokumen pendukung seperti lembar penilaian, lembar konsultasi, lembar persetujuan, dan format sampul laporan. Pengguna dapat melihat dan mengunduh setiap dokumen sesuai kebutuhan praktikum. Tampilan halaman Atribut Praktikum ditunjukkan pada Gambar 4.6.



Gambar 4.6 Halaman Atribut Praktikum

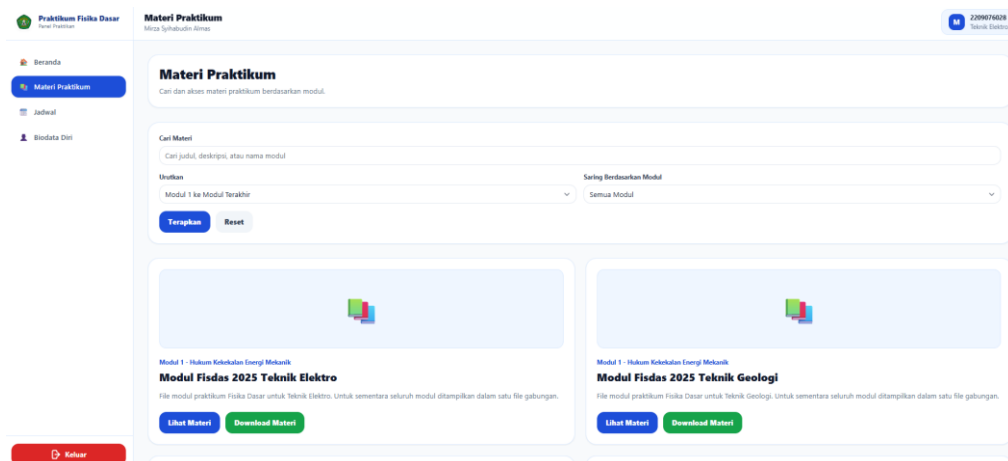
4.3.2 Tampilan Halaman Praktikan

Sistem menampilkan beranda praktikan yang memuat identitas, kelompok, sesi, *QR Code* presensi, informasi asisten, jumlah modul, nilai, dan rekap kehadiran. Halaman tersebut membantu praktikan memperoleh informasi penting sebelum mengakses materi, jadwal, dan biodata. Tampilan beranda praktikan ditunjukkan pada Gambar 4.7.



Gambar 4.7 Halaman Beranda Praktikan

Sistem menyediakan halaman materi agar praktikan dapat mencari, melihat, dan mengunduh modul praktikum. Praktikan juga dapat menyaring materi berdasarkan modul dan urutan yang dipilih. Tampilan halaman materi praktikan ditunjukkan pada Gambar 4.8.



Gambar 4.8 Halaman Materi Praktikum Praktikan

Sistem menampilkan halaman jadwal yang memuat program studi, kelompok, sesi, asisten pemeriksa, dan jadwal pengumpulan laporan. Informasi tersebut membantu praktikan mengetahui jadwal kegiatan yang harus diikuti. Tampilan halaman jadwal ditunjukkan pada Gambar 4.9.

Praktikum Fisika Dasar
Panel Praktikan

Jadwal Praktikan
Mirza Syihabudin Almas

M 2209076028
Teknik Elektro

Beranda
Materi Praktikum
Jadwal
Biodata Diri

Jadwal Praktikan
Halaman ini menampilkan jadwal yang berkaitan dengan praktikan berdasarkan program studi, kelompok, dan sesi praktikum.

Program Studi
Teknik Elektro

Kelompok
Kelompok 1

Sesi
Sesi 1

Asisten Pemeriksa
Per Modul

Jadwal Pengambilan dan Pengumpulan Laporan Praktikum
Jadwal laporan ditampilkan berdasarkan program studi Anda.

Hari	Jam	Keterangan
Jumat	12:00 - 13:00 WITA	Elektro

Keluar

Gambar 4.9 Halaman Jadwal Praktikan

Sistem menyediakan halaman biodata diri untuk menampilkan dan memperbarui informasi pribadi praktikan. Data yang tersedia meliputi *email*, nama lengkap, NIM, program studi, dan identitas lainnya. Tampilan halaman biodata ditunjukkan pada Gambar 4.10.

Praktikum Fisika Dasar
Panel Praktikan

Biodata Diri
Mirza Syihabudin Almas

M 2209076028
Teknik Elektro

Beranda
Materi Praktikum
Jadwal
Biodata Diri

Biodata Diri Praktikan
Praktikan dapat memperbarui biodata pribadi. Email tidak dapat diubah dari halaman ini.

Email
praktikan.2209076028@fisdas.test

Nama Lengkap
Mirza Syihabudin Almas

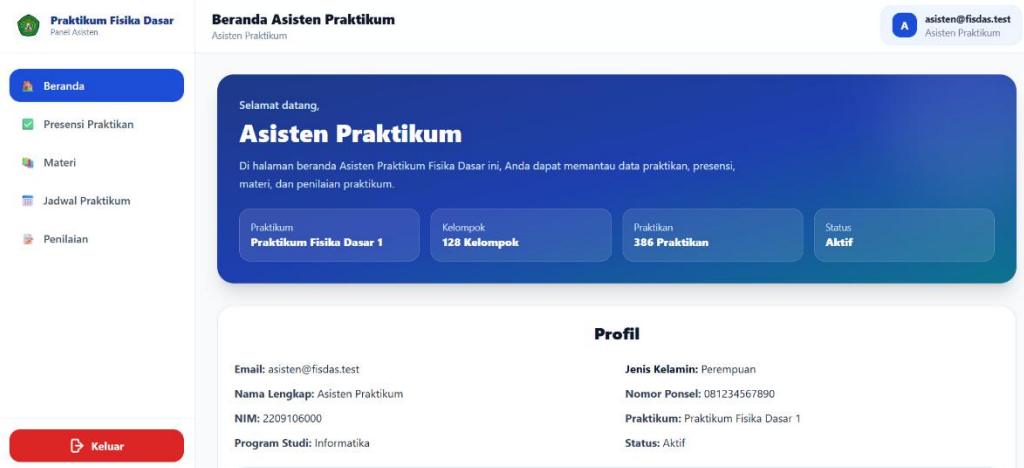
NIM
2209076028

Program Studi
Teknik Elektro

Gambar 4.10 Halaman Biodata Diri Praktikan

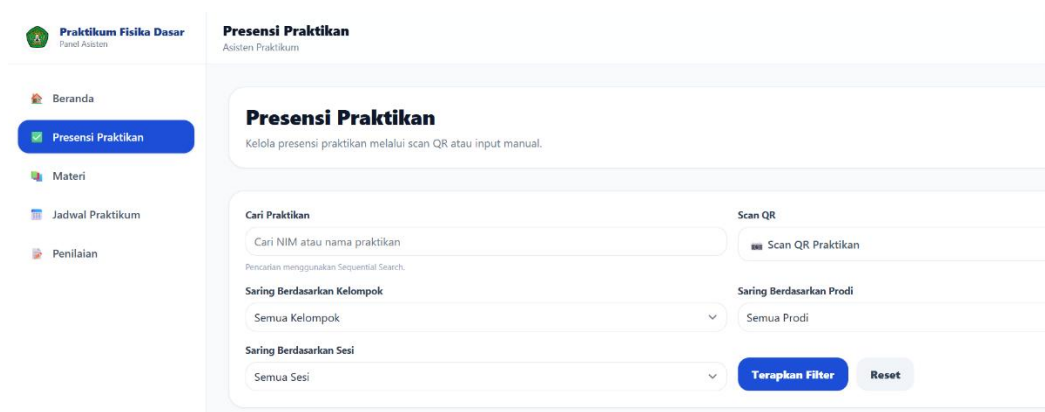
4.3.3 Tampilan Halaman Asisten Praktikum

Sistem menampilkan halaman beranda asisten yang memuat ringkasan praktikum, jumlah kelompok, jumlah praktikan, status praktikum, dan profil asisten. Halaman tersebut menjadi pusat informasi awal bagi asisten. Tampilan beranda asisten ditunjukkan pada Gambar 4.11.



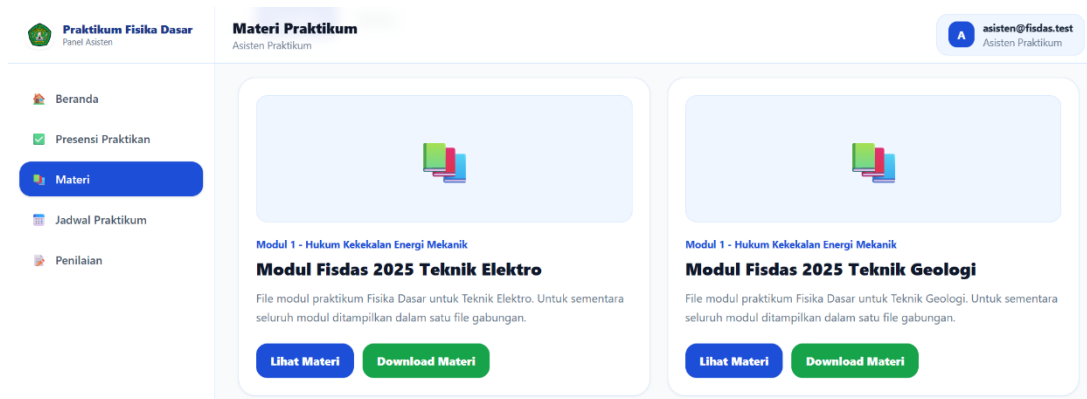
Gambar 4.11 Beranda Asisten Praktikum

Sistem menyediakan halaman presensi untuk mengelola kehadiran praktikan melalui *scan QR Code* atau *input* manual. Asisten juga dapat mencari dan menyaring data berdasarkan kelompok, program studi, dan sesi. Tampilan halaman presensi ditunjukkan pada Gambar 4.12.



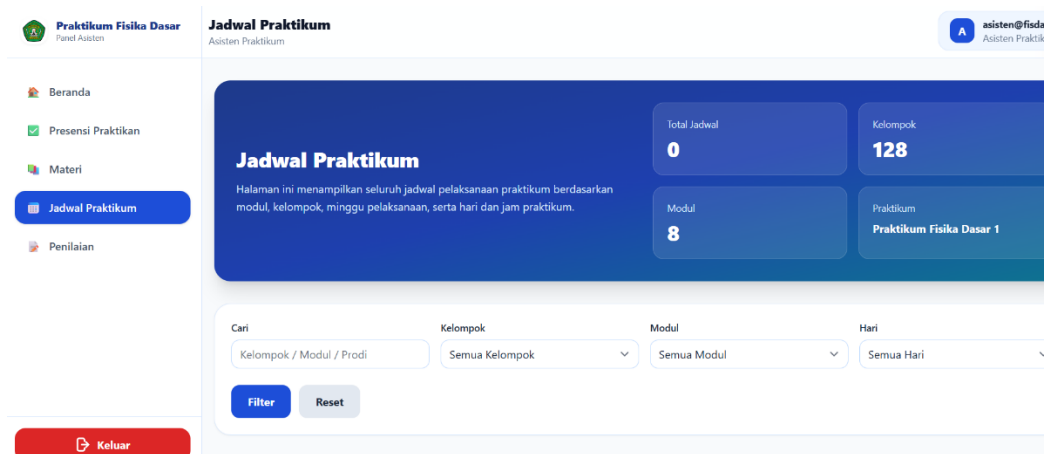
Gambar 4.12 Halaman Presensi Praktikan

Sistem menampilkan halaman materi yang memuat daftar modul sesuai program studi. Asisten dapat melihat atau mengunduh materi melalui tombol yang tersedia pada setiap kartu. Tampilan halaman materi ditunjukkan pada Gambar 4.13.



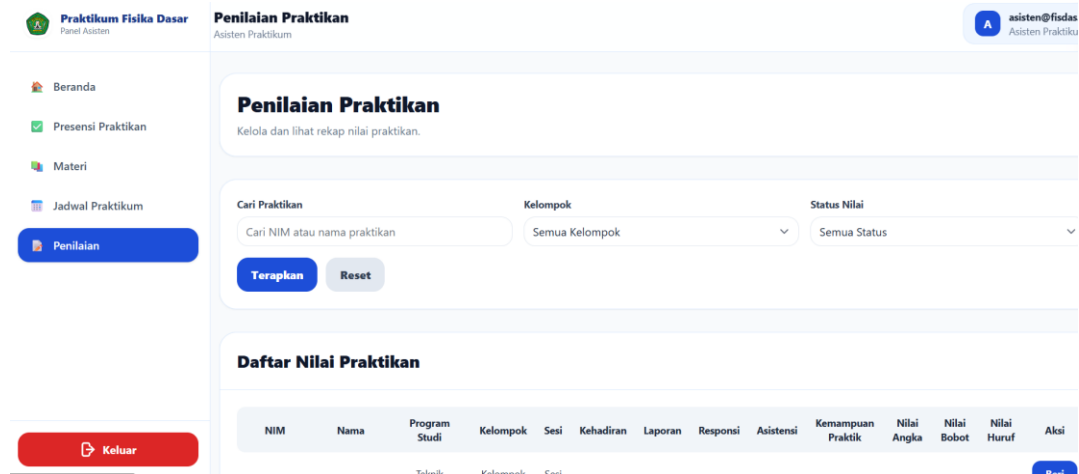
Gambar 4.13 Halaman Materi Praktikum Asisten

Sistem menyediakan halaman jadwal yang menampilkan informasi terkait pelaksanaan praktikum, meliputi kelompok, modul, hari, dan waktu pelaksanaan praktikum. Informasi tersebut disajikan dalam bentuk tabel sehingga memudahkan asisten dalam melihat dan mengetahui jadwal praktikum yang tersedia. Asisten juga dapat menggunakan fitur pencarian dan filter untuk menemukan jadwal berdasarkan informasi yang diperlukan. Dengan adanya fitur tersebut, proses pencarian jadwal dapat dilakukan dengan lebih mudah. Tampilan halaman jadwal ditunjukkan pada Gambar 4.14.



Gambar 4.14 Halaman Jadwal Praktikum Asisten

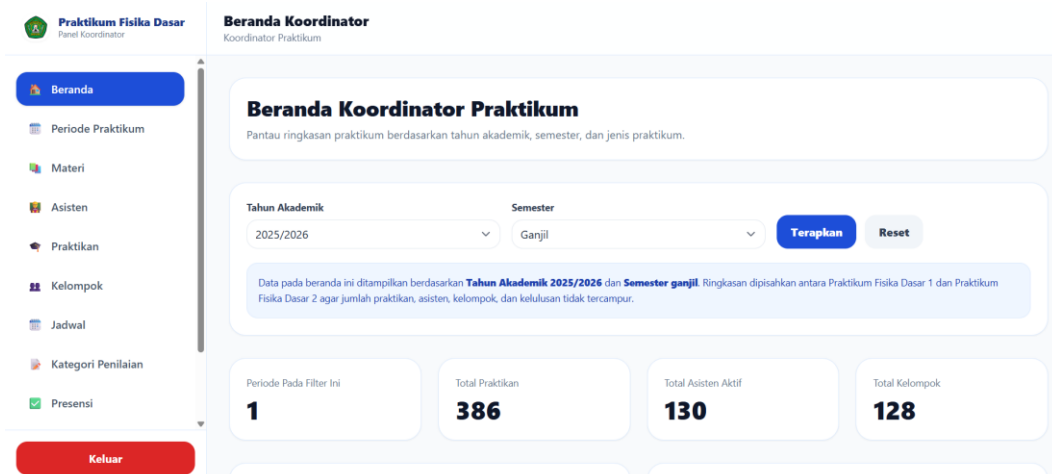
Sistem menampilkan halaman penilaian yang memuat daftar nilai praktikan. Asisten dapat mencari data, menyaring berdasarkan kelompok dan status nilai, serta membuka proses pemberian nilai. Tampilan halaman penilaian ditunjukkan pada Gambar 4.15.



Gambar 4.15 Halaman Penilaian Praktikan

4.3.4 Tampilan Halaman Koordinator Praktikum

Sistem menampilkan beranda koordinator yang memuat ringkasan periode, jumlah praktikan, jumlah asisten, dan jumlah kelompok. Koordinator dapat memilih tahun akademik dan semester untuk menampilkan data yang sesuai. Tampilan beranda koordinator ditunjukkan pada Gambar 4.16.



Gambar 4.16 Halaman Beranda Koordinator Praktikum

Sistem menyediakan halaman periode praktikum untuk menambahkan dan mengelola nama praktikum, jenis praktikum, tahun akademik, semester, dan status periode aktif. Tampilan halaman periode ditunjukkan pada Gambar 4.17.

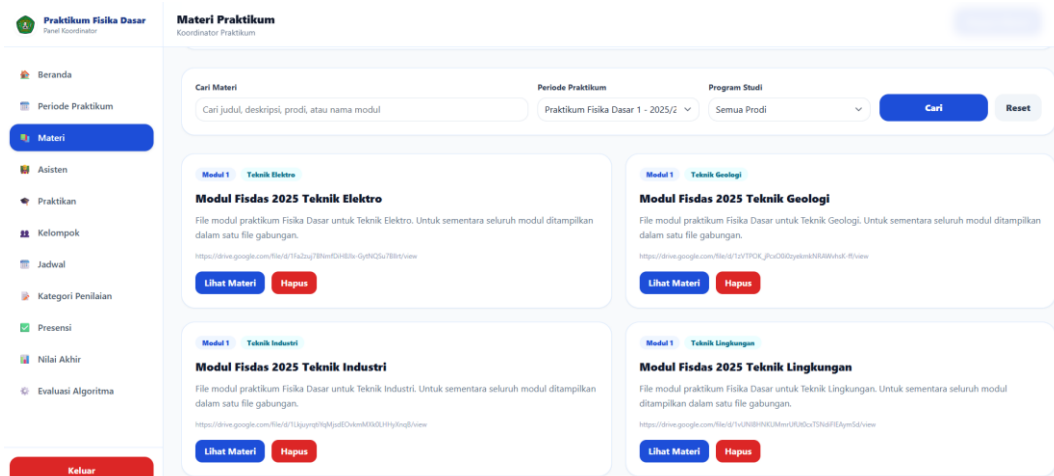
Gambar 4.17 Halaman Periode Praktikum

Sistem menampilkan halaman detail periode yang memuat informasi jumlah praktikan, asisten, jumlah kelompok pada setiap program studi, serta rata-rata nilai akhir praktikan. Tampilan detail periode ditunjukkan pada Gambar 4.18.

Program Studi	Jumlah Kelompok
Teknik Elektro	16
Teknik Geologi	16
Teknik Industri	16
Teknik Lingkungan	24
Teknik Pertambangan	25
Total Kelompok	128

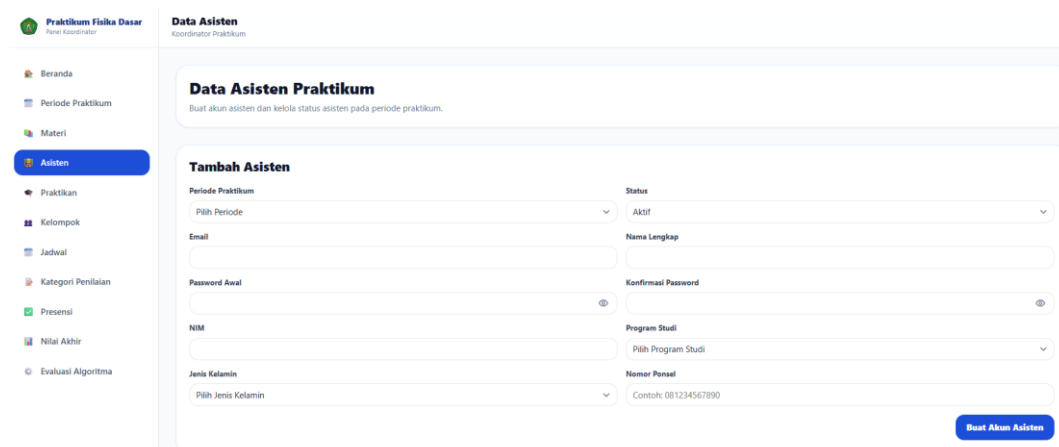
Gambar 4.18 Halaman Detail Periode Praktikum

Sistem menyediakan halaman materi bagi koordinator untuk mencari dan mengelola modul berdasarkan periode dan program studi. Koordinator dapat melihat materi serta menghapus data yang tidak diperlukan. Tampilan halaman materi ditunjukkan pada Gambar 4.19.



Gambar 4.19 Halaman Materi Praktikum Koordinator

Sistem menampilkan halaman data asisten yang dilengkapi formulir pembuatan akun baru. Formulir tersebut memuat periode, identitas, program studi, status, dan informasi kontak asisten. Tampilan halaman data asisten ditunjukkan pada Gambar 4.20.



Gambar 4.20 Halaman Data Asisten

Sistem menyediakan halaman data praktikan yang menampilkan informasi praktikan secara terstruktur dan dilengkapi fitur pencarian menggunakan *Sequential Search* serta pengurutan data menggunakan *Merge Sort*. Koordinator juga dapat menyaring data berdasarkan periode, program studi, kelompok, sesi, dan urutan NIM untuk memudahkan pencarian serta pengelolaan data sesuai kebutuhan. Fitur tersebut membantu koordinator memperoleh data praktikan secara lebih cepat dan terorganisir. Tampilan halaman data praktikan ditunjukkan pada Gambar 4.21.

Data Praktikan
Koordinator Praktikum

Data Praktikan
Lihat data praktikan yang terdaftar pada periode praktikum.

Pencarian
Menggunakan Sequential Search berdasarkan NIM dan nama praktikan.

Pengurutan
Data diurutkan menggunakan Merge Sort berdasarkan Prodi, lalu NIM.

Cari Praktikan
Cari NIM atau nama praktikan

Periode Praktikum
Praktikum Fisika Dasar 1 - 2025/2026

Program Studi
Semua Prodi

Kelompok
Semua Kelompok

Sesi Praktikum
Semua Sesi

Urutan NIM
NIM kecil ke besar

Cari **Reset**

Daftar Praktikan
Data ditampilkan berdasarkan hasil Sequential Search dan diurutkan dengan Merge Sort.

No.	NIM	Nama	No. Ponsel	Email	Prodi	Kelompok	Sesi
-----	-----	------	------------	-------	-------	----------	------

Gambar 4.21 Halaman Data Praktikan

Sistem menampilkan halaman kelompok untuk membentuk kelompok berdasarkan periode, program studi, sesi, serta rentang NIM. Halaman tersebut juga menyediakan pencarian dan daftar praktikan yang telah tergabung dalam kelompok. Tampilan halaman kelompok ditunjukkan pada Gambar 4.22.

Kelompok Praktikum
Koordinator Praktikum

Kelompok Praktikum
Kelola kelompok praktikum berdasarkan program studi, sesi, dan asisten pemeriksa per modul.

Tambah Kelompok

Periode Praktikum
Praktikum Fisika Dasar 1 - 2025/2026 genjil

Program Studi
Contoh: Teknik Industri

Nama Kelompok
Contoh: Kelompok 1

Sesi Praktikum
Contoh: Sesi 1

NIM Awal
2209106001

NIM Akhir
2209106005

Maukulan NIM teratas

Maukulan NIM teratas. Sistem akan memaukulan praktikum dengan awalan NIM yang sama dan nomor urut 001 sampai 005.

Tambah Kelompok

Cari NIM, nama, kelompok **Praktikum Fisika Dasar 1** **Semua Prodi** **Semua Sesi** **Cari** **Reset**

Kelompok Praktikum Fisika Dasar Semua Program Studi
Laboratorium Layanan Dasar

No.	NIM	Nama	Kelompok	Sesi	Aksi
-----	-----	------	----------	------	------

Gambar 4.22 Halaman Kelompok Praktikum

Sistem menyediakan halaman jadwal yang digunakan untuk mengatur jadwal pengambilan dan pengumpulan laporan praktikum. Koordinator dapat menambahkan dan mengatur informasi jadwal dengan menentukan program studi, hari, jam mulai, jam selesai, serta keterangan yang diperlukan. Data jadwal yang telah diatur akan ditampilkan pada halaman sehingga dapat digunakan sebagai informasi pelaksanaan pengambilan dan pengumpulan laporan praktikum. Pengelolaan jadwal ini membantu koordinator dalam

mengatur waktu kegiatan secara lebih terstruktur. Tampilan halaman jadwal ditunjukkan pada Gambar 4.23.

Gambar 4.23 Halaman Jadwal Praktikum Koordinator

Sistem menampilkan halaman kategori penilaian yang memuat komponen nilai akhir praktikan, yaitu kehadiran, laporan, responsi, asistensi, dan kemampuan praktik. Halaman ini juga menampilkan persentase setiap komponen serta rumus perhitungan nilai akhir. Tampilan halaman kategori penilaian ditunjukkan pada Gambar 4.24.

Gambar 4.24 Halaman Kategori Penilaian

Sistem menampilkan halaman presensi praktikan yang memuat data kehadiran berdasarkan hasil *scan QR Code* atau *input* manual oleh asisten. Koordinator dapat mencari dan menyaring data berdasarkan periode, program studi, modul, status kehadiran,

kelompok, dan sesi praktikum. Tampilan halaman presensi praktikan ditunjukkan pada Gambar 4.25.

Timestamp	Modul	Asisten	NIM	Nama	Prodi	Kelompok	Sesi	Status	Keterangan
15/05/2026 12:05:40	Modul 8 Hambatan dalam Baterai dan Rangkaian Pembagi Tegangan	asisten@fisdas.test	2209076028	Mirza Syihava Almas	Teknik Elektro	Kelompok 1	Sesi 1	Hadir	Hadir
15/05/2026 12:05:39	Modul 7 Bayangan, Cermin, Warna dan Pembiasan	asisten@fisdas.test	2209076028	Mirza Syihava Almas	Teknik Elektro	Kelompok 1	Sesi 1	Hadir	Hadir
15/05/2026 12:05:38	Modul 6 Rangkaian Seri-Paralel	asisten@fisdas.test	2209076028	Mirza Syihava Almas	Teknik Elektro	Kelompok 1	Sesi 1	Hadir	Hadir
15/05/2026 12:04:56	Modul 3 Simulasi Pencemaran	asisten@fisdas.test	2209076028	Mirza Syihava Almas	Teknik Elektro	Kelompok 1	Sesi 1	Izin	Izin

Gambar 4.25 Halaman Presensi Praktikan

Sistem menampilkan halaman nilai akhir yang memuat hasil penilaian praktikan berdasarkan seluruh komponen nilai. Koordinator dapat mencari data menggunakan *Sequential Search*, mengurutkan data menggunakan *Merge Sort*, serta menyaring hasil berdasarkan periode, program studi, status kelulusan, dan urutan NIM. Tampilan halaman nilai akhir ditunjukkan pada Gambar 4.26.

No.	NIM	Nama	Prodi	Kelompok	Kehadiran	Laporan	Respons	Asistensi	Praktikum	Angka	Bobot	Huruf	Kelulusan
-----	-----	------	-------	----------	-----------	---------	---------	-----------	-----------	-------	-------	-------	-----------

Gambar 4.26 Halaman Nilai Akhir Praktikan

Sistem menampilkan halaman evaluasi algoritma yang memuat grafik perbandingan waktu eksekusi dan penggunaan memori. Halaman ini membandingkan

Merge Sort dengan MySQL *ORDER BY* serta *Sequential Search* dengan MySQL *WHERE/LIKE* pada *dataset* berukuran 100, 500, 1.000, dan 5.000 entri. Tampilan halaman evaluasi algoritma ditunjukkan pada Gambar 4.27.



Gambar 4.27 Halaman Evaluasi Algoritma

4.4 Hasil Pengujian

Hasil pengujian menyajikan data yang diperoleh dari pengujian fungsional sistem dan evaluasi kinerja algoritma. Pengujian fungsional dilakukan untuk memastikan setiap fitur berjalan sesuai kebutuhan pengguna. Pengujian kinerja dilakukan untuk mengetahui performa *Merge Sort*, *Sequential Search*, MySQL *ORDER BY*, dan MySQL *WHERE/LIKE* berdasarkan waktu eksekusi serta penggunaan memori pada *dataset* berukuran 100, 500, 1.000, dan 5.000 data. Hasil tersebut dianalisis untuk mengetahui kesesuaian fungsi sistem dan kinerja metode yang diterapkan pada sistem praktikum berbasis web.

4.4.1 Hasil Pengujian Fungsional Sistem

Pengujian fungsional dilakukan menggunakan metode *Black Box Testing* untuk memastikan kesesuaian masukan dan keluaran pada fitur sistem, meliputi registrasi, *login*, hak akses, pengelolaan data, presensi, nilai, serta evaluasi algoritma. Hasil pengujian dinyatakan sesuai apabila keluaran sistem memenuhi hasil yang diharapkan. Pengujian

otentikasi dan hak akses pada Tabel 4.8 menunjukkan bahwa sistem mampu membedakan akses praktikan, asisten, dan koordinator praktikum sesuai kewenangannya.

Tabel 4.7 Pengujian Autentikasi dan Hak Akses

No.	Bagian Pengujian	Skenario Pengujian	Hasil yang Diharapkan	Hasil Pengujian
1	Registrasi	Semua data pendaftaran diisi dengan benar	Akun praktikan berhasil dibuat	Sesuai
		Email sudah terdaftar	Sistem menolak pendaftaran dan menampilkan pesan kesalahan	Sesuai
		Kata sandi dan konfirmasi kata sandi tidak sama	Sistem menampilkan pesan kesalahan	Sesuai
		Salah satu data wajib tidak diisi	Sistem menolak pendaftaran dan menampilkan peringatan	Sesuai
		Pendaftaran dilakukan menggunakan akun Google	Akun berhasil dibuat melalui autentikasi Google	Sesuai
2	Login	Email dan kata sandi benar	Sistem mengarahkan pengguna ke halaman sesuai peran	Sesuai
		Email atau kata sandi salah	Sistem menolak login dan menampilkan pesan kesalahan	Sesuai
		Email atau kata sandi kosong	Sistem menampilkan peringatan bahwa data wajib diisi	Sesuai
3	Hak Akses	Praktikan mengakses halaman asisten atau koordinator	Sistem menolak akses	Sesuai
		Asisten mengakses halaman koordinator	Sistem menolak akses	Sesuai
4	Sesi Pengguna	Pengguna melakukan <i>logout</i>	Sistem mengakhiri sesi dan kembali ke halaman <i>login</i>	Sesuai
		Pengguna mengakses halaman tanpa <i>login</i>	Sistem mengarahkan pengguna ke halaman <i>login</i>	Sesuai

Pengujian fitur berdasarkan peran pengguna memeriksa kesesuaian menu dan fungsi yang tersedia bagi praktikan, asisten, dan koordinator. Hasil pengujian pada Tabel

4.9 menunjukkan bahwa setiap pengguna hanya memperoleh fitur yang sesuai dengan tugas dan kebutuhannya.

Tabel 4.8 Pengujian Fitur Berdasarkan Peran Pengguna

No.	Peran Pengguna	Bagian Pengujian	Skenario Pengujian	Hasil yang Diharapkan	Hasil Pengujian
1	Praktikan	Beranda	Melihat ringkasan identitas, kelompok, sesi, presensi, modul, dan nilai	Sistem menampilkan ringkasan data praktikan	Sesuai
		Materi	Mencari, melihat, dan mengunduh materi praktikum	Materi tampil dan dapat diakses	Sesuai
		Jadwal	Melihat jadwal berdasarkan program studi, kelompok, dan sesi	Jadwal praktikan ditampilkan sesuai data	Sesuai
		Biodata Diri	Melihat dan memperbarui biodata	Data tampil dan perubahan berhasil disimpan	Sesuai
2	Asisten	Beranda	Melihat ringkasan praktikum, kelompok, dan praktikan	Sistem menampilkan ringkasan data asisten	Sesuai
		Presensi	Mencatat presensi <i>melalui QR Code</i> atau <i>input</i> manual	Data presensi berhasil disimpan	Sesuai
		Materi	Melihat dan mengunduh materi praktikum	Materi tampil dan dapat diakses	Sesuai
		Jadwal	Melihat jadwal berdasarkan kelompok, modul, dan hari	Jadwal ditampilkan sesuai filter	Sesuai
		Penilaian	Mengisi dan melihat nilai praktikan	Data nilai berhasil ditampilkan dan disimpan	Sesuai
3	Koordinator	Beranda	Melihat ringkasan praktikum berdasarkan tahun dan semester	Sistem menampilkan ringkasan sesuai periode	Sesuai
		Periode	Menambah dan mengelola periode praktikum	Data periode berhasil dikelola	Sesuai
		Materi	Mencari, melihat, menambah, dan menghapus materi	Data materi berhasil dikelola	Sesuai
		Asisten	Membuat akun dan mengelola data asisten	Data asisten berhasil disimpan	Sesuai
		Praktikan	Melihat, mencari, menyaring, dan mengurutkan data praktikan	Data praktikan tampil sesuai kriteria	Sesuai
		Kelompok	Menambah dan mengelola kelompok praktikum	Data kelompok berhasil dikelola	Sesuai

Tabel 4.9 Pengujian Fitur Berdasarkan Peran Pengguna (Lanjutan)

No.	Peran Pengguna	Bagian Pengujian	Skenario Pengujian	Hasil yang Diharapkan	Hasil Pengujian
3	Koordinator	Jadwal	Menambah dan mengelola jadwal praktikum	Data jadwal berhasil dikelola	Sesuai
		Kategori Penilaian	Melihat komponen dan bobot nilai akhir	Komponen serta rumus nilai ditampilkan	Sesuai
		Presensi	Melihat dan menyaring rekap presensi	Data presensi tampil sesuai filter	Sesuai
		Nilai Akhir	Melihat dan mengunduh rekap nilai akhir	Rekap nilai tampil dan dapat diunduh	Sesuai
		Evaluasi Algoritma	Melihat statistik dan grafik hasil pengujian	Sistem menampilkan hasil evaluasi algoritma	Sesuai

Pengujian pengelolaan data praktikum memeriksa fungsi yang digunakan koordinator dalam mengelola data pada satu periode. Hasil pengujian pada Tabel 4.10 mencakup pengelolaan periode, asisten, praktikan, kelompok, jadwal, materi, presensi, kategori penilaian, dan nilai akhir.

Tabel 4.9 Pengujian Pengelolaan Data Praktikum

No.	Bagian Pengujian	Skenario Pengujian	Hasil yang Diharapkan	Hasil Pengujian
1	Periode Praktikum	Menambah periode praktikum	Data periode berhasil disimpan	Sesuai
		Menentukan periode aktif	Status periode aktif berhasil diperbarui	Sesuai
		Melihat detail periode	Sistem menampilkan jumlah kelompok setiap program studi	Sesuai
2	Data Asisten	Menambah akun asisten	Data asisten berhasil disimpan	Sesuai
		Mengubah status asisten	Status asisten berubah sesuai pilihan	Sesuai
3	Data Praktikan	Melihat daftar praktikan berdasarkan periode	Data praktikan tampil sesuai periode	Sesuai
		Menyaring berdasarkan program studi, kelompok, dan sesi	Data tampil sesuai <i>filter</i>	Sesuai
4	Kelompok	Menambah kelompok berdasarkan rentang NIM	Kelompok dan anggota berhasil dibentuk	Sesuai
		Mencari data kelompok	Data yang sesuai kata kunci ditampilkan	Sesuai

Tabel 4.10 Pengujian Pengelolaan Data Praktikum (Lanjutan)

No.	Bagian Pengujian	Skenario Pengujian	Hasil yang Diharapkan	Hasil Pengujian
5	Jadwal	Menambah jadwal pengambilan atau pengumpulan laporan	Data jadwal berhasil disimpan	Sesuai
		Mengubah atau menghapus jadwal	Data jadwal berhasil diperbarui atau dihapus	Sesuai
6	Materi	Menampilkan materi berdasarkan periode dan program studi	Materi tampil sesuai <i>filter</i>	Sesuai
		Menghapus materi	Data materi berhasil dihapus	Sesuai
7	Presensi	Melihat presensi berdasarkan periode, modul, dan kelompok	Data presensi tampil sesuai filter	Sesuai
8	Kategori Penilaian	Melihat komponen dan bobot penilaian	Sistem menampilkan komponen serta rumus nilai	Sesuai
9	Nilai Akhir	Melihat rekap nilai akhir	Rekap nilai tampil sesuai data praktikan	Sesuai
		Mengunduh rekap nilai akhir berdasarkan program studi	Berkas rekap nilai berhasil diunduh	Sesuai

Pengujian pencarian, pengurutan, penyaringan, dan navigasi memeriksa fungsi pendukung yang digunakan saat mengakses data. Hasil pengujian pada Tabel 4.11 menunjukkan bahwa sistem dapat menampilkan data berdasarkan kata kunci, urutan, dan *filter* yang dipilih serta mempertahankan sesi pengguna saat halaman dimuat kembali.

Tabel 4.10 Pengujian Pencarian, Pengurutan, Penyaringan, dan Navigasi

No.	Bagian Pengujian	Skenario Pengujian	Hasil yang Diharapkan	Hasil Pengujian
1	Pencarian	Mencari data yang tersedia	Data yang sesuai kata kunci ditampilkan	Sesuai
		Mencari data yang tidak tersedia	Sistem menampilkan keterangan data tidak ditemukan	Sesuai
		Kata kunci pencarian dikosongkan	Sistem menampilkan seluruh data	Sesuai
		Kata kunci memiliki beberapa hasil yang sama	Sistem menampilkan seluruh data yang sesuai	Sesuai
2	Pengurutan	Mengurutkan NIM dari kecil ke besar	Data tampil secara menaik	Sesuai
		Mengurutkan NIM dari besar ke kecil	Data tampil secara menurun	Sesuai

**Tabel 4.11 Pengujian Pencarian, Pengurutan, Penyaringan, dan Navigasi
(Lanjutan)**

No.	Bagian Pengujian	Skenario Pengujian	Hasil yang Diharapkan	Hasil Pengujian
3	Penyaringan	Menyaring berdasarkan periode, program studi, kelompok, atau sesi	Data tampil sesuai <i>filter</i> yang dipilih	Sesuai
		Menyaring berdasarkan modul, status kehadiran, atau kelulusan	Data tampil sesuai <i>filter</i> yang dipilih	Sesuai
4	Navigasi	Memilih menu pada sidebar atau tombol aksi	Sistem menampilkan halaman tujuan tanpa kesalahan	Sesuai
		Menekan tombol kembali pada <i>browser</i>	Halaman tetap dapat dimuat dengan benar	Sesuai
5	Sesi Pengguna	Melakukan <i>refresh</i> setelah <i>login</i>	Sesi tetap aktif dan halaman dapat dimuat kembali	Sesuai

Pengujian fungsional algoritma memeriksa ketepatan hasil pengurutan dan pencarian pada sistem. Hasil pengujian pada Tabel 4.12 menunjukkan bahwa *Merge Sort* dapat mengurutkan data dalam beberapa kondisi, sedangkan *Sequential Search* dapat menghasilkan keluaran sesuai dengan kata kunci pencarian.

Tabel 4.11 Pengujian Fungsional Algoritma

No.	Bagian Pengujian	Skenario Pengujian	Hasil yang Diharapkan	Hasil Pengujian
1	<i>Merge Sort</i>	Mengurutkan data acak secara menaik	Data terurut dari kecil ke besar	Sesuai
		Mengurutkan data acak secara menurun	Data terurut dari besar ke kecil	Sesuai
		Mengurutkan data yang sudah terurut	Data tetap terurut tanpa mengalami kesalahan	Sesuai
		Mengurutkan data dengan nilai duplikat	Data tetap terurut dan data duplikat tidak hilang	Sesuai
2	<i>Sequential Search</i>	Mencari kata kunci yang tersedia	Data yang sesuai ditemukan dan ditampilkan	Sesuai
		Mencari kata kunci yang tidak tersedia	Sistem menampilkan keterangan data tidak ditemukan	Sesuai
		Mengosongkan kata kunci pencarian	Sistem menampilkan seluruh data	Sesuai
		Mencari kata kunci yang memiliki hasil duplikat	Seluruh data yang sesuai ditampilkan	Sesuai

Seluruh hasil pengujian fungsional menunjukkan bahwa sistem memberikan keluaran yang sesuai dengan skenario pengujian. Fitur autentikasi, pembatasan hak akses,

pengelolaan data praktikum, presensi, penilaian, pencarian, pengurutan, penyaringan, dan evaluasi algoritma dapat berjalan sesuai kebutuhan pengguna.

4.4.2 Hasil Pengujian *Merge Sort*

Pengujian *Merge Sort* menggunakan empat skenario, yaitu data sudah urut, data duplikat, data acak menaik, dan data acak menurun. Pengujian mencatat waktu eksekusi, penggunaan memori, jumlah pengulangan bersih, dan jumlah *outlier* pada setiap ukuran *dataset*. Ringkasan hasil pengujian *Merge Sort* disajikan pada Tabel 4.13.

Tabel 4.12 Hasil Pengujian *Merge Sort*

Data	Skenario	Run Bersih	IQR	Mean (ms)	Median (ms)	Std. Deviasi	Memori (KB)
100	Sudah Urut	50/50	0	0,385561	0,370502	0,097034	6,6328
100	Duplikat	50/50	0	0,592618	0,683069	0,148113	6,6328
100	<i>Random Asc</i>	38/50	12	0,383057	0,383019	0,012623	6,6328
100	<i>Random Desc</i>	45/50	5	0,438023	0,387907	0,104021	6,6328
500	Sudah Urut	44/50	6	1,691477	1,660943	0,090056	29,3047
500	Duplikat	50/50	0	3,763275	3,813624	0,919227	29,4297
500	<i>Random Asc</i>	46/50	4	2,615986	2,414942	0,396892	29,4297
500	<i>Random Desc</i>	49/50	1	2,964531	2,599001	0,705699	29,4297
1.000	Sudah Urut	50/50	0	4,435458	3,975630	0,990744	53,4141
1.000	Duplikat	50/50	0	6,549382	5,770445	1,561969	53,5391
1.000	<i>Random Asc</i>	50/50	0	6,235733	5,753994	1,182181	53,5391
1.000	<i>Random Desc</i>	44/50	6	5,618345	5,379081	0,560469	53,5391
5.000	Sudah Urut	50/50	0	27,659516	24,467468	7,191716	301,7422
5.000	Duplikat	50/50	0	65,071449	70,680499	28,294781	301,8672
5.000	<i>Random Asc</i>	50/50	0	46,954765	44,885993	11,522801	301,8672
5.000	<i>Random Desc</i>	48/50	2	42,330955	38,846970	9,799106	301,8672

Tabel 4.13 menunjukkan bahwa waktu eksekusi *Merge Sort* meningkat seiring bertambahnya jumlah data. Rata-rata waktu pada dataset 100 masih berada di bawah 1 ms, sedangkan dataset 5.000 membutuhkan waktu antara 27,659516 ms dan 65,071449 ms. Skenario data duplikat menghasilkan waktu tertinggi pada dataset 5.000, sementara data sudah urut menghasilkan waktu terendah.

Penggunaan memori *Merge Sort* meningkat dari sekitar 6,6328 KB pada 100 data menjadi sekitar 301,8672 KB pada 5.000 data. Susunan awal data tidak menimbulkan perbedaan memori yang besar karena kebutuhan memori lebih dipengaruhi oleh jumlah data yang diproses.

4.4.3 Hasil Pengujian *Sequential Search*

Pengujian *Sequential Search* menggunakan empat skenario, yaitu kata kunci ditemukan, kata kunci tidak ditemukan, kata kunci kosong, dan kata kunci yang menghasilkan data duplikat. Algoritma memeriksa data secara berurutan berdasarkan NIM atau nama praktikan. Ringkasan hasil pengujian *Sequential Search* disajikan pada Tabel 4.14.

Tabel 4.13 Hasil Pengujian *Sequential Search*

Data	Skenario	Run Bersih	IQR	Mean (ms)	Median (ms)	Std. Deviasi	Memori (KB)
100	Duplikat	49/50	1	0,046808	0,041008	0,013003	0,5000
100	Kosong	47/50	3	0,005438	0,005007	0,001073	0,3672
100	Keyword Ada	49/50	1	0,064261	0,070095	0,014585	0,4922
100	Tidak Ada	50/50	0	0,059085	0,067234	0,014293	0,5000
500	Duplikat	50/50	0	0,175257	0,153422	0,040115	0,5000
500	Kosong	47/50	3	0,007401	0,006914	0,001847	0,3672
500	Keyword Ada	43/50	7	0,171772	0,157118	0,029586	0,4922
500	Tidak Ada	45/50	5	0,150961	0,148058	0,013216	0,5000
1.000	Duplikat	46/50	4	0,446066	0,456452	0,097324	0,5000
1.000	Kosong	47/50	3	0,007858	0,007868	0,001958	0,3672
1.000	Keyword Ada	41/50	9	0,313270	0,303984	0,035915	0,4922
1.000	Tidak Ada	49/50	1	0,411798	0,421047	0,102954	0,5000
5.000	Duplikat	48/50	2	3,391112	3,433943	1,522373	0,5000
5.000	Kosong	48/50	2	0,009820	0,010014	0,001995	0,3672
5.000	Keyword Ada	41/50	9	4,533239	4,477978	0,384047	0,4922
5.000	Tidak Ada	49/50	1	2,203659	1,983166	0,570995	0,5000

Tabel 4.14 menunjukkan bahwa waktu *Sequential Search* meningkat ketika algoritma harus memeriksa lebih banyak data. Skenario kata kunci ditemukan pada dataset 5.000 menghasilkan waktu rata-rata tertinggi sebesar 4,533239 ms. Skenario kata kunci kosong tetap berada di bawah 0,01 ms karena sistem tidak menjalankan pemeriksaan kata kunci terhadap setiap data.

Penggunaan memori *Sequential Search* berada pada kisaran 0,3672 KB sampai 0,5000 KB. Nilai tersebut relatif stabil pada seluruh ukuran *dataset* karena algoritma hanya menggunakan kata kunci, penghitung operasi, dan variabel penampung hasil selama proses pencarian.

4.4.4 Hasil Pengujian *Baseline* MySQL

Pengujian *baseline* MySQL membandingkan algoritma pada lapisan aplikasi dengan mekanisme pengolahan data pada basis data. MySQL *ORDER BY* menjadi *baseline* pengurutan, sedangkan MySQL *WHERE/LIKE* menjadi *baseline* pencarian. Pengujian menggunakan ukuran *dataset* dan skenario yang sama dengan *Merge Sort* dan *Sequential Search*.

Tabel 4.14 Hasil Pengujian MySQL *ORDER BY*

Data	Skenario	Run Bersih	IQR	Mean (ms)	Median (ms)	Std. Deviasi	Memori (KB)
100	Sudah Urut	50/50	0	10,254135	9,817958	2,241877	109,0000
100	Duplikat	50/50	0	12,328019	13,504505	2,584452	109,0000
100	<i>Random Asc</i>	45/50	5	8,529128	8,137941	1,152699	109,0000
100	<i>Random Desc</i>	50/50	0	9,335771	8,585453	2,065762	109,0000
500	Sudah Urut	44/50	6	11,126735	10,903358	0,669310	417,9063
500	Duplikat	50/50	0	16,423674	16,201973	2,959052	417,9063
500	<i>Random Asc</i>	46/50	4	11,556641	11,066079	1,419895	417,9063
500	<i>Random Desc</i>	50/50	0	13,126674	12,095332	2,673058	417,9063
1.000	Sudah Urut	50/50	0	24,660392	23,040414	4,170262	804,3516
1.000	Duplikat	48/50	2	23,396651	21,789074	3,973366	804,3516
1.000	<i>Random Asc</i>	48/50	2	22,366285	21,441936	2,944801	804,3516
1.000	<i>Random Desc</i>	47/50	3	21,854071	21,004915	2,411894	804,3516
5.000	Sudah Urut	50/50	0	76,059265	70,047975	18,004447	3.863,9141
5.000	Duplikat	50/50	0	88,988743	88,991523	22,473748	3.863,9141
5.000	<i>Random Asc</i>	50/50	0	80,542226	78,994989	18,124854	3.863,9141
5.000	<i>Random Desc</i>	50/50	0	74,959998	72,077990	15,732227	3.863,9141

Tabel 4.15 menunjukkan bahwa waktu eksekusi MySQL *ORDER BY* meningkat dari kisaran 8,529128–12,328019 ms pada 100 data menjadi 74,959998–88,988743 ms pada 5.000 data. Penggunaan memori juga meningkat dari 109,0000 KB menjadi 3.863,9141 KB karena MySQL perlu menjalankan *query*, mengambil data, mengurutkan hasil, dan mengirimkannya ke lapisan aplikasi.

Tabel 4.15 Hasil Pengujian MySQL *WHERE/LIKE*

Data	Skenario	Run Bersih	IQR	Mean (ms)	Median (ms)	Std. Deviasi	Memori (KB)
100	Duplikat	49/50	1	9,427586	8,203030	2,228221	46,3516
100	Kosong	47/50	3	8,601797	8,002043	1,345663	109,0000
100	<i>Keyword Ada</i>	50/50	0	11,442280	11,888981	2,241592	44,4766
100	Tidak Ada	50/50	0	11,346302	11,885524	2,508893	43,6797
500	Duplikat	50/50	0	11,611447	11,188984	1,989577	56,8672
500	Kosong	48/50	2	11,939084	11,714458	1,465885	417,9063

Tabel 4.16 Hasil Pengujian MySQL *WHERE/LIKE* (Lanjutan)

Data	Skenario	Run Bersih	IQR	Mean (ms)	Median (ms)	Std. Deviasi	Memori (KB)
500	Keyword Ada	49/50	1	11,281948	10,859013	1,593332	44,4766
500	Tidak Ada	48/50	2	10,420829	10,242462	0,897377	43,6797
1.000	Duplikat	49/50	1	27,122108	28,384924	3,869128	70,0547
1.000	Kosong	49/50	1	22,919280	22,855043	3,501134	804,3516
1.000	Keyword Ada	48/50	2	17,846122	16,329527	2,859697	44,4766
1.000	Tidak Ada	50/50	0	22,542062	20,940900	4,620788	43,6797
5.000	Duplikat	49/50	1	56,345078	53,123951	14,170824	193,5547
5.000	Kosong	49/50	1	58,600547	56,957960	11,623873	3.863,9141
5.000	Keyword Ada	49/50	1	60,028874	60,711861	11,905579	44,4766
5.000	Tidak Ada	46/50	4	46,129999	44,481397	7,074500	43,6797

Tabel 4.16 menunjukkan bahwa waktu MySQL *WHERE/LIKE* meningkat ketika jumlah data bertambah. Rata-rata waktu pada *dataset* 5.000 berada pada kisaran 46,129999–60,028874 ms. Skenario kata kunci kosong menggunakan memori paling besar karena *query* mengembalikan seluruh data yang tersedia.

Skenario kata kunci ditemukan dan tidak ditemukan menggunakan memori yang relatif stabil karena *query* hanya menghasilkan sedikit data. Skenario duplikat mengalami peningkatan memori karena sistem menerima lebih dari satu hasil yang memiliki kata kunci sama.

4.4.5 Perbandingan Kinerja Algoritma dan MySQL

Perbandingan kinerja menggunakan rata-rata dari seluruh skenario pada setiap ukuran *dataset*. Nilai tersebut berasal dari data bersih setelah *outlier* IQR dikeluarkan. Perbandingan *sorting* disajikan pada Tabel 4.17.

Tabel 4.16 Perbandingan *Merge Sort* dan MySQL *ORDER BY*

Jumlah Data	<i>Merge Sort</i> (ms)	MySQL <i>ORDER BY</i> (ms)	Memori <i>Merge Sort</i> (KB)	Memori MySQL (KB)
100	0,449815	10,111763	6,6328	109,0000
500	2,758817	13,058431	29,3984	417,9063
1.000	5,709730	23,069350	53,5078	804,3516
5.000	45,504172	80,137558	301,8359	3.863,9141

Tabel 4.17 menunjukkan bahwa *Merge Sort* menghasilkan rata-rata waktu eksekusi lebih rendah daripada MySQL *ORDER BY* pada seluruh ukuran *dataset*. *Merge Sort* memproses data yang telah tersedia pada memori aplikasi, sedangkan MySQL

ORDER BY mencakup proses *query*, pengambilan data, pengurutan, dan pengiriman hasil ke aplikasi.

Merge Sort juga menggunakan memori lebih rendah. Pada dataset 5.000, *Merge Sort* menggunakan rata-rata memori sekitar 301,8359 KB, sedangkan MySQL *ORDER BY* menggunakan 3.863,9141 KB. Perbedaan tersebut menunjukkan bahwa proses *query* basis data memiliki beban tambahan yang tidak terdapat pada pengurutan data di lapisan aplikasi.

Tabel 4.17 Perbandingan *Sequential Search* dan MySQL *WHERE/LIKE*

Jumlah Data	<i>Sequential Search</i> (ms)	MySQL <i>WHERE/LIKE</i> (ms)	Memori <i>Sequential Search</i> (KB)	Memori MySQL (KB)
100	0,043898	10,204491	0,4648	60,8770
500	0,126348	11,313327	0,4648	140,7324
1.000	0,294748	22,607393	0,4648	240,6406
5.000	2,534457	55,276125	0,4648	1.036,4063

Tabel 4.18 menunjukkan bahwa *Sequential Search* memiliki rata-rata waktu lebih rendah daripada MySQL *WHERE/LIKE*. *Sequential Search* membutuhkan 2,534457 ms pada *dataset* 5.000, sedangkan MySQL *WHERE/LIKE* membutuhkan 55,276125 ms. Penggunaan memori *Sequential Search* juga relatif tetap pada seluruh ukuran dataset.

Perbandingan tersebut tidak hanya menunjukkan perbedaan algoritma, tetapi juga menunjukkan perbedaan lapisan pemrosesan. *Sequential Search* memeriksa data yang telah tersedia dalam memori aplikasi, sedangkan MySQL *WHERE/LIKE* menjalankan *query* terhadap basis data dan mengembalikan hasilnya ke aplikasi. Kedua metode tetap memiliki fungsi berbeda sesuai lokasi data yang sedang diproses.

4.4.6 Statistik Deskriptif dan Pembersihan *Outlier* IQR

Sistem menghasilkan 3.200 data pengujian utama dari seluruh kombinasi ukuran *dataset*, metode, dan skenario. Proses IQR mengeluarkan 129 data yang berada di luar batas kewajaran, sehingga sistem menggunakan 3.071 data bersih untuk perhitungan statistik dan pembentukan grafik. Seluruh data bersih tersebut memiliki status valid, sedangkan data tidak valid berjumlah 0. Hasil penyaringan ini memastikan data yang digunakan dalam analisis telah melalui proses pemeriksaan terhadap nilai pencilan. Dengan demikian, hasil statistik dan grafik yang diperoleh didasarkan pada data pengujian yang telah dibersihkan dan dinyatakan valid.

Tabel 4.18 Hasil Pembersihan *Outlier* Berdasarkan Ukuran *Dataset*

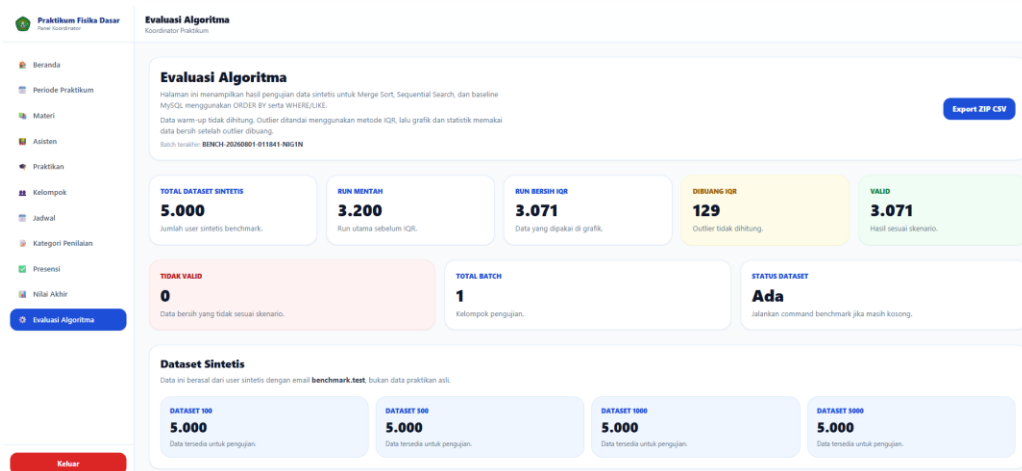
Jumlah Data	<i>Run</i> Mentah	<i>Run</i> Bersih	<i>Outlier</i> IQR
100	800	769	31
500	800	759	41
1.000	800	766	34
5.000	800	777	23
Total	3.200	3.071	129

Tabel 4.19 menunjukkan bahwa *dataset* 500 memiliki jumlah *outlier* terbanyak, yaitu 41 data, sedangkan *dataset* 5.000 memiliki jumlah *outlier* paling sedikit, yaitu 23 data. Sistem tidak menggunakan data *outlier* dalam perhitungan *mean*, *median*, standar deviasi, dan grafik agar lonjakan pengukuran tidak memengaruhi hasil akhir.

Sistem menggunakan batas bawah $Q1 - 1,5 \times IQR$ dan batas atas $Q3 + 1,5 \times IQR$ untuk menentukan *outlier*. Nilai yang berada di dalam batas tersebut tetap digunakan sebagai data bersih. Proses ini menghasilkan perhitungan yang lebih mewakili kecenderungan umum dari setiap metode dan skenario.

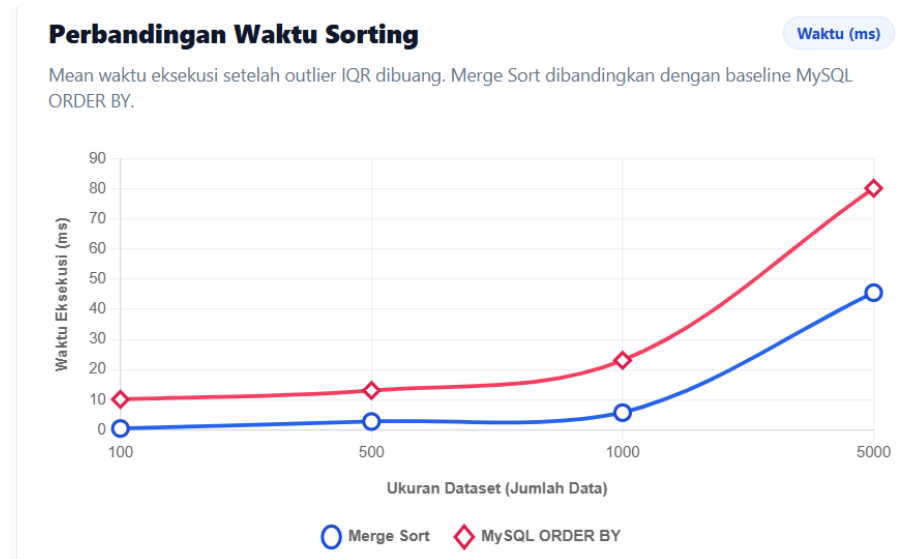
4.4.7 Visualisasi Hasil Pengujian

Sistem menampilkan ringkasan hasil evaluasi yang memuat jumlah *dataset* sintetis, *run* mentah, *run* bersih, *outlier*, data valid, data tidak valid, jumlah *batch*, dan status *dataset*. Halaman tersebut juga menyediakan fitur ekspor ZIP yang berisi data mentah, data bersih, statistik ringkasan, dan *dataset* sintetis. Tampilan ringkasan hasil pengujian ditunjukkan pada Gambar 4.28.



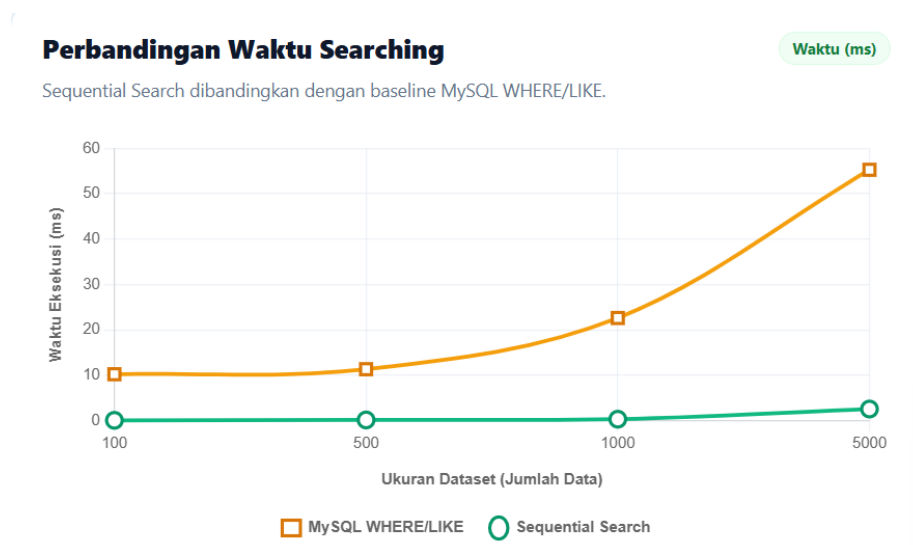
Gambar 4.28 Ringkasan Hasil Evaluasi Algoritma

Grafik perbandingan waktu *sorting* memperlihatkan bahwa waktu eksekusi *Merge Sort* dan MySQL *ORDER BY* meningkat seiring bertambahnya ukuran *dataset*. *Merge Sort* menghasilkan waktu lebih rendah pada seluruh ukuran *dataset*, sedangkan MySQL *ORDER BY* mengalami peningkatan paling tinggi pada *dataset* 5.000. Hasil perbandingan tersebut ditunjukkan pada Gambar 4.29.



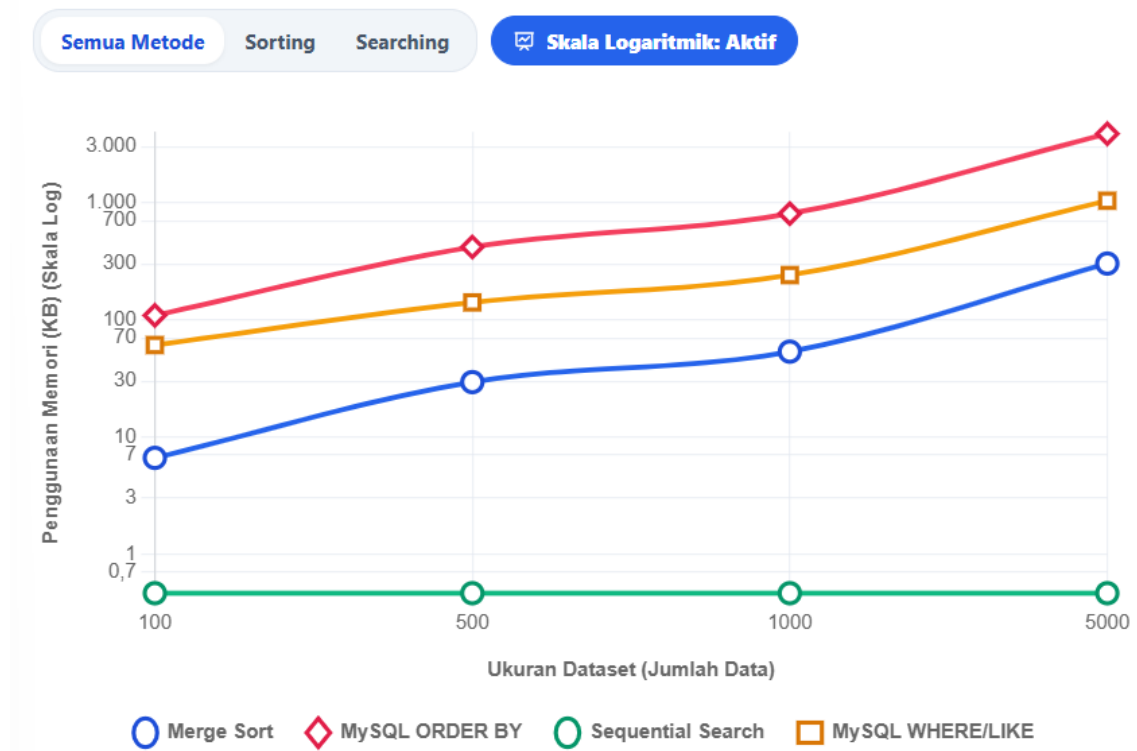
Gambar 4.29 Perbandingan Waktu *Sorting*

Grafik perbandingan waktu *searching* menunjukkan bahwa *Sequential Search* memiliki waktu eksekusi lebih rendah dibandingkan MySQL *WHERE/LIKE*. Selisih waktu kedua metode semakin terlihat pada *dataset* 1.000 dan 5.000. Hasil perbandingan tersebut ditunjukkan pada Gambar 4.30.



Gambar 4.30 Perbandingan Waktu *Searching*

Grafik penggunaan memori menunjukkan bahwa kebutuhan memori meningkat seiring bertambahnya ukuran *dataset*. MySQL *ORDER BY* menggunakan memori paling besar, disusul MySQL *WHERE/LIKE* menempati urutan berikutnya dan mengalami kenaikan yang cukup jelas pada dataset 5.000.



Gambar 4.31 Perbandingan Penggunaan Memori

Merge Sort mengalami peningkatan penggunaan memori, tetapi nilainya masih lebih rendah dibandingkan kedua metode MySQL. *Sequential Search* menggunakan memori paling rendah dan relatif stabil pada seluruh ukuran *dataset*. Hasil perbandingan penggunaan memori ditunjukkan pada Gambar 4.31.

4.5 Pembahasan

Pembahasan penelitian ini membahas hasil implementasi dan pengujian sistem berdasarkan rumusan masalah yang telah ditetapkan. Analisis mencakup performa *Merge Sort*, performa *Sequential Search*, perbandingan algoritma pada lapisan aplikasi dengan *baseline* MySQL, serta penggunaan waktu eksekusi dan memori. Pembahasan juga menghubungkan hasil pengujian fungsional dan kinerja algoritma dengan tujuan pengembangan sistem pengelolaan data praktikan berbasis web.

4.5.1 Pembahasan Performa *Merge Sort*

Hasil pengujian menunjukkan bahwa *Merge Sort* mampu mengurutkan data praktikan pada lapisan aplikasi dengan waktu eksekusi yang meningkat seiring bertambahnya ukuran *dataset*. *Dataset* 100 menghasilkan waktu rata-rata di bawah 1 ms pada seluruh skenario, sedangkan *dataset* 5.000 menghasilkan waktu antara 27,659516 ms dan 65,071449 ms. Peningkatan tersebut terjadi karena algoritma harus membagi, membandingkan, dan menggabungkan lebih banyak elemen ketika jumlah data bertambah.

Skenario data sudah urut menghasilkan waktu terendah pada *dataset* 5.000, yaitu 27,659516 ms. Skenario data duplikat menghasilkan waktu tertinggi sebesar 65,071449 ms, sedangkan skenario *Random Asc* dan *Random Desc* menghasilkan waktu masing-masing sebesar 46,954765 ms dan 42,330955 ms. Perbedaan tersebut menunjukkan bahwa kondisi data memengaruhi proses perbandingan dan penggabungan meskipun *Merge Sort* tetap menggunakan tahapan pembagian data yang sama.

Penggunaan memori *Merge Sort* meningkat dari sekitar 6,6328 KB pada *dataset* 100 menjadi sekitar 301,8672 KB pada *dataset* 5.000. Kenaikan tersebut terjadi karena *Merge Sort* membutuhkan ruang tambahan untuk menyimpan bagian kiri dan kanan selama proses pembagian serta penggabungan data. Nilai memori pada setiap skenario dalam ukuran *dataset* yang sama relatif serupa, sehingga jumlah data memberikan pengaruh lebih besar dibandingkan susunan awal data.

Hasil tersebut sesuai dengan karakteristik *Merge Sort* yang memiliki pola pemrosesan stabil pada berbagai kondisi data. Penelitian ini menerapkan *Merge Sort* secara langsung pada fitur pengurutan data praktikan berdasarkan program studi dan NIM. Penerapan tersebut tidak hanya menguji algoritma secara terpisah, tetapi juga menunjukkan penggunaannya pada sistem praktikum berbasis web.

4.5.2 Pembahasan Performa *Sequential Search*

Hasil pengujian menunjukkan bahwa waktu eksekusi *Sequential Search* meningkat ketika jumlah data yang diperiksa semakin besar. *Dataset* 100 menghasilkan waktu rata-rata antara 0,005438 ms dan 0,064261 ms. *Dataset* 5.000 menghasilkan waktu antara 0,009820 ms dan 4,533239 ms sesuai dengan skenario pencarian yang dijalankan.

Skenario kata kunci ditemukan menghasilkan waktu tertinggi pada *dataset* 5.000, yaitu 4,533239 ms. Skenario duplikat menghasilkan waktu sebesar 3,391112 ms, sedangkan skenario tidak ditemukan menghasilkan waktu sebesar 2,203659 ms. Perbedaan tersebut muncul karena jumlah elemen yang diperiksa dan jumlah hasil yang dikembalikan dapat berbeda pada setiap skenario.

Skenario kata kunci kosong menghasilkan waktu paling rendah pada seluruh ukuran *dataset*. Sistem tidak menjalankan pemeriksaan kecocokan kata kunci secara penuh ketika pengguna tidak memasukkan kata kunci, sehingga waktu eksekusinya hanya meningkat dari 0,005438 ms pada 100 data menjadi 0,009820 ms pada 5.000 data. Kondisi tersebut berbeda dengan pencarian kata kunci yang mengharuskan sistem memeriksa NIM atau nama praktikan secara berurutan.

Penggunaan memori Sequential Search berada pada kisaran 0,3672 KB sampai 0,5000 KB. Nilai tersebut relatif stabil karena algoritma hanya menggunakan kata kunci, variabel penghitung, dan tempat penyimpanan hasil pencarian. *Sequential Search* tidak memerlukan data yang sudah terurut sehingga sistem dapat langsung mencari NIM atau nama praktikan pada urutan data yang tersedia.

Hasil pengujian tersebut sesuai dengan karakteristik *Sequential Search* yang memeriksa elemen secara berurutan. Waktu pencarian cenderung meningkat ketika jumlah data bertambah, tetapi hasil pengujian menunjukkan bahwa algoritma masih dapat memproses *dataset* 5.000 dengan waktu di bawah 5 ms. Penerapan tersebut mendukung kebutuhan pencarian data praktikan pada lapisan aplikasi tanpa mewajibkan proses pengurutan sebelumnya.

4.5.3 Perbandingan Algoritma dengan *Baseline* MySQL

Hasil perbandingan menunjukkan bahwa algoritma aplikasi dan *baseline* MySQL memiliki karakteristik pemrosesan yang berbeda. *Merge Sort* dan *Sequential Search* bekerja setelah data tersedia pada memori aplikasi. MySQL *ORDER BY* dan *WHERE/LIKE* bekerja melalui *query* yang melibatkan pembacaan data dari basis data serta pengiriman hasil ke aplikasi.

Merge Sort menghasilkan waktu rata-rata yang lebih rendah dibandingkan MySQL *ORDER BY* pada seluruh ukuran *dataset*. *Dataset* 100 menghasilkan rata-rata

waktu *Merge Sort* sekitar 0,449815 ms dan MySQL *ORDER BY* sekitar 10,111763 ms. *Dataset* 5.000 menghasilkan rata-rata waktu *Merge Sort* sekitar 45,504172 ms, sedangkan MySQL *ORDER BY* membutuhkan sekitar 80,137558 ms.

Perbedaan tersebut terjadi karena *Merge Sort* hanya mengurutkan data yang telah dimuat ke memori aplikasi. MySQL *ORDER BY* harus menjalankan *query*, membaca data dari tabel, mengurutkan data, dan mengembalikan hasilnya ke aplikasi. Hasil ini tidak menunjukkan bahwa *Merge Sort* dapat menggantikan MySQL, tetapi menunjukkan keuntungan pemrosesan pada lapisan aplikasi ketika data sudah tersedia di memori.

Sequential Search juga menghasilkan waktu lebih rendah dibandingkan MySQL *WHERE/LIKE*. Rata-rata waktu *Sequential Search* meningkat dari sekitar 0,043898 ms pada *dataset* 100 menjadi 2,534457 ms pada *dataset* 5.000. MySQL *WHERE/LIKE* meningkat dari sekitar 10,204491 ms menjadi 55,276125 ms pada ukuran *dataset* yang sama.

Perbandingan pencarian perlu mempertimbangkan lokasi pemrosesan data. *Sequential Search* memeriksa data yang telah tersedia di aplikasi, sedangkan MySQL *WHERE/LIKE* melakukan pencarian secara langsung pada basis data. MySQL tetap berperan penting ketika sistem membutuhkan data terbaru yang belum dimuat ke aplikasi, sedangkan *Sequential Search* mendukung pencarian lanjutan terhadap kumpulan data yang telah tersedia.

4.5.4 Analisis Waktu Eksekusi, Memori, dan Pembersihan Data

Waktu eksekusi menunjukkan durasi yang dibutuhkan setiap metode untuk menyelesaikan proses pengurutan atau pencarian. *Merge Sort* mengalami kenaikan waktu karena seluruh data harus dibagi dan digabungkan kembali. *Sequential Search* mengalami kenaikan waktu ketika sistem harus memeriksa lebih banyak elemen. *Baseline* MySQL membutuhkan waktu tambahan karena prosesnya melibatkan komunikasi antara aplikasi dan basis data.

Penggunaan memori menunjukkan tambahan sumber daya selama pengujian berlangsung. *Merge Sort* menggunakan memori yang meningkat dari sekitar 6,6328 KB menjadi 301,8672 KB. *Sequential Search* menggunakan memori yang rendah dan relatif stabil. MySQL *ORDER BY* menggunakan memori paling besar hingga mencapai

3.863,9141 KB pada *dataset* 5.000, sedangkan penggunaan memori MySQL *WHERE/LIKE* bergantung pada jumlah data yang dikembalikan oleh *query*.

Skenario kata kunci kosong pada MySQL *WHERE/LIKE* menggunakan memori lebih besar karena *query* mengembalikan seluruh data. Skenario kata kunci ditemukan, tidak ditemukan, dan duplikat menggunakan memori lebih rendah karena hasil *query* yang dikembalikan lebih sedikit. Kondisi tersebut menunjukkan bahwa penggunaan memori MySQL dipengaruhi oleh jumlah data hasil *query*, bukan hanya jumlah data yang tersedia pada *dataset*.

Sistem menjalankan 3.200 pengujian utama dari seluruh kombinasi metode, ukuran *dataset*, dan skenario. Metode IQR mengeluarkan 129 hasil yang berada di luar batas kewajaran, sehingga sebanyak 3.071 hasil bersih digunakan dalam statistik dan grafik. Proses pembersihan tersebut mengurangi pengaruh lonjakan waktu yang dapat terjadi akibat kondisi sementara pada *server* atau lingkungan pengujian.

Nilai *mean* menunjukkan rata-rata hasil pengujian, median menunjukkan nilai tengah, dan standar deviasi menunjukkan tingkat penyebaran hasil. Nilai *mean* dan median yang berdekatan menunjukkan hasil yang relatif konsisten, sedangkan standar deviasi yang lebih besar menunjukkan variasi waktu yang lebih tinggi. Penggunaan beberapa ukuran statistik membuat hasil pengujian tidak hanya bergantung pada satu nilai rata-rata.

4.5.5 Implikasi terhadap Sistem Praktikum Berbasis Web

Hasil implementasi menunjukkan bahwa sistem berbasis web mampu memusatkan pengelolaan data praktikum dalam satu basis data yang mencakup data praktikan, asisten, periode, kelompok, jadwal, materi, presensi, penilaian, dan nilai akhir. Pemusatan ini mempermudah pencarian dan rekapitulasi serta mengurangi penggunaan *spreadsheet* terpisah. *Merge Sort* digunakan untuk mengurutkan data praktikan berdasarkan program studi dan NIM, sedangkan *Sequential Search* digunakan untuk mencari data berdasarkan NIM atau nama tanpa memerlukan data terurut. Kedua algoritma diterapkan pada lapisan aplikasi setelah data diperoleh dari basis data. Hasil pengujian fungsional menunjukkan bahwa autentikasi, hak akses, pengelolaan data, presensi, penilaian, pencarian, pengurutan, penyaringan, dan evaluasi algoritma berjalan sesuai skenario. Setiap

pengguna memperoleh fitur sesuai kewenangannya, sehingga fungsi utama sistem telah berjalan sesuai rancangan.

Hasil pengujian kinerja menunjukkan bahwa *Merge Sort* dan *Sequential Search* mampu memproses dataset hingga 5.000 entri dengan waktu relatif rendah. *Merge Sort* digunakan untuk mengurutkan data yang tersedia di aplikasi, sedangkan *Sequential Search* untuk pencarian fleksibel berdasarkan NIM atau nama. MySQL tetap menjadi sumber utama penyimpanan dan pengambilan data melalui *ORDER BY* serta *WHERE/LIKE*. Penerapan algoritma aplikasi dan mekanisme MySQL saling melengkapi, dengan MySQL menangani penyimpanan dan pengambilan data, sedangkan *Merge Sort* dan *Sequential Search* menangani pemrosesan lanjutan pada data yang telah dimuat. Hasil ini menunjukkan bahwa sistem dapat mendukung pengelolaan data praktikan secara terstruktur sekaligus memberikan gambaran kinerja antara lapisan aplikasi dan basis data.

```
$isWarmup = $run <= $warmup;

PerformanceLog::query()->create([
    'batch_code' => $batchCode,
    'timestamp' => now(),
    'dataset_size' => $size,
    'test_type' => $testType,
    'method' => $method,
    'scenario' => $scenario,
    'field_name' => $fieldName,
    'direction' => $direction,
    'keyword' => $keyword,
    'run_order' => $run,
    'is_warmup' => $isWarmup,
    'is_outlier' => false,

    'sort_time' => $method === 'merge_sort_application'
        ? $measurement['time_ms']
        : null,
    'search_time' => $method === 'sequential_search_application'
        ? $measurement['time_ms']
        : null,
    'baseline_time' => in_array($method, [
        'mysql_order_by',
        'mysql_where_like',
    ])
        ? $measurement['time_ms']
        : null,

    'memory_usage' => $measurement['memory_kb'] ?? 0,
    'iteration_count' => $measurement['iterations'] ?? 0,
    'comparison_count' => $measurement['comparisons'] ?? 0,
    'result_count' => $measurement['result_count'] ?? 0,
    'is_valid' => $measurement['is_valid'] ?? true,
```

Source Code 4.9 Pencatatan Hasil Pengujian Algoritma pada Tabel Log

```

        'explain_result' => $measurement['explain_result'] ?? null,
    ]);

    if (!$isWarmup && $method === 'merge_sort_application') {
        LogSorting::query()->create([
            'jumlah_data' => $size,
            'field_pengurutan' => $fieldName,
            'arah_pengurutan' => $direction,
            'waktu_pemrosesan_server' => $measurement['time_ms'],
            'jumlah_perbandingan' => $measurement['comparisons'] ?? 0,
            'penggunaan_memori' => $measurement['memory_kb'] ?? 0,
            'waktu_pengujian' => now(),
            'keterangan_pengujian' => $scenario,
        ]);
    }

    if (!$isWarmup && $method === 'sequential_search_application') {
        LogSearch::query()->create([
            'jumlah_data' => $size,
            'field_pencarian' => $fieldName,
            'kata_kunci_pencarian' => $keyword ?? '',
            'waktu_pemrosesan_server' => $measurement['time_ms'],
            'jumlah_iterasi' => $measurement['iterations'] ?? 0,
            'jumlah_perbandingan' => $measurement['comparisons'] ?? 0,
            'penggunaan_memori' => $measurement['memory_kb'] ?? 0,
            'waktu_pengujian' => now(),
            'keterangan_pengujian' => $scenario,
        ]);
    }
}

```

Source Code 4.9 Pencatatan Hasil Pengujian Algoritma pada Tabel Log (Lanjutan)

Source Code 4.9 menunjukkan proses penyimpanan hasil pengujian berdasarkan metode yang dijalankan. Tabel `performance_logs` menyimpan seluruh hasil *Merge Sort*, *Sequential Search*, *MySQL ORDER BY*, dan *MySQL WHERE/LIKE*. Kolom `sort_time`, `search_time`, dan `baseline_time` menyimpan waktu eksekusi sesuai metode pengujian. Sistem juga mencatat ukuran *dataset*, skenario, penggunaan memori, jumlah iterasi, jumlah perbandingan, jumlah hasil pencarian, validitas, status *warm-up*, dan hasil EXPLAIN MySQL.

Sistem menyimpan hasil utama *Merge Sort* secara khusus pada tabel `log_sortings`, sedangkan hasil utama *Sequential Search* disimpan pada tabel `log_searches`. Kondisi `!$isWarmup` mencegah data *warm-up* masuk ke kedua tabel tersebut. Tabel `performance_logs` tetap dapat mencatat data *warm-up* melalui nilai `is_warmup` agar sistem dapat membedakannya dari pengujian utama ketika melakukan analisis.

BAB V

KESIMPULAN DAN SARAN

5.1 Kesimpulan

Penelitian ini menghasilkan sistem pengelolaan Praktikum Fisika Dasar berbasis web dengan penerapan *Merge Sort* dan *Sequential Search*. Berdasarkan hasil implementasi dan pengujian, kesimpulan penelitian ini adalah sebagai berikut.

1. Sistem berhasil mengelola data praktikan, asisten, periode, kelompok, materi, jadwal, presensi, penilaian, dan nilai akhir secara terpusat. Hasil *Black Box Testing* menunjukkan bahwa fitur utama dan pembagian hak akses berjalan sesuai rancangan.
2. *Merge Sort* berhasil mengurutkan data praktikan berdasarkan program studi dan NIM. Waktu eksekusi meningkat dari 0,383057 – 0,592618 ms pada 100 data menjadi 27,659516–65,071449 ms pada 5.000 data. Penggunaan memorinya meningkat dari sekitar 6,6328 KB menjadi 301,8672 KB.
3. *Sequential Search* berhasil mencari data berdasarkan NIM atau nama tanpa memerlukan data terurut. Waktu eksekusi berada pada rentang 0,005438–0,064261 ms pada 100 data dan 0,009820–4,533239 ms pada 5.000 data. Penggunaan memorinya relatif stabil pada kisaran 0,3672–0,5000 KB.
4. Hasil pengujian menunjukkan bahwa *Merge Sort* dan *Sequential Search* memiliki waktu eksekusi serta penggunaan memori lebih rendah dibandingkan *baseline* MySQL pada lingkungan pengujian. Kedua algoritma tersebut melengkapi fungsi MySQL dalam pengolahan data pada lapisan aplikasi.

5.2 Saran

Penelitian ini masih dapat dikembangkan melalui penambahan ukuran *dataset*, perluasan variasi skenario pengujian, dan peningkatan fitur sistem. Pengembangan tersebut dapat memberikan gambaran kinerja *Merge Sort*, *Sequential Search*, dan *baseline* MySQL pada kondisi yang lebih beragam. Berdasarkan hasil penelitian, beberapa saran yang dapat dipertimbangkan adalah sebagai berikut.

1. Penelitian selanjutnya disarankan untuk memperluas pengujian performa menggunakan variasi dataset skala besar secara bertingkat dan spesifik, seperti 10.000, 25.000, 50.000, hingga 100.000 data entri. Rentang pengujian yang lebih luas ini diperlukan untuk mengamati secara nyata titik infleksi (*divergence point*) penurunan performa algoritma $O(n \log n)$ pada *Merge Sort* dan $O(n)$ ada *Sequential Search*, serta mendeteksi ambang batas kehabisan alokasi memori (*memory exhaustion threshold*) pada server berbasis PHP/Laravel sebelum sistem mengalami kegagalan proses (*fatal error*).
2. Penelitian berikutnya dapat menjalankan sistem pada *server* produksi atau layanan *hosting* agar hasil pengujian mencerminkan pengaruh spesifikasi *server*, kestabilan jaringan, titik batas memori (*memory exhaustion*), dan jumlah pengguna yang mengakses sistem secara bersamaan.
3. Pengembangan sistem selanjutnya dapat menambahkan perbandingan dengan algoritma pengurutan lain (seperti *Bubble Sort*) dan algoritma pencarian lain (seperti *Sequential Search*) untuk memperkaya analisis efisiensi pada sistem informasi berbasis web.
4. Penelitian berikutnya disarankan untuk melakukan pengujian keamanan (*security testing*), pengujian beban (*load testing*), dan pengujian ketahanan (*stress testing*) agar kesiapan sistem sebelum diterapkan pada skala operasional riil dapat dipastikan.

DAFTAR PUSTAKA

1. Aditya, M., Zenitha, H., Wahyu, S., Novala, B., Muhammad, D., & Khudori, A. N. (2025). *JESICA: Journal of Enhance Studies in Informatics and Computer Applications Performance Analysis of Sorting Algorithm in Student Data Processing*. 2(2), 52–61.
2. Aleryani, A. Y. (2024). Analyzing Data Flow: A Comparison between Data Flow Diagrams (DFD) and User Case Diagrams (UCD) in Information Systems Development. *European Modern Studies Journal*, 8(1), 313–320. [https://doi.org/10.59573/emsj.8\(1\).2024.28](https://doi.org/10.59573/emsj.8(1).2024.28)
3. Anggreani, D., Wibawa, A. P., Purnawansyah, & Herman. (2020). Perbandingan Efisiensi Algoritma Sorting dalam Penggunaan Bandwidth. *ILKOM Jurnal Ilmiah*, 12(2), 96–103. <https://doi.org/10.33096/ilkom.v12i2.538.96-103>
4. Bagus Sadewa, E., Wijiyanto, & Nurohman. (2024). Sistem Informasi Akademik Berbasis Web Pada Smk Al-Islam Surakarta. In *Journal of Information System Management (JOISM) e-ISSN* (Vol. 6, Number 1). <https://doi.org/10.24076/joism.2024v6i1.1647>
5. Datin, Suendri, & Sibarani. (2024). Sistem Informasi Komoditas Harga Pangan Menggunakan Algoritma Sequential Search Berbasis Web pada Perusahaan Umum Daerah Pasar Kota Medan. *INFORMASI (Jurnal Informatika Dan Sistem Informasi)*, 16(120), 32–45. <https://doi.org/10.37424/informasi.v16i1.289>
6. Diana, & Putra, Y. D. (2022). Implementasi Algoritma Sequential Search Pada Aplikasi Pencarian E-Dokumen Berbasis Web. *CESS (Journal of Computing Engineering, System and Science)*, 7, 106–115. <https://doi.org/10.24114/cess.v7i1.26114>

7. Esau Taiwo, O., Christianah, A. O., Oluwatobi, A. N., Aderonke, K. A., & Kehinde, A. J. (2020). Comparative Study Of Two Divide And Conquer Sorting Algorithms: Quicksort And Mergesort. *Procedia Computer Science*, 171, 2532–2540. <https://doi.org/10.1016/j.procs.2020.04.274>
8. Gunawan, A., Wahyuni, N., Kiswindyatmoko, R., B. Katili, P., & R. Barleany, D. (2023). Perancangan Sistem Informasi Pengelolaan Praktikum Berbasis Website. *Jurnal Teknik*, 21(1), 17–31. <https://doi.org/10.37031/jt.v21i1.284>
9. Györödi, C. A., Dumșe-Burescu, D. V., Györödi, R., Zmaranda, D. R., Bandici, L., & Popescu, D. E. (2021). Performance impact of optimization methods on MySQL document-based and relational databases. *Applied Sciences (Switzerland)*, 11(15). <https://doi.org/10.3390/app11156794>
10. Hadiewijaya, A., & Wasito, B. (2024). Perbandingan Efisiensi Waktu dan Memori Pada C# dan Java dengan Metode Benchmarking. *Bit-Tech*, 7(2), 554–560. <https://doi.org/10.32877/bt.v7i2.1906>
11. Khan, S., & Mahadik, S. (2022). A Comparative Study of Agile and Waterfall Software Development Methodologies. *International Journal of Advanced Research in Science, Communication and Technology*, 2, 399–402. <https://doi.org/10.48175/ijarsct-5696>
12. Laaber, C., Yue, T., & Ali, S. (2024). Evaluating Search-Based Software Microbenchmark Prioritization. *IEEE*, 50, 1687–1703. <https://doi.org/10.1109/TSE.2024.3380836>
13. Liu Peike. (2024). An in-depth study of sorting algorithms. *Applied and Computational Engineering*, 92(1), 187–195. <https://doi.org/10.54254/2755-2721/92/20241750>

14. Lontaan, R. J., & Sinadia, A. R. (2024). Design and Development of a Web-Based High School Information System. *COGITO Smart Journal*, 10(2). <http://doi.org/10.31154/cogito.v10i2.858.593-606>
15. Magar, V., Ruikar, D., Bhoite, S., & Mente, R. (2024). Innovative Inter Quartile Range-based Outlier Detection and Removal Technique for Teaching Staff Performance Feedback Analysis. *Journal of Engineering Education Transformations*, 37(3), 176–184. <https://doi.org/10.16920/jeet/2024/v37i3/24013>
16. Melati, A., Oktarina, D., Gusrianty, & Joni Kurniawan, W. (2022). File Search in Employee Archives Using Sequential Search Algorithm. *IT Journal Research and Development*, 7(1), 61–72. <https://doi.org/10.25299/itjrd.2022.9210>
17. Nova, S. (2025). Pembuatan Website Inventory Urban Material: Pendekatan PHP & MYSQL. *Jurnal Minfo Polgan*, 13(2), 2248–2256. <https://doi.org/10.33395/jmp.v13i2.14425>
18. Plevris, V., & Solorzano, G. (2022). A Collection of 30 Multidimensional Functions for Global Optimization Benchmarking. *Data*, 7(4). <https://doi.org/10.3390/data7040046>
19. Pujiono, I. P., Rachmawanto, E. H., & Winarsih, N. A. S. (2025). Array Sorting Algorithm vs Traditional Sorting Algorithm: Memory and Time Efficiency Analysis. *Jurnal Manajemen Informatika (JAMIKA)*, 15(1), 47–59. <https://doi.org/10.34010/jamika.v15i1.13230>
20. Pulungan, S. M., Febrianti, R., Lestari, T., Gurning, N., & Fitriana, N. (2022). Analisis Teknik Entity-Relationship Diagram Dalam Perancangan Database. *01(2)*, 143–147. <https://doi.org/10.47233/jemb.v2i1.533>
21. Putri, N. I. A. G., & Setiawan, R. (2020). Rancang Bangun Aplikasi Elearning. *Jurnal Sistem Informasi Dan Sains Teknologi*, 2(1).

22. Raehan, M., Suwardoyo, U., & Sahidin, S. (2025). Aplikasi Manual Book Alat Berat Berbasis Android. *Jurnal Ilmiah Sistem Informasi Dan Teknik Informatika (JISTI)*, 8(2), 201–207. <https://doi.org/10.57093/jisti.v8i2.329>
23. Rahmawati, L., & Sumarsono, S. (2024). Desain Pengembangan Website dengan Arsitektur Model View Controller pada Framework Laravel. *Jurnal Teknologi Dan Sistem Informasi Bisnis*, 6(4), 785–790. <https://doi.org/10.47233/jteksis.v6i4.1497>
24. Reswan, Y., Darnita, Y., Walad Mahfuzhi, A. R., & Elviani. (2024). Implementation Of Sequential Searching Algorithm In Car Sales E-Commerce Web. *Jurnal Komputer, Informasi Dan Teknologi*, 4(1), 1–14. <https://doi.org/10.53697/jkomitek.v4i1.18>
25. Ridwan, Sunardi, & Alam. (2025). Implementasi Algoritma Sequential Search dalam Pencarian Data Cuti pada Aplikasi Cuti Pegawai. *MALCOM: Indonesian Journal of Machine Learning and Computer Science*, 5(1), 198–206. <https://doi.org/10.57152/malcom.v5i1.1735>
26. Sahar, Y. (2024). Rancang Bangun Sistem Pelayanan Masyarakat Di Kantor Desa Sangtandung Berbasis Website. *Jurnal Riset Sistem Informasi*, 1, 27–36. <https://doi.org/10.69714/9apd5w42>
27. Sallu, S., Harsono, Y., & Fajarianto, O. (2023). *Implementation of Waterfall Method in Model Development to Improve Learning Quality of Computer Network Courses*. 25(3), 496–513. <https://doi.org/10.21009/JTP2001.6>
28. Saputra, M. H., & Kusniyati, H. (2023). Aplikasi Penjualan Berbasis Web Menggunakan Algoritma Selection Sort dan Sequential Search pada Toko Mba Tik. *CESS (Journal of Computer Engineering, System and Science)*, 8, 506–517. <https://doi.org/10.24114/cess.v8i2.46827>

29. Smrti, N. N. E., Andisana, I. P. G. S., Rahayu, N. K. D. T., Adnan, & Juliantara. KW, P. P. O. (2023). Flowgorithm Sebagai Penunjang Pembelajaran Algoritma dan Pemrograman. *Jurnal Bangkit Indonesia*, 12, 56–64. <https://doi.org/10.52771/bangkitindonesia.v12i1.218>
30. Tanjung, A. H., Marlina, L., & Yazid, A. (2025). Pembuatan Aplikasi Pembelajaran Pengembangan Website HTML dan CSS dengan Animasi Interaktif Menggunakan Macromedia Flash 8. *Jurnal Minfo Polgan*, 13(2), 2557–2567. <https://doi.org/10.33395/jmp.v13i2.14544>
31. Traini, L., Cortellessa, V., Di Pompeo, D., & Tucci, M. (2023). Towards effective assessment of steady state performance in Java software: are we there yet? *Empirical Software Engineering*, 28(1). <https://doi.org/10.1007/s10664-022-10247-x>
32. Werner, T., Burchert, J., Stubbemann, M., & Schmidt-Thieme, L. (2024). A Cross-Domain Benchmark for Active Learning. *Advances in Neural Information Processing Systems*, 37. [/https://github.com/wernerth94/A-Cross-Domain-Benchmark-for-Active-Learning](https://github.com/wernerth94/A-Cross-Domain-Benchmark-for-Active-Learning)
33. Wiguna, I. (2020). Implementasi Algoritma Sequential Search Pada Aplikasi Kamus Istilah Informatika. *Jurnal Sistem Informasi Kaputama (JSIK)*, 4(2), 144–150.
34. Yulianti, Y., Khaidar, A. Al, Fazriansyah, R., Ramadhan, S. G., Putra, W. A., & Sitio, S. L. M. (2023). Pengujian Black Box pada Website MyUnpam menggunakan Teknik Equivalence Partitioning. *Jurnal Teknologi Sistem Informasi Dan Aplikasi*, 6(2), 154–161. <https://doi.org/10.32493/jtsi.v6i2.25501>

LAMPIRAN

Lampiran 1 Repositori Kode Sumber Sistem Praktikum Fisika Dasar

Tautan Repositori GitHub : <https://github.com/VistaAtsfi/praktikum-fisdas>

Keterangan : Memuat kode sumber aplikasi Laravel 10, implementasi algoritma Merge Sort dan Sequential Search, database migration, seeder data pengujian, serta perintah pengujian otomatis (Artisan Command).

Lampiran 2 Hasil wawancara dengan Koordinator Praktikum Fisika Dasar

HASIL WAWANCARA PENGELOLAAN DATA PRAKTIKUM FISIKA DASAR FAKULTAS TEKNIK UNIVERSITAS MULAWARMAN

Judul Skripsi : Implementasi dan Evaluasi Kinerja Merge Sort Dan Sequential Search Untuk Pengelolaan Data Praktikan Pada Sistem Praktikum Berbasis Web

Wawancara ini dilakukan dengan narasumber dari Tim Praktikum Fisika Dasar Fakultas Teknik Universitas Mulawarman sebagai koordinator praktikum guna memperoleh informasi terkait proses pengelolaan data praktikan, kebutuhan sistem, serta permasalahan yang dihadapi sebelum penerapan algoritma pengurutan dan pencarian.

A. Data & Pengelolaan Data Praktikum

No	Pertanyaan	Jawaban
1	Data apa saja yang saat ini dicatat dalam praktikum Fisika Dasar?	Data yang dicatat dalam praktikum Fisika Dasar meliputi absensi praktikan, penilaian praktikum, dan penilaian laporan.
2	Bagaimana proses pendaftaran praktikan yang diterapkan saat ini? Media apa saja yang digunakan dalam proses tersebut?	Proses pendaftaran praktikum saat ini dilakukan melalui Google Form yang disebarakan melalui ketua tingkat masing-masing program studi.
3	Data apa saja yang dikumpulkan dari praktikan pada tahap pendaftaran awal?	Pada tahap pendaftaran awal, data yang dikumpulkan meliputi Nama Lengkap, NIM, Email dan No. WhatsApp.
4	Apakah data pendaftaran langsung digunakan atau perlu pengolahan ulang?	Data pendaftaran tidak langsung digunakan, tetapi dicek ulang jumlahnya agar sesuai dengan data mahasiswa dari dosen pengampu dan diverifikasi kebenaran nama lengkap serta NIM praktikan melalui konfirmasi di grup WhatsApp sebelum dipakai untuk administrasi praktikum.
5	Dalam format apa data praktikum disimpan saat ini (Excel, Google Sheet, dll)?	Data praktikum seperti penilaian disimpan dalam format Google Sheet dan dokumen yang bersifat administratif dalam format Word dan PDF.
6	Siapa saja yang terlibat dalam pengelolaan data praktikum?	Pihak yang terlibat dalam pengelolaan data praktikum yaitu asisten dan Kepala Lab.
7	Apakah data disimpan terpisah berdasarkan kelas, modul, atau periode?	Data tersimpan terpisah berdasarkan kelas (program studi) dan periode praktikum.

8	Berapa jumlah data praktikan Fisika Dasar 1 dan Fisika Dasar 2 yang dikelola dalam satu periode praktikum? (sampel tahun: 2025)	Pada satu periode praktikum tahun 2025, jumlah praktikan Fisika Dasar 1 sebanyak 382 mahasiswa dan praktikan Fisika 2 sebanyak 156 mahasiswa. Praktikum fisika dasar terdiri dari beberapa Program Studi dari Fakultas Teknik Universitas Mulawarman, yaitu Program Studi : Teknik Elektro, Teknik Lingkungan, Teknik Industri, Teknik Geologi, Teknik Pertambangan, Teknik Sipil dan Teknik Kimia.
9	Apakah jumlah data praktikan meningkat dari tahun ke tahun?	Jumlah praktikan cenderung berubah mengikuti jumlah mahasiswa yang mengambil mata kuliah, sehingga tidak menunjukkan peningkatan yang konsisten setiap tahun.
10	Apakah pernah terjadi duplikasi atau inkonsistensi data? Harap jabarkan.	Pernah terjadi duplikasi data saat pengisian form pendaftaran praktikum dan pernah ditemukan inkonsistensi data seperti kesalahan input NIM saat mengisi gform pendaftaran praktikum.
11	Seberapa besar pengaruh jumlah praktikan terhadap beban kerja pengelolaan data praktikum?	Jumlah praktikan berpengaruh signifikan terhadap beban kerja pengelolaan data praktikum. Semakin banyak praktikan, semakin banyak waktu dan tenaga yang dibutuhkan untuk pengecekan data, penilaian, dll.
12	Apakah data praktikum tahun sebelumnya mudah diakses kembali? Mengapa?	Data praktikum tahun 2024 relatif mudah diakses melalui Google Sites, meskipun arsip Google Drive tidak berisi data lengkap. Sedangkan untuk data tahun-tahun sebelum 2024, data dapat diakses melalui Google Drive.
13	Apakah terdapat perbedaan signifikan dalam pengelolaan data antara Fisika Dasar 1 dan Fisika Dasar 2?	Pada pengelolaan data antara Fisika Dasar 1 dan Fisika 2 tidak terdapat perbedaan yang signifikan.

14	Bagaimana proses penyimpanan dan pengarsipan data praktikum dari tahun ke tahun?	Data praktikum disimpan dan diarsipkan di Google Drive. Untuk Fisika Dasar 1, data diarsipkan berdasarkan tahun pelaksanaan praktikum. Sementara, Fisika 2 baru mulai dilaksanakan pada tahun 2025, sehingga data disimpan dalam folder tersendiri.
15	Apakah pernah terjadi kesulitan dalam mengakses data praktikum dari tahun sebelumnya?	Pernah terjadi kesulitan, terutama Ketika struktur <i>folder</i> yang kurang terstruktur dan penamaan <i>file</i> yang berbeda, sehingga membutuhkan waktu lebih untuk menemukan data yang diperlukan.
16	Jika satu hal yang harus diperbaiki dari pengelolaan data saat ini, apa itu?	Hal yang perlu diperbaiki adalah sistem pengelolaan data praktikum. Akan lebih efektif dan efisien jika menggunakan sistem terpusat berbasis web.

B. Alur Praktikum & Proses Kerja

No	Pertanyaan	Jawaban
1	Bagaimana alur pelaksanaan praktikum Fisika Dasar dari pendaftaran hingga nilai akhir?	Alur pelaksanaan dimulai dari pendaftaran praktikum melalui gform yang disebarkan melalui ketua tingkat masing-masing prodi, kemudian dilakukan pembagian kelompok dan jadwal praktikum masing-masing prodi, pelaksanaan praktikum (<i>pretest</i> , praktikum) pengumpulan laporan, penginputan nilai akhir praktikum.
2	Bagaimana proses pembagian kelas, kelompok, dan jadwal praktikum?	Proses pembagian kelompok dibagi berdasarkan daftar praktikan yang terdaftar.
3	Apakah pembagian tersebut dilakukan secara manual atau menggunakan sistem?	Pembagian kelompok dilakukan secara manual.
4	Bagaimana mekanisme presensi praktikan setiap pertemuan?	Mekanisme dilakukan dengan verifikasi langsung oleh asisten. Setiap praktikan membawa print QR CODE <i>personal</i> yang mengarah ke gform, kemudian asisten melakukan scan di awal atau saat praktikum berlangsung.
5	Presensi dicatat dan disimpan dalam bentuk apa?	Presensi disimpan dalam bentuk Google Sheet.

6	Apakah data presensi terhubung dengan data penilaian?	Presensi saat ini belum terhubung dengan data penilaian.
7	Bagaimana pelaksanaan pretest praktikum?	Pretest saat ini dilakukan secara <i>online</i> melalui gform.
8	Media apa yang digunakan untuk pretest?	Media yang digunakan untuk <i>pretest</i> adalah gform.
9	Apakah terdapat tugas, responsi, posttest atau penilaian lain selain pretest?	Tidak ada tugas lain.
10	Jika ada, bagaimana mekanisme pelaksanaannya dan media apa yang digunakan untuk penugasan dan responsi?	-
11	Bagaimana mekanisme pengumpulan dan penilaian tugas praktikan tersebut?	-
12	Apakah setiap komponen penilaian memiliki bobot tersendiri? Harap jabarkan.	Bobot penilaian praktikum, <i>pretest</i> (10%), kehadiran (10%), laporan praktikum (10%), kemampuan praktik (50%), asistensi (20%).
13	Selain pretest, posttest, tugas, dan/atau responsi, apakah ada penugasan lain yang memiliki kontribusi bobot terhadap nilai akhir? Apa itu?	-

C. Pengelolaan Nilai & Validasi

No	Pertanyaan	Jawaban
1	Bagaimana proses input nilai praktikum dilakukan saat ini?	Proses penginputan nilai praktikum dilakukan secara manual.
2	Siapa saja yang berwenang melakukan input dan pengelolaan nilai?	Pihak yang melakukan input nilai adalah asisten praktikum. Namun dalam pengelolaannya, Kepala Lab berperan mengawasi dan mengambil kebijakan akhir.
3	Apakah pernah terjadi kesalahan input nilai? Pada tahap apa?	Pernah. Kesalahan biasanya terjadi pada tahap pemindahan nilai dari lembar penilaian fisik (laporan/kemampuan) ke dalam

		spreadsheet, atau kesalahan pengetikan (<i>human error</i>) saat menginput nilai.
4	Bagaimana proses pengecekan dan validasi nilai sebelum rekap nilai akhir?	Dilakukan kroscek ulang oleh asisten antara nilai tercatat dengan spreadsheet. Selain itu, ada tahap "transparansi nilai" di mana praktikan diminta mengecek nilai mereka sebelum difinalisasi.
5	Bagaimana proses rekapitulasi nilai akhir praktikum dilakukan?	Rekapitulasi dilakukan menggunakan rumus (formula) pada spreadsheet yang menggabungkan berbagai komponen nilai sesuai dengan bobot yang telah ditentukan.
6	Media apa yang digunakan untuk rekap nilai akhir (misalnya spreadsheet, dokumen manual, atau lainnya)?	Media yang digunakan adalah Spreadsheet.
7	Bagaimana mekanisme koreksi atau komplain nilai dari praktikan?	Praktikan diberikan waktu sanggah (masa komplain) setelah nilai sementara diumumkan. Komplain dilakukan dengan menghubungi asisten terkait dan menunjukkan bukti (misal: laporan yang sudah dinilai) jika ada ketidaksesuaian.

D. Pencarian, Pengurutan & Permasalahan Data

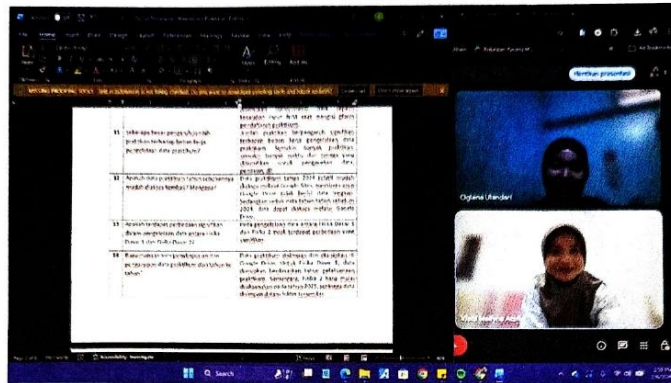
No	Pertanyaan	Jawaban
1	Bagaimana proses pencarian data praktikan dilakukan saat ini?	Proses pencarian data saat ini dilakukan menggunakan fitur <i>Find</i> (ctrl+F) pada spreadsheet atau melakukan pencarian satu per satu (<i>scrolling</i>) pada daftar nama/NIM yang tersedia.
2	Kendala apa yang sering dihadapi saat mencari data dalam jumlah besar?	Kendala utamanya adalah jika ada duplikasi data saat pendaftaran atau praktikan salah menginput NIM di awal, sehingga asisten harus melakukan validasi manual untuk memastikan data yang dicari benar.
3	Apakah pengurutan data praktikan sering dilakukan? Dalam konteks apa (misalnya berdasarkan NIM, nilai, atau kelas)?	Pengurutan dilakukan sekali di awal (saat pendaftaran dan <i>pretest</i>) untuk menyeragamkan daftar nama di semua sheet penilaian. Pengurutan dilakukan berdasarkan NIM agar memudahkan asisten saat menginput nilai secara berurutan.

4	Bagaimana proses pengurutan data dilakukan saat ini?	Pengurutan dilakukan di awal periode praktikum menggunakan fitur <i>sort</i> di spreadsheet. Setelah data urut, asisten tinggal mengisi kolom yang tersedia tanpa perlu melakukan pengurutan ulang setiap hari.
5	Data apa yang paling sering dicari atau diurutkan?	Data yang paling sering dicari adalah NIM dan Nama. Sedangkan data yang paling sering diurutkan adalah NIM.
6	Apa dampak jika pencarian dan pengurutan data berjalan lambat?	Menghambat proses rekapitulasi nilai, memperlama waktu pelayanan saat praktikan bertanya soal nilai, dan meningkatkan risiko kelelahan asisten yang berujung pada ketidakteelitian data.
7	Apakah kecepatan pencarian data merupakan kebutuhan penting? Mengapa?	Ya, kebutuhan penting. Karena asisten harus mengelola banyak kelas dalam waktu terbatas. Kecepatan pencarian data sangat membantu saat proses validasi kehadiran di laboratorium.

E. Kebutuhan Sistem & Fitur

No	Pertanyaan	Jawaban
1	Apakah sistem administrasi praktikum berbasis web diperlukan untuk membantu proses pengelolaan data?	Sangat diperlukan. Agar semua data (presensi, nilai, dan administrasi) terpusat dalam satu pintu dan tidak terpecah di banyak file Google Sheets.
2	Fitur apa saja yang paling dibutuhkan dalam sistem pengelolaan data praktikum agar sistem benar-benar bermanfaat?	Fitur presensi berbasis QR Code yang langsung masuk ke database nilai, fitur pretest berbasis gform yang langsung masuk ke database nilai, fitur validasi data otomatis untuk mencegah NIM ganda.
3	Bagian proses pendataan praktikum apa yang paling memakan waktu saat ini?	Proses sinkronisasi data antara formulir pendaftaran, presensi, dan rekapitulasi nilai akhir. Serta pengecekan manual untuk memastikan tidak ada nilai yang terlewat di- <i>input</i> .

Dokumentasi wawancara:



Samarinda, 6 Februari 2026

Narasumber

Ogiana Ulandari
NIM. 1809075013

Pewawancara

Vista Mellyna Atsfi
NIM. 2209106096

Mengetahui,
Kepala Lab. Layanan Dasar



Ir. Adi Pandu Wirawan, S.T., M.T
NIP. 199301112022031011

Lampiran 3 Surat balasan dari Lab. Layanan Dasar FT Unmul mengenai konfirmasi penerimaan survei dan penyerahan data penelitian

Samarinda, 10 Februari 2026

Nomor : 02/LAB-LD/2026
Perihal : Konfirmasi Penerimaan Penelitian dan Pengambilan Data
Lampiran : -

Kepada Yth.
Dekan Fakultas Teknik Universitas
Mulawarman
Di
Samarinda

Dengan hormat,

Sehubungan dengan surat pengantar penelitian dan permohonan permintaan data penelitian dari mahasiswa Program Studi Informatika Fakultas Teknik Universitas Mulawarman di bawah ini:

Nama : Vista Mellyna Atsfi
NIM : 2209106096

Dengan ini menyampaikan bahwa kami telah menerima pelaksanaan penelitian, serta akan memberikan data yang dibutuhkan guna menunjang penelitian skripsi mahasiswa yang bersangkutan. Adapun data yang akan diberikan meliputi:

1. Data Mahasiswa Praktikum Fisika Dasar 1 dan Fisika Dasar 2 Tahun 2025
2. Data Nilai dan Komponen Penilaian Mahasiswa Praktikum Fisika Dasar 1 dan Fisika Dasar 2 Tahun 2025
3. Data Presensi Mahasiswa Praktikum Fisika Dasar 1 dan Fisika Dasar 2 Tahun 2025
4. Data Nilai Akhir Mahasiswa Praktikum Fisika Dasar 1 dan Fisika Dasar 2 Tahun 2025
5. Data Asisten dan Penanggung Jawab/Koordinator Praktikum Fisika Dasar 1 dan Fisika Dasar 2 Tahun 2025
6. Data Bobot Penilaian Praktikum Fisika Dasar 1 dan Fisika Dasar 2

Data tersebut diberikan untuk keperluan penelitian akademik dengan ketentuan digunakan secara bertanggung jawab dan sesuai dengan etika penelitian yang berlaku. Demikian surat ini disampaikan. Atas perhatian dan kerja samanya diucapkan terima kasih.

Kepala Laboratorium Layanan Dasar
Fakultas Teknik Universitas
Mulawarman



Ir. Adi Pandu Wirawan, M.T., IPM.
NIP. 19930111 202203 1 011

FAKULTAS TEKNIK
UNIVERSITAS MULAWARMAN
Jl. Sambaliung No. 9 Kampus Gunung Kelua
Samarinda 75119
Telp./ Fax. : (0541)736834/749315